TiendaFlow: ERP System for Convenience-Store Chains

Project Definition (Sprint 0) and Requirements, Product Backlog, UML Models and Prototype (Sprint 1)

Juan Sebastian Rodriguez Blanco

Samuel Castellanos Rivera

Sergio Steeven Moreno Forero

Jonathan Alejandro Yacuma Rivera

Systems and Computer Engineering Program

Faculty of Engineering

University of Cundinamarca

CADI: Software Engineering II

Carlos Eduardo Mujica Reyes

October 1, 2026

Table of Contents

1. Evaluation of Advantages and Disadvantages....6
1.1 Justification of the Project's Validity....7
2. General Project Definition....8
2.1 Problem Definition....8
2.2 Proposed Solution....8
2.3 Justification....8
2.4 End Users....9
2.5 Usefulness and Return on Investment....9
3. Project Management: Scope, Time and Cost....10
3.1 Scope....10
3.2 Time....11
3.3 Team Roles and Rates....14
3.4 Costs....14
3.4.1 Personnel Costs....14
3.4.2 Software Licensing Costs....15
3.4.3 Other Costs and Total Project Cost....16
3.5 Commercial Pricing Model and Break-Even Point....17
4. Workspace....18
5. Background: Competitor Analysis....19
5.1 SAP Business One....19
5.2 Odoo....19
5.3 Square for Retail....20
5.4 Alegra....21
5.5 Comparative Table....21
5.6 Features Adopted and TiendaFlow Differentiators....22
6. Product Identity....23
7. Selected Technologies....24
8. Requirements Specification Framework....25
8.1 Specification Formats....25
8.2 Modules, CRUD Operations and Actors....26
8.3 System Functional Decomposition Tree....29
8.4 System Actors and General Use Case Diagram....31
8.5 System-Wide Non-Functional Requirements....33

8.6 Coverage of the ISO/IEC 25010 Categories....34
9. Module 1: Inventory Management....36
9.1 Module Description....36
9.2 Functional Decomposition Tree....36
9.3 Functional Requirements....38
9.4 Non-Functional Requirements....39
9.5 Use Case Diagram....40
9.6 Use Case Specification....42
9.7 Epic E-01 and User Stories....42
10. Module 2: Sales and Billing Management....45
10.1 Module Description....45
10.2 Functional Decomposition Tree....45
10.3 Functional Requirements....46
10.4 Non-Functional Requirements....47
10.5 Use Case Diagram....48
10.6 Use Case Specification....50
10.7 Epic E-02 and User Stories....51
11. Module 3: Supplier and Purchasing Management....53
11.1 Module Description....53
11.2 Functional Decomposition Tree....53
11.3 Functional Requirements....54
11.4 Non-Functional Requirements....55
11.5 Use Case Diagram....56
11.6 Use Case Specification....56
11.7 Epic E-03 and User Stories....57
12. Module 4: Human Resources Management....60
12.1 Module Description....60
12.2 Functional Decomposition Tree....60
12.3 Functional Requirements....61
12.4 Non-Functional Requirements....62
12.5 Use Case Diagram....63
12.6 Use Case Specification....64
12.7 Epic E-04 and User Stories....65
13. Module 5: Customer Management (CRM)....67

13.1 Module Description....67
13.2 Functional Decomposition Tree....67
13.3 Functional Requirements....68
13.4 Non-Functional Requirements....69
13.5 Use Case Diagram....70
13.6 Use Case Specification....72
13.7 Epic E-05 and User Stories....72
14. Module 6: Security and Reporting....75
14.1 Module Description....75
14.2 Functional Decomposition Tree....75
14.3 Functional Requirements....77
14.4 Non-Functional Requirements....78
14.5 Use Case Diagram....78
14.6 Use Case Specification....79
14.7 Epic E-06 and User Stories....80
15. Product Backlog....82
15.1 Epics and Release Plan....82
15.2 Definition of Done....82
16. Architecture and Data Models....83
16.1 Class Diagram of the Inventory Module....83
16.2 Entity-Relationship Diagram....85
16.3 C4 Context Diagram....87
16.4 C4 Container Diagram....88
16.5 C4 Component Diagram of the API....90
17. Mockups and Functional Prototype....92
17.1 Design System....92
17.2 Mockups....94
17.3 Interactive Prototype in Figma....97
17.4 Prototype Generated With Figma Make....101
18. Reflection Questions....103
18.1 Why Define Acceptance Criteria Before Developing a User Story?....103
18.2 How Does PlantUML Help Keep the Architecture Documentation Up to Date?....103
18.3 What Advantages Does Figma Make Offer for Rapid ERP Prototyping?....103
19. Documentation and Information Storage....104

References.....105
Appendix A: PlantUML Source Code of the Architecture and Data Models.....108
A.1 Class Diagram of the Inventory Module.....108
A.2 Entity-Relationship Diagram.....109
A.3 C4 Context Diagram.....113
A.4 C4 Container Diagram.....113
A.5 C4 Component Diagram.....114

TiendaFlow: ERP System for Convenience-Store Chains

This document defines TiendaFlow, a modular enterprise resource planning (ERP) system specialized in convenience-store chains, developed by the team as the semester project of the Software Engineering II course. It integrates the deliverables of the first two sprints, following the course guides (Mujica Reyes, 2026a, 2026b).

The Sprint 0 content presents the evaluation of the idea, the general definition of the project, its management plan in terms of scope, time and cost (including team roles, rates and software licensing), the analysis of competing products, the product identity and the technologies selected. The Sprint 1 content develops the functional and non-functional requirements in the format of the course guide with complete CRUD coverage, the Product Backlog organized in epics with its Definition of Done, the class, entity-relationship and C4 models, and the interface prototype built in Figma.

The requirements specification is organized **by module**. For each of the six modules of the system (Inventory, Sales and Billing, Supplier and Purchasing, Human Resources, Customer Management, and Security and Reporting), the document gathers in one place the module description, its functional decomposition tree, its functional and non-functional requirements, its use case diagram, the specification of its main use case, and its epic with the user stories. All diagrams were modeled in PlantUML.

1. Evaluation of Advantages and Disadvantages

Before defining the project scope, the team evaluated the feasibility of developing an ERP system for the convenience-store sector, a retail segment characterized by small-format stores, extended or 24/7 opening hours, high product turnover, perishable goods and a large number of low-value transactions per day. Table 1 compares the advantages and disadvantages

of the idea from four perspectives: technical aspects, relevance of the problem, value delivered to the end user, and viability.

Table 1

Advantages and Disadvantages of Developing an ERP for Convenience-Store Chains

Aspect	Advantages	Disadvantages
Technical	The Microsoft platform (ASP.NET Core, Blazor and SQL Server) is mature for transactional, multi-store web systems. A modular design allows incremental delivery, one module per sprint.	Integrating six modules and external services (DIAN electronic invoicing, payment gateways) is demanding for a 16-week semester. High concurrency requires careful design for performance and availability.
Relevance of the problem	Managing many distributed points of sale is a real challenge for convenience-store chains in Colombia. High turnover and perishable products make real-time inventory control critical.	There is limited access to real operational data from store chains; processes are modeled from public information and reasonable assumptions about the sector.
Value to the end user	The system reduces losses from stockouts, overstock and expired products, speeds up checkout, simplifies 24/7 staff scheduling and supports customer loyalty.	Store staff with low digital literacy require an extremely simple point-of-sale interface and short training.
Viability	Franchise and standard-store models allow replicable processes. A per-store subscription can recover the investment (Section 3.5). Market references exist (SAP Business One, Odoo, Square, Alegra).	Competing with established vendors requires clear differentiation. Microsoft licensing increases the initial investment (Section 3.4.2).

Note. Prepared by the authors.

1.1 Justification of the Project's Validity

Considering the technical aspects (the need for a distributed, multi-store system with real-time transaction processing), the relevance of the problem (efficient inventory, sales and staff management in convenience-store chains), and the value delivered to the end user (fewer losses, operational agility and a better shopping experience), the team considers it valid and appropriate to develop TiendaFlow as the project of this course. The disadvantages identified are mitigated through a clearly bounded scope (Section 3.1), a sprint plan (Section 3.2) and the prioritization of requirements with the MoSCoW technique.

2. General Project Definition

2.1 Problem Definition

Convenience-store chains operate under a high-transaction-volume model with multiple, geographically distributed points of sale that open for extended hours or 24 hours a day. In many small and medium chains, inventory, sales, purchasing, staff scheduling and customer information are handled in separate tools or spreadsheets. As a result, managers cannot see stock in real time, restocking depends on manual counts, perishable products expire on the shelves, shifts are planned by hand and there is no reliable information about customers. These problems cause losses from stockouts, overstock and expired products, long waiting times at the checkout and low customer loyalty.

2.2 Proposed Solution

TiendaFlow is a modular, web-based ERP specialized in convenience-store chains. It centralizes six integrated modules on a single platform: five business modules (Inventory, Sales and Billing, Supplier and Purchasing, Human Resources, and Customer Management (CRM)) and a cross-cutting Security and Reporting module, so that every store operates synchronized with the others and with central management. The expected product is a responsive web application built on the Microsoft platform, with a point-of-sale interface optimized for speed, native DIAN electronic invoicing and real-time consolidated reports.

2.3 Justification

TiendaFlow generates value by solving a real, high-impact operational problem of high-turnover retail businesses. Automating inventory, sales, purchasing and staff processes reduces losses, improves managerial decision-making through real-time information and standardizes processes across stores, which is especially valuable in franchise and standard-store models.

Unlike generic ERPs, TiendaFlow is preconfigured for the convenience-store operation, which lowers the implementation effort and cost for the customer.

2.4 End Users

The system is aimed at the following users:

1. **Store managers:** supervise inventory, sales, staff, approvals and customer requests of their store.
2. **Cashiers:** use the point of sale for daily transactions and register customers.
3. **Stock clerks:** receive merchandise, perform counts and register losses.
4. **Logistics coordinators:** manage suppliers, purchase orders and deliveries.
5. **HR officers:** manage employee records, contracts, terminations and the payroll pre-settlement.
6. **Corporate managers:** review consolidated reports and run loyalty campaigns for the whole chain.
7. **System administrators:** manage users, roles, system parameters and the audit log.
8. **Employees:** consult their shifts, request swaps and register their attendance.
9. **Suppliers:** consult and confirm purchase orders through a free web portal.
10. **End customers:** benefit from faster service, product availability and the loyalty program.

2.5 Usefulness and Return on Investment

For the customer chain, the return on investment comes from (a) fewer losses due to expired or out-of-stock products, (b) less operational time in checkout, shift planning and purchasing, (c) higher sales through the loyalty program and targeted promotions, and (d) lower training and support costs thanks to standardized processes across stores. For the development

team, Section 3.5 estimates the break-even point of the project under the proposed per-store subscription model.

3. Project Management: Scope, Time and Cost

The project is managed considering the triple constraint of scope, time and cost (Project Management Institute [PMI], 2021): any change in one of these dimensions affects the others. The work is organized with the Scrum framework (Schwaber & Sutherland, 2020) in short sprints, each ending with a reviewable increment.

3.1 Scope

The scope of the project is a functional minimum viable product (MVP) of TiendaFlow for one convenience-store model that can be replicated across multiple stores of a chain. The following modules are **included**:

1. Inventory Management (catalog, stock per store, receipts with batch and expiration, transfers, counts and reports).
2. Sales and Billing Management (POS, multiple payment methods, promotions, DIAN electronic invoicing, returns, voided sales and cash register control).
3. Supplier and Purchasing Management (suppliers, reorder suggestions, purchase orders, supplier portal and evaluation).
4. Human Resources Management (employees, contracts, 24/7 shifts, attendance and basic payroll pre-settlement).
5. Customer Management (CRM) (customers with data consent, loyalty points and tiers, promotions, purchase history and PQRS).
6. Security and Reporting (users, roles, two-factor authentication, audit log, system parameters and consolidated reports).

The following elements are **excluded** and remain as future work: full accounting and tax integration, the payment of the payroll and the transmission of the electronic payroll to DIAN, distribution logistics between central warehouses, e-commerce, native mobile applications (the web application is responsive) and advanced predictive business intelligence.

If the time constraint is at risk, the scope is adjusted according to the MoSCoW priority of each user story (Section 15): *Must* stories form the MVP and are delivered first, followed by *Should* stories and, finally, *Could* stories.

3.2 Time

As established for the course, the development time of the project corresponds to the duration of the current academic semester: **16 weeks**. Each member dedicates 12 hours per week (the 8 hours of guided work and 16 hours of autonomous work defined for each two-week laboratory guide), for a total of 192 hours per member and 768 hours for the team. Table 2 describes the sprints and the epics of the Product Backlog delivered in each one, and Figure 1 presents the schedule.

Table 2

Sprint Plan for the 16-Week Semester

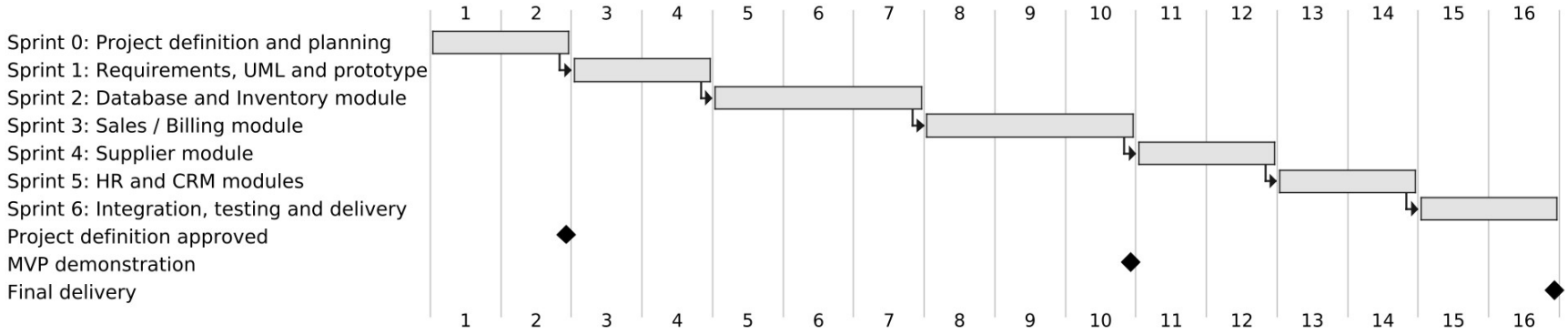
Sprint	Weeks	Objective	Main deliverables
0	1 - 2	Project definition and planning	Project definition document, repository and documentation page, presentation
1	3 - 4	Requirements, UML modeling and prototype	RF and RNF with CRUD coverage, Product Backlog and Definition of Done, class, entity-relationship and C4 models, Figma prototype
2	5 - 7	Security base and Inventory module	Users, roles and login (E-06); Inventory module (E-01)
3	8 - 10	Sales and Billing module	POS, payments, DIAN electronic invoicing (E-02); MVP demonstration
4	11 - 12	Supplier and Purchasing module	Purchase orders, supplier portal and evaluation (E-03)

Sprint	Weeks	Objective	Main deliverables
5	13 - 14	Human Resources and CRM modules	Shifts, attendance, payroll pre-settlement (E-04); loyalty program and PQRS (E-05)
6	15 - 16	Reporting, integration and delivery	Consolidated reports and audit log (E-06), integration and usability tests, deployment, final presentation

Note. Weeks are counted from the beginning of the academic semester. E-01 to E-06 are the epics of the Product Backlog.

Figure 1

Project Schedule (Gantt Chart) in Weeks of the Semester



Note. Diagram created by the authors in PlantUML. Diamonds indicate milestones.

3.3 Team Roles and Rates

Each member of the team assumes a main role, described in Table 3. The hourly rates were defined from the monthly salaries of software developers in Colombia in 2026, between COP 3,800,000 and 4,800,000 for junior profiles and between COP 5,500,000 and 7,500,000 for semi-senior profiles (Caneva, 2026), divided by approximately 180 working hours per month (42-hour work week). Leadership roles are placed in the semi-senior band and the remaining roles between the junior and semi-senior bands.

Table 3

Team Roles, Responsibilities and Hourly Rates

Member	**Role**	**Main responsibilities**	**Rate (COP/hour)**	**Monthly equivalent (COP)**
Sergio Steeven Moreno Forero	Project Manager and Scrum Master	Planning, sprint coordination, cost and time control, communication with the instructor, documentation	35,000	6,300,000
Samuel Castellanos Rivera	Software Architect and Backend Developer	Architecture, ASP.NET Core APIs, integration with DIAN and payment services, repository administration	38,000	6,840,000
Juan Sebastian Rodriguez Blanco	Frontend Developer and UI/UX Designer	Blazor interfaces, visual identity, prototype and usability of the POS and dashboards	30,000	5,400,000
Jonathan Alejandro Yacuma Rivera	Database Administrator and QA Tester	SQL Server data model, backups, test plans and quality assurance	30,000	5,400,000

Note. Monthly equivalent calculated with 180 hours per month. Rates are estimates for the purpose of the academic project.

3.4 Costs

3.4.1 Personnel Costs

Table 4 presents the personnel cost of the project, obtained by multiplying the 192 hours of each member by the corresponding rate.

Table 4

Personnel Cost for the Semester

Member	Role	Hours	Rate (COP/hour)	Cost (COP)
Sergio Steeven Moreno Forero	Project Manager and Scrum Master	192	35,000	6,720,000
Samuel Castellanos Rivera	Software Architect and Backend Developer	192	38,000	7,296,000
Juan Sebastian Rodriguez Blanco	Frontend Developer and UI/UX Designer	192	30,000	5,760,000
Jonathan Alejandro Yacuma Rivera	Database Administrator and QA Tester	192	30,000	5,760,000
Total		**768**		**25,536,000**

Note. Prepared by the authors.

3.4.2 Software Licensing Costs

TiendaFlow uses only Microsoft tools, platforms and technologies for the product, as required by the course. During the academic development, the team uses free editions (Table 5): ASP.NET Core and Blazor are free and open source, SQL Server Developer and Express editions are free (Microsoft, n.d.), and Visual Studio Community can be used for academic purposes.

Table 5

Development Environment Used During the Semester

Tool	Edition used in development	Cost (COP)
ASP.NET Core and Blazor (.NET)	Open-source framework	0
SQL Server 2022	Developer edition	0
Visual Studio	Community edition (academic use)	0
Visual Studio Code with PlantUML extension	Free	0
Git, GitHub and GitHub Pages	Free plan	0
Microsoft Teams and Microsoft 365	Institutional license of the university	0

Tool	Edition used in development	Cost (COP)
Figma and Figma Make; Google Colab	Free plans (tools prescribed by the Sprint 1 guide)	0

Note. Free editions are valid for development and academic purposes, not for commercial production.

To calculate the real cost of TiendaFlow as a commercial product, Table 6 presents the licenses required to develop it professionally and to run a pilot deployment on a central server for two stores with 10 named users. Prices in US dollars were converted at the representative market rate (TRM) of COP 3,109.30 per dollar of September 15, 2026 (Portafolio, 2026).

Table 6

Commercial Licensing Cost of the Microsoft Platform for the Pilot Deployment

Item	Licensing model	Qty.	Unit price (USD)	Total (USD)	Total (COP)	Source
Visual Studio Professional	Perpetual license per developer	4	499.99	1,999.96	6,218,476	TrustedTech (n.d.)
SQL Server 2022 Standard	Server license (Server + CAL model)	1	989.00	989.00	3,075,098	Microsoft (n.d.)
SQL Server 2022 Standard CAL	Client access license per user	10	230.00	2,300.00	7,151,390	Microsoft (n.d.)
Windows Server 2022 Standard	16-core license	1	526.99	526.99	1,638,570	TrustedTech (n.d.)
Windows Server 2022 CAL	Client access license per user	10	38.99	389.90	1,212,316	TrustedTech (n.d.)
Total				**6,205.85**	**19,295,850**	

Note. SQL Server can also be licensed per core (USD 3,945 per 2-core pack; minimum 4 cores, USD 7,890). With the Server + CAL model (USD 989 + USD 230 per user), the per-core model becomes cheaper beyond 30 users; therefore, it is recommended when the chain grows beyond the pilot.

3.4.3 Other Costs and Total Project Cost

Other costs include internet and electricity for the four members, estimated at COP 100,000 per member per month during four months (COP 1,600,000). The team works with its own computers and the university laboratory, so no additional hardware is purchased; the server

hardware of the pilot is assumed to be provided by the customer. A contingency reserve of 10% is added to cover risks. Table 7 consolidates the final cost of the project.

Table 7

Total Cost of the TiendaFlow Project

Concept	**Cost (COP)**	**Cost (USD)**
Personnel (Table 4)	25,536,000	8,212.78
Microsoft licensing for commercial development and pilot (Table 6)	19,295,850	6,205.85
Other costs (internet and electricity)	1,600,000	514.59
Subtotal	46,431,850	14,933.22
Contingency reserve (10%)	4,643,185	1,493.32
Total project cost	**51,075,035**	**16,426.54**

Note. USD values calculated with a TRM of COP 3,109.30. In the purely academic scenario, which uses only free editions, the cost would be COP 29,849,600 (personnel and other costs plus 10% contingency).

3.5 Commercial Pricing Model and Break-Even Point

TiendaFlow is offered as a monthly subscription of **COP 190,000 per store**, including up to five users per store and electronic invoicing, without per-transaction fees. Table 8 compares this price with the approximate monthly cost of the competitors analyzed in Section 5 for a store with five users.

Table 8

Approximate Monthly Cost for One Store With Five Users

Product	**Pricing basis**	**Approx. monthly cost (COP)**
SAP Business One	Starter cloud plan, USD 110 per user (Costbench, 2026)	1,710,115
Odoo	Standard plan, about USD 31.10 per user (ERP Research, 2026a)	483,496
Square for Retail	Plus plan, USD 49 per location plus 2.5% + USD 0.15 per in-person transaction (NerdWallet, 2026)	152,356 + fees
Alegra	Plus plan (up to 8 users), COP 319,900 (Becerra, 2026)	319,900

Product	Pricing basis	Approx. monthly cost (COP)
TiendaFlow	Per-store subscription, up to 5 users, no transaction fees	**190,000**

Note. USD prices converted with a TRM of COP 3,109.30. Competitor prices exclude implementation, hardware and taxes.

With a total cost of COP 51,075,035, the project reaches its break-even point after approximately 269 store-months of subscription (51,075,035 / 190,000); for example, 23 stores subscribed during 12 months. This estimate does not include the operating costs of hosting and support after the semester, which should be added in a later business plan.

4. Workspace

The project repository was created on GitHub, and all the members of the team were added as collaborators with write permissions. The documentation of the project is published on a page built with GitHub Pages, where the progress of each deliverable is updated. The PlantUML source files of all the diagrams of this document are stored in the repository so that they can be versioned together with the code.

1. **Repository:** https://github.com/samucr27/tiendaflow-erp
2. **Documentation page:** https://samucr27.github.io/tiendaflow-erp/
3. **Diagram sources:** PlantUML files (.puml) in the docs/uml folder of the repository.
4. **Google Colab notebook ERP_PlantUML_TiendaFlow (all the diagrams rendered with iplantuml):**
   https://colab.research.google.com/drive/1VtObCkwC1xnCBrrpza1oSu0oOpki-RqI?usp=sharing
5. **Figma design file (design system, mockups and interactive prototype):**
   https://www.figma.com/design/3LW57RjiNETxaukic2S6JE
6. **Prototype generated with Figma Make:** https://early-hope-36544320.figma.site/

5. Background: Competitor Analysis

Each member of the team researched at least one application similar to TiendaFlow: Juan Sebastian Rodriguez Blanco analyzed SAP Business One, Samuel Castellanos Rivera analyzed Odoo, Jonathan Alejandro Yacuma Rivera analyzed Square for Retail and Sergio Steeven Moreno Forero analyzed Alegra, a Colombian solution. For each product, the team studied its price, its strengths, the features that TiendaFlow can adopt and its weaknesses with respect to the convenience-store operation.

5.1 SAP Business One

SAP Business One is an ERP for small and medium enterprises licensed per named user, either as a cloud subscription or as a perpetual on-premise license. Its cloud plans range from USD 110 per user per month (Starter) to USD 219 (Professional, single tenant) (Costbench, 2026). A perpetual Professional license costs about USD 3,200 to 3,600 per user, implementation for a small business ranges from USD 10,000 to more than USD 25,000, and annual maintenance represents 18% to 22% of the license cost (Hyperbots, 2026).

Its strengths are the integration of inventory and warehouse management, sales orders and CRM, and its reporting based on Crystal Reports; its weaknesses include a very basic HR module that usually requires a third-party add-on and an interface that feels dated compared to cloud-native ERPs (ERP Research, 2026b). **TiendaFlow adopts** its role-based access and approval workflows. **TiendaFlow improves** on it with a much lower cost, basic HR included, and a configuration ready for convenience stores that does not require an implementation partner.

5.2 Odoo

Odoo is a modular, open-source-based ERP with more than 40 applications. Its One App Free plan allows one application with unlimited users; the Standard plan costs about USD 24.90

per user per month on promotion and about USD 31.10 at renewal, and the Custom plan about USD 49 to 61; implementation costs range from USD 5,000 to USD 200,000 (ERP Research, 2026a). Its strengths are its modern interface, its large community marketplace and the possibility of activating only the applications needed; its weaknesses are that advanced features require the paid edition, its basic reporting and the dependency on partners for enterprise support.

Because Odoo charges per internal user while portal users are free (Swell, 2026), costs grow with the number of cashiers, a relevant issue for stores with three daily shifts. **TiendaFlow adopts** the modular activation of functions and the idea of a free external portal (applied to suppliers). **TiendaFlow improves** on it with per-store pricing, which does not penalize 24/7 stores with many employees, and functions specific to convenience stores.

5.3 Square for Retail

Square is a point-of-sale platform with a Free plan, a Plus plan at USD 49 per location per month and a Premium plan at USD 149; processing fees range from 2.4% to 2.6% plus USD 0.15 per in-person transaction, and hardware goes from USD 59 for a card reader to USD 1,189 for a register kit (NerdWallet, 2026). The Plus plan adds advanced inventory tools, team management and sales reports (UpMenu, 2026).

Its strengths are the simplicity and speed of its POS and its per-location pricing; its weaknesses are that processing fees become the largest expense as card sales grow (UpMenu, 2026), that it does not cover supplier management or payroll as an ERP does, and that it is not offered in Colombia, so it does not support DIAN electronic invoicing. **TiendaFlow adopts** its per-location pricing model and the simplicity of its POS. **TiendaFlow improves** on it with no transaction fees, DIAN compliance, and supplier and HR modules.

5.4 Alegra

Alegra is a Colombian cloud platform that integrates electronic invoicing, accounting, inventory, POS and payroll in one account and is authorized by DIAN. Its plans in Colombia are Emprendedor (COP 74,900 per month, 1 user), Pyme (COP 163,900, 2 users and 2 warehouses), Pro (COP 250,900, 3 users and 10 warehouses) and Plus (COP 319,900, 8 users and 100 warehouses); its POS issues the POS electronic equivalent document, and its payroll adjusts to the 42-hour work week (Becerra, 2026).

Its strengths are its regulatory compliance, its transparent prices in COP and a free user for the accountant; its weakness for this segment is its accounting orientation and its limited number of users per plan, while functions such as FEFO expiration control, 24/7 shift planning, loyalty tiers and a supplier portal are not part of the plans reviewed. **TiendaFlow adopts** native DIAN compliance (including the POS equivalent document), published prices in COP and a free read-only access for an external accountant. **TiendaFlow improves** on it with operational functions specific to convenience stores.

5.5 Comparative Table

Table 9 summarizes the cost and the modules offered by each application, following the format proposed in the laboratory guide, and includes TiendaFlow for reference.

Table 9

Comparison of Similar Applications by Cost and Module

Application (web address)	Cost	Supplier module	HR module	Inventory / warehouse module	Billing / sales module	CRM / customers module
SAP Business One (sap.com)	USD 110 - 219 per user per month (cloud); USD 3,200 - 3,600 per user (perpetual)	Yes	Partial (basic; add-on)	Yes	Yes	Yes
Odoo (odoo.com)	Free (one app); about USD 24.90 - 61 per user per month	Yes	Yes	Yes	Yes	Yes
Square for Retail (squareup.com)	USD 0 - 149 per location per month plus processing fees	Partial	Partial (team management)	Yes	Yes (not in Colombia)	Partial
Alegra (alegra.com)	COP 74,900 - 319,900 per month	Partial	Yes (payroll)	Yes	Yes (DIAN)	Partial
TiendaFlow (proposed)	COP 190,000 per store per month	Yes (with portal)	Yes (basic, 24/7 shifts)	Yes (FEFO)	Yes (DIAN, offline POS)	Yes (loyalty, PQRS)

Note. Prices from Costbench (2026), Hyperbots (2026), ERP Research (2026a), NerdWallet (2026) and Becerra (2026). Partial indicates limited coverage or coverage through add-ons, based on the vendor information reviewed by the team.

5.6 Features Adopted and TiendaFlow Differentiators

Table 10 lists the features that TiendaFlow adopts from its competitors and how they are adapted to convenience stores, and Table 11 summarizes the differentiators of TiendaFlow (what it offers that the reviewed products do not offer together at a comparable price).

Table 10

Features Adopted From Competitors

Competitor	Feature adopted	Adaptation in TiendaFlow
SAP Business One	Role-based access and approval workflows	Roles per store (cashier, clerk, manager); approvals for purchase orders, returns and stock adjustments
Odoo	Modular activation and free portal users	Modules activated per store; free supplier portal

Competitor	Feature adopted	Adaptation in TiendaFlow
Square for Retail	Per-location pricing and simple POS	Per-store subscription; POS designed for fast checkout
Alegra	Native DIAN compliance and prices in COP	Electronic invoice and POS equivalent document; transparent prices in COP; free read-only accountant user

Note. Prepared by the authors.

Table 11

TiendaFlow Differentiators

Differentiator	Description	Gap addressed
Per-store pricing without transaction fees	Fixed monthly price per store with up to five users	Per-user costs (SAP, Odoo) and per-transaction fees (Square)
Preconfigured for convenience stores	Ready-made catalog structure, 24/7 shifts and fast POS	Implementation costs of generic ERPs (USD 5,000 - 200,000)
FEFO expiration control	Batch tracking and alerts for perishable products	Losses from expired products on the shelves
Offline POS	Cash sales continue without internet and synchronize later	Sales lost during connectivity failures
Basic HR for 24/7 operation	Rotating shifts, swaps, attendance and surcharges	Basic or absent HR functions in SAP and Square
Complete CRM	Loyalty points and tiers, segmented promotions and PQRS	Partial CRM in Square and Alegra
Supplier portal and evaluation	Suppliers confirm orders; performance ranking	Purchasing managed by phone or email

Note. Prepared by the authors from the competitor analysis.

6. Product Identity

The ERP developed by the team is named **TiendaFlow**, a name that reflects the continuous, real-time flow of operations (inventory, sales and staff coordination) across a chain of convenience stores. Its visual identity uses a red-orange tone (#E8412C), which represents the energy and fast pace of retail operations; a charcoal gray (#2B2B2B), which conveys reliability and technological solidity; and a light neutral background (#F5F5F5) for clarity and readability of the interfaces.

The logo, shown in Figure 2, combines a shelf icon with a pulse line that symbolizes the real-time movement of data between stores. This palette and logo are applied consistently to all the screens of the system (RNF-GEN-09) and to the prototype of Section 17.

Figure 2

TiendaFlow Logo



Note. Designed by the authors.

7. Selected Technologies

In accordance with the course guidelines, the product is built only with Microsoft tools, platforms and technologies. Table 12 lists the technologies selected and their use in the project.

Table 12

Technologies Selected for TiendaFlow

Category	**Technology**	**Use in TiendaFlow**
Backend	ASP.NET Core (C#) Web API and Worker Service	REST APIs of the six modules, business logic, background processing of DIAN documents, alerts and notifications
Frontend	Blazor (WebAssembly for the POS; Server for the back office and supplier portal)	Offline-capable POS, administrative panel and supplier portal
Database	SQL Server 2022 with Entity Framework Core	Transactional data of all modules, migrations, backups and encryption
IDE	Visual Studio	Development, debugging and testing

Category	Technology	Use in TiendaFlow
Testing	xUnit with coverlet; Postman; Accessibility Insights	Unit and integration tests, coverage and accessibility audits
Modeling	Visual Studio Code with PlantUML	Functional trees, use case, class, entity-relationship, C4 and schedule diagrams
Server operating system	Windows Server 2022	Hosting of the application and database in the pilot
Version control, CI and documentation	Git, GitHub, GitHub Actions and GitHub Pages	Source code, pipelines, diagrams and project documentation page
Collaboration	Microsoft Teams and Microsoft 365	Meetings, communication and shared documents
Methodology	Scrum	Sprint planning, reviews and retrospectives
Prototyping and diagram rendering	Figma and Figma Make; Google Colab with iplantuml	Mockups and interactive prototype; rendering of the PlantUML diagrams

Note. Prepared by the authors. GitHub is a Microsoft subsidiary. Figma and Google Colab are used only because the Sprint 1 guide prescribes them for prototyping and diagramming; they are not part of the product.

8. Requirements Specification Framework

8.1 Specification Formats

The requirements of each module are specified with the formats of the Sprint 1 guide (Mujica Reyes, 2026b), described in Table 13. Functional requirements follow the IEEE 830 structure (IEEE, 1998): a unique identifier, a statement written with the verb "shall", the CRUD operation it covers, the actor and a verifiable acceptance criterion. Non-functional requirements are classified according to the quality characteristics of ISO/IEC 25010 (ISO/IEC, 2011) and always include a verification metric. User stories follow the format proposed by Cohn (2004) and are grouped in one epic per module, with acceptance criteria written as Given-When-Then scenarios, as recommended in requirements engineering practice (Sommerville, 2011).

Priorities use the MoSCoW technique (Must, Should, Could, Won't), and user stories are estimated in story points with Planning Poker on the Fibonacci scale (1, 2, 3, 5, 8, 13). In Sprint 0, the functional requirements were written as IEEE 830 cards; in Sprint 1 they were rewritten in

the tabular format of the course guide, adding the CRUD operation and the acceptance criterion of each requirement, and the identifiers were aligned with that guide (RF, RNF and HU).

Table 13

Formats Used for the Requirements Specification

Artifact	**Identifier**	**Fields**	**Reference**
Functional requirement	RF-<module>-<nnn>	Identifier, requirement ("The system shall..."), CRUD operation, actor, acceptance criterion, MoSCoW priority	IEEE (1998); Mujica Reyes (2026b)
Non-functional requirement	RNF-<module or GEN>-<nn>	Identifier, ISO/IEC 25010 category, requirement, verification metric	ISO/IEC (2011); Mujica Reyes (2026b)
Use case specification	UC-<module>-<nn>	Actors, description, preconditions, main flow, alternative flows, postconditions, traceability	Kruchten (1995); Sommerville (2011)
User story	HU-<module>-<nn> within epic E-<nn>	"As a <role>, I want <goal>, so that <benefit>"; related RF; MoSCoW priority; story points; acceptance criteria	Cohn (2004); Mujica Reyes (2026b)

Note. Module codes: INV = Inventory, VEN = Sales and Billing, SUP = Supplier and Purchasing, RH = Human Resources, CRM = Customer Management, SEG = Security and Reporting; GEN = system-wide.

8.2 Modules, CRUD Operations and Actors

Table 14 summarizes the six modules of TiendaFlow, the four CRUD operations (create, read, update and delete) covered by each one, the additional functions beyond CRUD, and their main and secondary actors. Deletions are logical whenever the history must be preserved (products, suppliers, employees, users), and customer data is anonymized to comply with Law 1581 of 2012.

Table 14

Modules of TiendaFlow With Their CRUD Operations and Actors

Module	**CRUD operations**	**Additional functions**	**Main actors**	**Secondary actors**
Inventory Management	C: register product, receipt, shrinkage R: consult stock per store U: update prices, levels and location D: deactivate obsolete product	Minimum stock and FEFO alerts; transfers; physical counts; valuation report	Store Manager, Stock Clerk	Corporate Manager
Sales and Billing Management	C: open register, register sale, issue invoice R: consult sales history U: update invoice status, close register D: void sale with credit note	Split payments; promotions; offline sales; daily reports	Cashier, Store Manager	Customer, Corporate Manager, DIAN, Payment Gateway
Supplier and Purchasing Management	C: register supplier, create purchase order R: consult orders and purchase history U: update supplier data, approve and update orders D: deactivate supplier, cancel order	Reorder suggestions; supplier portal; supplier ranking	Logistics Coordinator	Store Manager, Supplier
Human Resources Management	C: register employee, register attendance R: consult employees and shifts U: update contract, plan shifts D: terminate employee (deactivation)	Shift swaps; payroll pre-settlement; final settlement report	HR Officer, Store Manager	Employee
Customer Management (CRM)	C: register customer, create campaign R: consult purchase history U: update customer data and consent D: anonymize customer data	Points accumulation and redemption; tiers; PQRS; recommendations	Cashier, Corporate Manager	Customer, Store Manager, Notification Service
Security and Reporting	C: create user R: consult users, audit log and reports U: update roles and parameters D: deactivate user	Two-factor authentication; consolidated dashboard	System Administrator	Corporate Manager, all users

Note. C = create, R = read, U = update, D = delete. Each operation is specified as a functional requirement in Sections 9 to 14.

The Sprint 1 guide describes a generic ERP with six modules: purchasing, inventory, suppliers, human resources, CRM, and billing and sales. Table 15 shows how they correspond to the modules of TiendaFlow. Purchasing and suppliers are combined in one module because, in a convenience-store chain, the same logistics coordinator manages both and the purchase order is the main transaction with a supplier; the module still covers the complete CRUD of both entities. The sixth module of TiendaFlow, Security and Reporting, groups the administration functions that all the other modules share.

Table 15

Correspondence Between the Modules of the Guide and the Modules of TiendaFlow

Module of the guide	**Module of TiendaFlow**	**Section**	**Coverage**
Purchasing (Compras)	Supplier and Purchasing Management	11	Create, consult, approve, update the status of and cancel purchase orders; reorder suggestions
Inventory (Inventarios)	Inventory Management	9	Products, stock per store (each store is its warehouse), receipts, transfers, counts and losses
Suppliers (Proveedores)	Supplier and Purchasing Management	11	Register, update and deactivate suppliers; purchase history, performance evaluation and ranking
Human resources (Talento Humano)	Human Resources Management	12	Employees, contracts, terminations, 24/7 shifts, attendance and payroll pre-settlement
CRM (Clientes)	Customer Management (CRM)	13	Customers with data consent, purchase history, segmentation, loyalty program, campaigns and PQRS
Billing and sales (Facturación y Ventas)	Sales and Billing Management	10	POS sales, DIAN electronic invoices, document status, voided sales with credit notes and sales reports
(not in the guide)	Security and Reporting	14	Users, roles, second factor, audit log, system parameters and consolidated reports

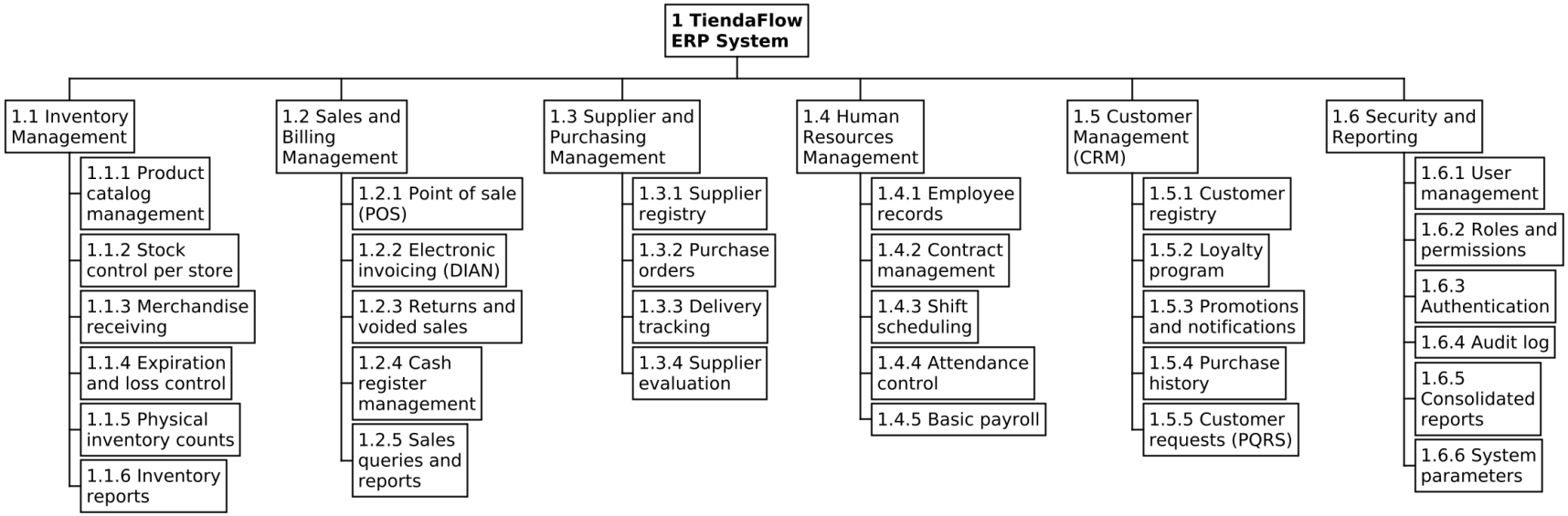
Note. Prepared by the authors from the Sprint 1 guide (Mujica Reyes, 2026b).

8.3 System Functional Decomposition Tree

Figure 3 presents the functional decomposition tree of the whole system, drawn as a work breakdown structure (WBS) in PlantUML, with its first three levels: the system (level 1), the six modules (level 2) and their 31 functions (level 3). The fourth level (concrete operations) is detailed in the tree of each module in Sections 9 to 14.

Figure 3

Functional Decomposition Tree of TiendaFlow (Levels 1 to 3)



Note. Diagram created by the authors in PlantUML as a work breakdown structure (@startwbs), as proposed in the Sprint 1 guide.

8.4 System Actors and General Use Case Diagram

Table 16 describes the actors that interact with TiendaFlow, and Figure 4 presents the general use case diagram, which includes the six modules and their actors. The detailed use case diagram of each module is presented in its own section.

Table 16

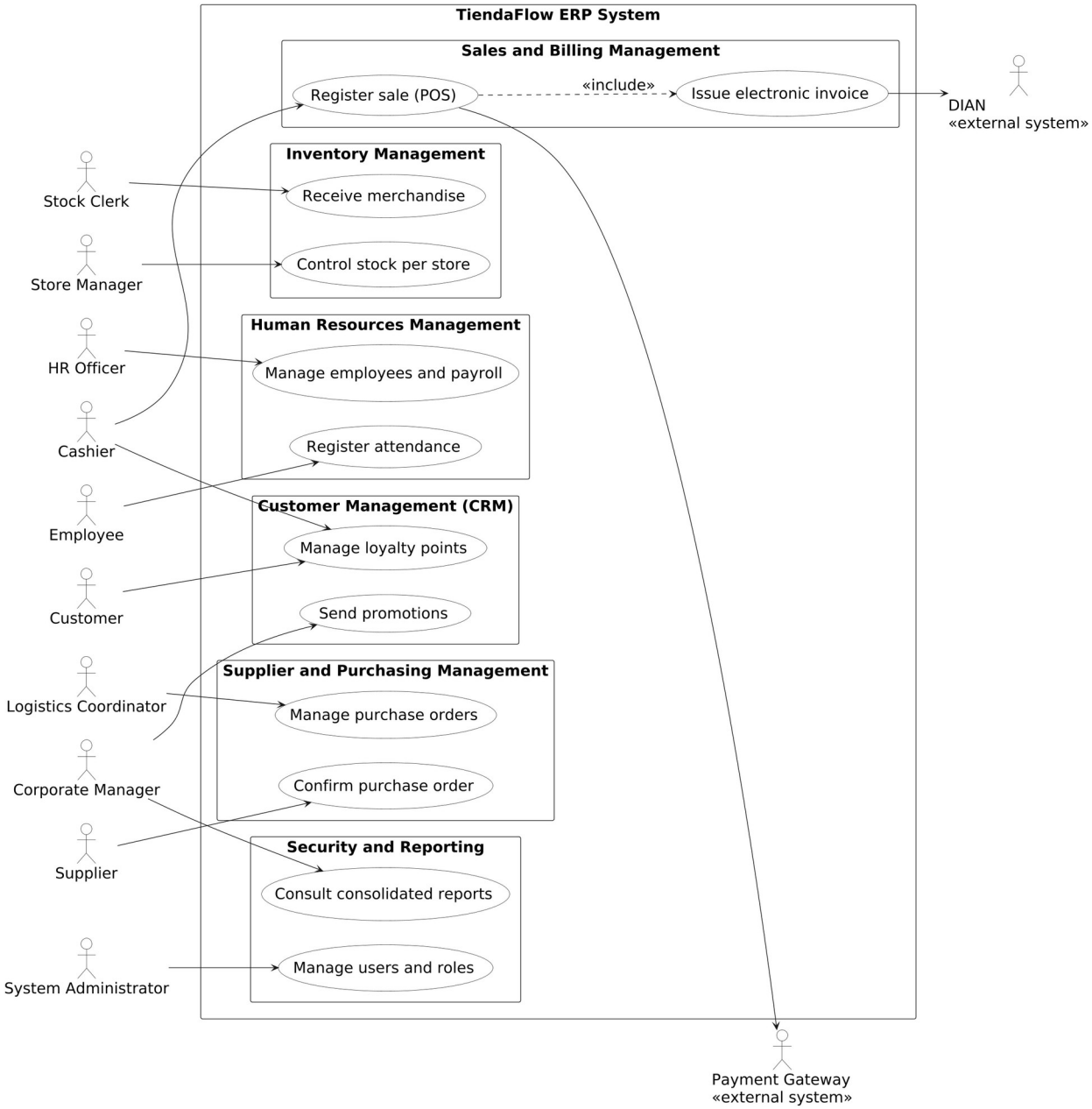
Actors of the TiendaFlow System

Actor	**Type**	**Description**	**Modules**
Cashier	Internal user	Operates the POS, registers sales, returns and customers	Sales, CRM, HR
Stock Clerk	Internal user	Receives merchandise, performs counts and registers losses	Inventory
Store Manager	Internal user	Supervises the store; approves adjustments, returns, orders and shift swaps	All business modules
Logistics Coordinator	Internal user	Manages suppliers, purchase orders and deliveries	Supplier and Purchasing
HR Officer	Internal user	Manages employee records, contracts, terminations and payroll pre-settlement	HR
Corporate Manager	Internal user	Consults consolidated reports and runs loyalty campaigns	Security and Reporting, Sales, Inventory, CRM
Employee	Internal user	Any staff member; consults shifts and registers attendance	HR
System Administrator	Internal user	Manages users, roles, system parameters and the audit log	Security and Reporting
Supplier	External user	Consults and confirms purchase orders in the portal	Supplier and Purchasing
Customer	External user	Accumulates and redeems points; files PQRS; manages its data	CRM
DIAN	External system	Validates electronic invoices and credit notes	Sales
Payment Gateway	External system	Authorizes card and digital wallet payments	Sales
Notification Service	External system	Sends emails and SMS messages	CRM, Supplier and Purchasing

Note. Prepared by the authors.

Figure 4

General Use Case Diagram of TiendaFlow



Note. Diagram created by the authors in PlantUML. Each package groups representative use cases of one module.

8.5 System-Wide Non-Functional Requirements

Table 17 specifies the non-functional requirements that apply to the whole system, in the format of the Sprint 1 guide. The requirements that apply only to one module are specified in the section of that module.

Table 17

System-Wide Non-Functional Requirements

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-GEN-01	Reliability (availability)	The system shall operate 24/7, since stores open continuously or for extended hours; planned maintenance only between 2:00 and 4:00 a.m. with prior notice.	Uptime monitoring (Azure Monitor / Application Insights): 99.5% or more per month.
RNF-GEN-02	Reliability (recoverability)	The database shall be backed up automatically (daily full backup and transaction log backups every 15 minutes).	Quarterly restore drill: recovery point of 15 minutes or less and recovery time of 4 hours or less.
RNF-GEN-03	Performance efficiency (time behavior)	Operations of the web application shall respond quickly under normal load.	Load test: 95% of operations in 2 seconds or less; reports in 10 seconds or less.
RNF-GEN-04	Performance efficiency (capacity)	The architecture shall support the growth of the chain without redesign.	Load test with 20 stores, 100 concurrent POS and 300 concurrent back-office users without degrading RNF-GEN-03.
RNF-GEN-05	Security (confidentiality)	All communications shall be encrypted with TLS 1.2 or higher, and sensitive data (personal data, salaries, payments) shall be encrypted at rest with AES-256.	Configuration inspection and security scan: zero unencrypted endpoints or sensitive columns.
RNF-GEN-06	Security (authenticity)	Access shall be controlled by roles, with salted password hashes, session timeout after 15 minutes of inactivity and account lockout after 5 failed attempts.	Access tests per role: no unauthorized operation succeeds.
RNF-GEN-07	Usability (learnability and operability)	A new cashier shall be able to complete a sale after 30 minutes of training.	Usability test with 5 users: at least 4 complete a sale unassisted; System Usability Scale score of 70 or more.

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-GEN-08	Usability (accessibility)	The interface shall comply with the Web Content Accessibility Guidelines (WCAG) 2.1, level AA (World Wide Web Consortium [W3C], 2018).	Accessibility audit with Accessibility Insights: zero critical issues.
RNF-GEN-09	Usability (user interface aesthetics)	All screens shall use the TiendaFlow palette, logo and typography defined in the design system.	Design review: 100% of screens compliant.
RNF-GEN-10	Compatibility (interoperability)	The system shall integrate with DIAN electronic invoicing, payment gateways and notification services through documented REST APIs (JSON, OpenAPI).	Integration tests in the DIAN habilitation environment and with the gateway sandbox pass.
RNF-GEN-11	Maintainability (modularity and testability)	The code shall follow a layered architecture separated by module, reviewed through pull requests in GitHub.	xUnit and coverlet report: unit test coverage of 80% or more of the business logic.
RNF-GEN-12	Maintainability (analysability)	Every error shall be logged with a correlation identifier to facilitate diagnosis.	100% of unhandled exceptions logged in Application Insights with correlation identifier.
RNF-GEN-13	Portability (adaptability)	The web application shall run on current browsers and adapt to POS screens and tablets.	Tests on the two latest versions of Edge, Chrome and Firefox; responsive from 1024 px.
RNF-GEN-14	Portability (installability)	A new store shall be added without interrupting the service of the others (multi-store configuration).	A test store is configured in 4 hours or less with no downtime and no degradation of RNF-GEN-03.
RNF-GEN-15	Functional suitability (completeness)	Every module shall provide the four CRUD operations on its main entities, with logical deletion where history must be preserved.	CRUD matrix (Table 14) complete and acceptance tests passed for every operation.

Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

8.6 Coverage of the ISO/IEC 25010 Categories

The Sprint 1 guide asks for at least two non-functional requirements per ISO/IEC 25010 category. Table 18 shows that the 32 non-functional requirements of TiendaFlow cover the eight categories of the standard with at least two requirements each.

Table 18

Non-Functional Requirements per ISO/IEC 25010 Category

ISO/IEC 25010 category	Requirements	Total
Functional suitability	RNF-GEN-15, RNF-INV-03, RNF-VEN-03	3
Performance efficiency	RNF-GEN-03, RNF-GEN-04, RNF-INV-01, RNF-VEN-01, RNF-SUP-02, RNF-CRM-02	6
Compatibility	RNF-GEN-10, RNF-VEN-05	2
Usability	RNF-GEN-07, RNF-GEN-08, RNF-GEN-09, RNF-RH-03	4
Reliability	RNF-GEN-01, RNF-GEN-02, RNF-VEN-02	3
Security	RNF-GEN-05, RNF-GEN-06, RNF-INV-02, RNF-VEN-04, RNF-SUP-01, RNF-RH-01, RNF-CRM-01, RNF-SEG-01, RNF-SEG-02	9
Maintainability	RNF-GEN-11, RNF-GEN-12, RNF-RH-02	3
Portability	RNF-GEN-13, RNF-GEN-14	2

Note. Includes the system-wide requirements (Section 8.5) and the module requirements (Sections 9 to 14).

9. Module 1: Inventory Management

9.1 Module Description

The Inventory Management module controls the stock of every store of the chain in real time. Each store acts as the warehouse of its own merchandise. The module maintains the product catalog, registers merchandise receipts with their batch number and expiration date, generates alerts for minimum stock and upcoming expirations, supports transfers between stores and physical counts, and produces inventory reports for decision-making.

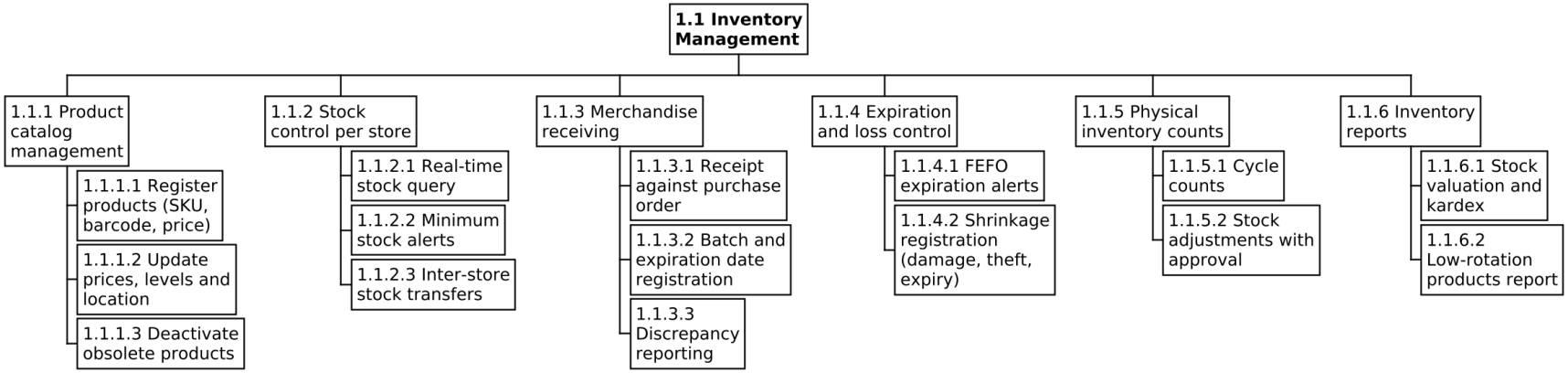
The module is fed by the Sales and Billing module, since every sale or return changes the available stock, and by the Supplier and Purchasing module, since every received purchase order increases it. Its minimum stock alerts are, in turn, the input of the automatic reorder suggestions of the Supplier and Purchasing module.

9.2 Functional Decomposition Tree

Figure 5 presents the functional decomposition of the module. The module (level 2 of the system tree) is broken down into six functions (level 3) and fifteen concrete operations (level 4).

Figure 5

Functional Decomposition Tree of the Inventory Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

9.3 Functional Requirements

Table 19 specifies the 11 functional requirements of the Inventory module. The module covers the four CRUD operations (create, read, update and delete).

Table 19

Functional Requirements of the Inventory Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-INV-001	The system shall allow registering products with SKU, barcode, name, category, unit of measure, cost, sale price and tax rate.	C	Store Manager	SKU and barcode must be unique; a duplicate is rejected with a clear message.	Must
RF-INV-002	The system shall allow consulting the current stock by product and store, updated with every sale, return, receipt, transfer and adjustment.	R	Store Manager, Stock Clerk	The query shows available quantity, location, batches and date of the last movement; a sale is reflected in 5 seconds or less.	Must
RF-INV-003	The system shall allow updating product data (prices, category, minimum and maximum stock, location in the store).	U	Store Manager	The change is recorded in the audit log and is applied immediately in the POS and in the stock reports.	Must
RF-INV-004	The system shall allow deactivating obsolete products (logical deletion).	D	Store Manager	A deactivated product cannot be sold or ordered; its movement history is preserved.	Should
RF-INV-005	The system shall allow registering the receipt of merchandise against an approved purchase order, recording the batch and expiration date of each item.	C	Stock Clerk	Stock increases by the received quantity; the expiration date is mandatory for perishables; differences with the order are recorded.	Must
RF-INV-006	The system shall generate alerts when a product reaches its minimum stock in a store.	R	Store Manager	The alert appears on the dashboard and is emailed within 1 minute of the movement; it feeds the reorder suggestion (RF-SUP-004).	Must

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-INV-007	The system shall alert, in FEFO (first expired, first out) order, about batches that expire within a configurable number of days.	R	Stock Clerk	Batches are listed in FEFO order with the number of days to expiration.	Should
RF-INV-008	The system shall allow registering shrinkage due to damage, theft or expiration.	C	Stock Clerk	Stock decreases; loss type, quantity, reason and user are recorded.	Should
RF-INV-009	The system shall allow transferring stock between stores of the chain.	C, U	Store Manager	Outgoing and incoming movements are recorded; stock changes in both stores only after the destination confirms the reception.	Should
RF-INV-010	The system shall allow recording physical counts and applying stock adjustments after approval.	U	Stock Clerk, Store Manager	Differences are reported; the adjustment is applied only after the approval of the store manager.	Should
RF-INV-011	The system shall generate inventory reports (stock valuation, kardex and low-rotation products) exportable to PDF and Excel.	R	Store Manager, Corporate Manager	Reports can be filtered by store and period and exported in both formats.	Could

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

9.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 20 specifies the non-functional requirements of the Inventory module.

Table 20

Non-Functional Requirements of the Inventory Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-INV-01	Performance efficiency (time behavior)	Stock queries shall respond quickly and reflect movements in near real time in all the stores.	Load test: 95th percentile below 1 second with 50 stores; movements visible in 5 seconds or less.
RNF-INV-02	Security (accountability)	Every stock movement shall record the user, date and time, type and reason; movement records cannot be edited or deleted.	100% of movements with an audit record; records retained for at least 5 years.

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-INV-03	Functional suitability (correctness)	Stock shall remain consistent: it can never be negative and every movement is transactional.	Concurrency tests: zero discrepancies between movements and stock.

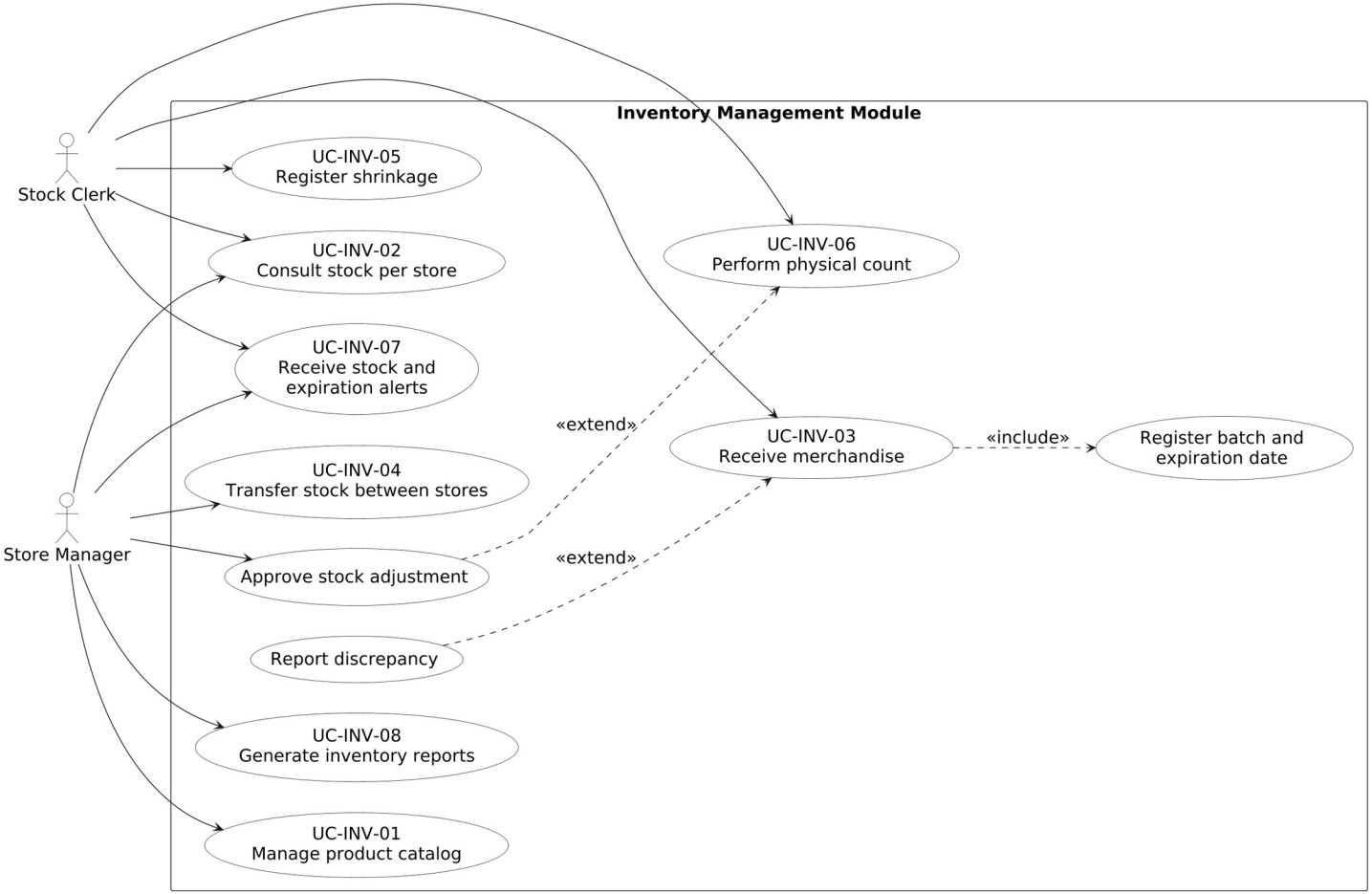
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

9.5 Use Case Diagram

Figure 6 presents the use case diagram of the Inventory module, with its actors and the include and extend relationships between use cases.

Figure 6

Use Case Diagram of the Inventory Module



Note. Diagram created by the authors in PlantUML.

9.6 Use Case Specification

Table 21 specifies UC-INV-03, the main use case of the module.

Table 21

Specification of Use Case UC-INV-03: Receive merchandise

UC-INV-03: Receive merchandise	
Actors	Primary: Stock Clerk. Secondary: Supplier and Purchasing module (purchase order).
Description	The stock clerk registers the merchandise delivered by a supplier against an approved purchase order, including batch and expiration data.
Preconditions	The stock clerk is authenticated. An approved purchase order exists for the store.
Main flow	1. The stock clerk selects Receive merchandise and scans or types the purchase order number. 2. The system displays the ordered items and quantities. 3. The stock clerk scans each received item and enters the received quantity. 4. The system requests the batch number and expiration date of each item (include: Register batch and expiration date). 5. The stock clerk confirms the receipt. 6. The system increases the store stock, changes the purchase order status to Received and generates the receipt record.
Alternative flows	A1 (step 3). The received quantity differs from the ordered quantity: the system flags the discrepancy, the clerk records the reason (extend: Report discrepancy) and the order is marked as Partially received. A2 (step 4). The expiration date is shorter than the minimum shelf life configured for the product: the system warns the clerk, who may reject the item.
Postconditions	Stock updated; batches registered with their expiration dates; delivery data available for supplier evaluation (RF-SUP-010).
Traceability	RF-INV-002, RF-INV-005; HU-INV-03

Note. Prepared by the authors.

9.7 Epic E-01 and User Stories

The module corresponds to epic **E-01: Inventory Management** of the Product Backlog. Table 22 presents its 11 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 51 story points, 29 of them in *Must* stories.

Table 22

User Stories of Epic E-01: Inventory Management

ID	User story	RF	Priority	SP	Acceptance criteria
HU-INV-01	As a store manager, I want to register products with their barcode and prices, so that they can be sold in every store.	RF-INV-001	Must	5	Given a new product with a unique SKU and barcode, when I save it, then it is available in the POS of all stores; a duplicate SKU is rejected.
HU-INV-02	As a store manager, I want to consult the stock of my store in real time, so that I can make restocking decisions.	RF-INV-002	Must	8	Given a completed sale, when I consult the stock, then the quantity already reflects the sale, with location and last movement.
HU-INV-03	As a stock clerk, I want to register the batch and expiration date of each product I receive, so that perishables can be controlled.	RF-INV-005	Must	8	Given an approved purchase order, when I confirm the receipt, then each item is stored with its batch and expiration date and the stock increases.
HU-INV-04	As a store manager, I want to update prices and stock levels, so that the catalog reflects current conditions.	RF-INV-003	Must	3	Given a product, when I change its price, then the POS uses the new price in the next sale and the change appears in the audit log.
HU-INV-05	As a store manager, I want to deactivate obsolete products, so that the catalog stays clean.	RF-INV-004	Should	3	Given a deactivated product, when a cashier scans it, then it cannot be sold and its history remains available.
HU-INV-06	As a store manager, I want to receive an alert when a product reaches its minimum stock, so that I can reorder it before it runs out.	RF-INV-006	Must	5	Given a minimum level of 10 units, when the stock drops to 10, then an alert appears on my dashboard and is emailed within 1 minute.
HU-INV-07	As a stock clerk, I want to be notified about products close to their expiration date, so that they are sold first or withdrawn.	RF-INV-007	Should	3	Given batches expiring within the configured days, when I open the alerts panel, then they are listed in FEFO order.
HU-INV-08	As a stock clerk, I want to register damaged, stolen or expired products, so that the stock reflects real losses.	RF-INV-008	Should	3	Given a damaged product, when I register the loss with its reason, then the stock decreases and the loss appears in the shrinkage report.
HU-INV-09	As a store manager, I want to transfer stock to another store of the chain, so that I can avoid stockouts without placing new orders.	RF-INV-009	Should	5	Given enough stock in my store, when the destination store confirms the reception, then the stock decreases in my store and increases in the destination.

ID	User story	RF	Priority	SP	Acceptance criteria
HU-INV-10	As a store manager, I want stock adjustments from physical counts to require my approval, so that inventory losses are controlled.	RF-INV-010	Should	5	Given a count with differences, when the clerk submits it, then the stock is adjusted only after my approval.
HU-INV-11	As a corporate manager, I want valuation and low-rotation reports, so that I can reduce dead stock.	RF-INV-011	Could	3	Given a selected period, when I generate the report, then products without sales are listed and the report can be exported to Excel.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

10. Module 2: Sales and Billing Management

10.1 Module Description

The Sales and Billing Management module supports the daily operation of the checkout counters. It registers sales through the point of sale (POS), processes multiple payment methods, applies promotions and loyalty point redemptions, and issues electronic invoices and POS electronic equivalent documents in accordance with the regulations of the Colombian tax authority (Dirección de Impuestos y Aduanas Nacionales [DIAN], 2023).

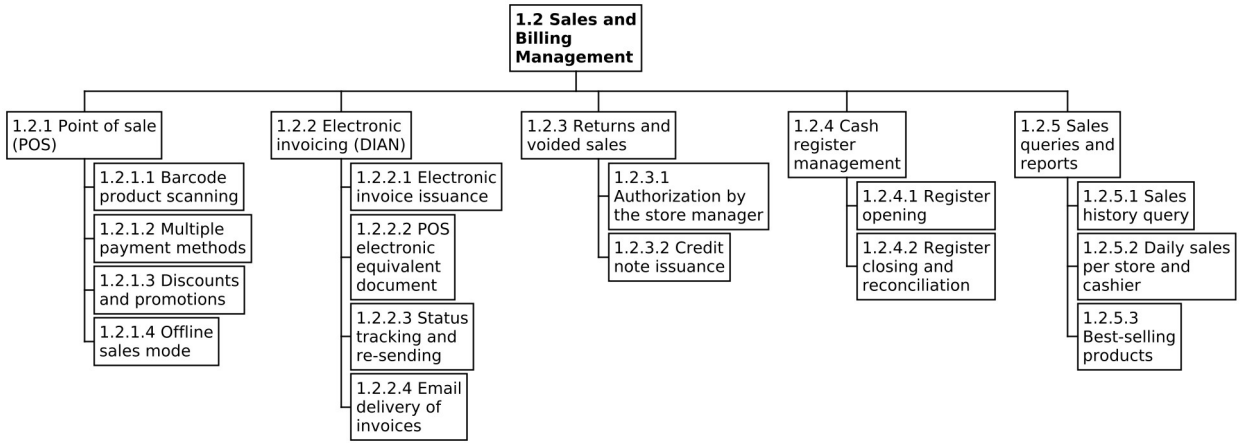
It also tracks the DIAN status of each electronic document, manages returns, voided sales and credit notes, and controls the opening, closing and reconciliation of each cash register. Every sale updates the stock (Inventory module) and the customer's points (CRM module), which makes this module the operational core of TiendaFlow.

10.2 Functional Decomposition Tree

Figure 7 presents the functional decomposition of the module into five functions (level 3) and fifteen operations (level 4).

Figure 7

Functional Decomposition Tree of the Sales and Billing Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

10.3 Functional Requirements

Table 23 specifies the 10 functional requirements of the Sales and Billing module. The module covers the four CRUD operations (create, read, update and delete).

Table 23

Functional Requirements of the Sales and Billing Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-VEN-001	The system shall allow opening a cash register with the initial cash base, the cashier and the shift.	C	Cashier	A sale cannot be registered on a register that has not been opened.	Must
RF-VEN-002	The system shall allow registering sales at the POS by scanning barcodes or searching products.	C	Cashier	The total with taxes (IVA) is calculated automatically; stock decreases when the sale is confirmed.	Must
RF-VEN-003	The system shall accept cash, card and digital wallet payments, including split payments within one sale.	C	Cashier; Payment Gateway	The sum of the payments equals the total; the change is calculated; card data is never stored.	Must
RF-VEN-004	The system shall automatically apply active promotions and loyalty point redemptions during the sale.	U	Cashier	Discounts are shown on the receipt and on the electronic invoice.	Should

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-VEN-005	The system shall issue the electronic invoice or the POS electronic equivalent document of each sale and send it to DIAN.	C	Cashier; DIAN	The document includes the fields required by DIAN, its status is stored, and the PDF and XML are emailed to the customer.	Must
RF-VEN-006	The system shall allow consulting the sales history by store, date range, cashier and payment method.	R	Store Manager	Results show date, cashier, payment method and total, and can be exported.	Should
RF-VEN-007	The system shall update the status of each electronic document (issued, accepted, rejected) and allow re-sending rejected documents.	U	Cashier, Store Manager	Status changes are reflected in real time; a rejected document can be corrected and re-sent.	Must
RF-VEN-008	The system shall allow voiding a sale or processing a return with the authorization of the store manager.	D	Store Manager; DIAN	An electronic credit note is issued, the stock is restored when the product can be sold again and the action is logged.	Should
RF-VEN-009	The system shall close and reconcile each cash register at the end of the shift.	U	Cashier	Expected and counted amounts per payment method are compared and differences are recorded.	Must
RF-VEN-010	The system shall generate daily sales reports per store, cashier and product, including best-selling products.	R	Store Manager, Corporate Manager	Reports are available for the current day and any period and can be exported.	Should

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

10.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 24 specifies the non-functional requirements of the Sales and Billing module.

Table 24

Non-Functional Requirements of the Sales and Billing Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-VEN-01	Performance efficiency (time behavior)	Registering an item and completing a payment at the POS shall not generate queues at the checkout.	Load test: 95th percentile of 2 seconds or less with up to 5 registers operating simultaneously per store.
RNF-VEN-02	Reliability (fault tolerance)	If the internet connection is lost, the POS shall keep registering cash sales locally and synchronize them automatically when connectivity returns.	Simulated 8-hour outage: zero sales lost after resynchronization.
RNF-VEN-03	Functional suitability (correctness)	Electronic documents and taxes shall comply with the DIAN technical annex in force (DIAN, 2023).	100% of documents validated in the DIAN habilitation environment; IVA test cases (19%, 5% and exempt) pass.
RNF-VEN-04	Security (confidentiality)	TiendaFlow shall never store card data; payments are processed through a PCI DSS-certified gateway using tokenization.	Security test: zero card numbers stored in the database.
RNF-VEN-05	Compatibility (co-existence)	The POS shall work with standard barcode readers, thermal receipt printers and cash drawers.	Tests with at least two models of each device connected through USB or Bluetooth.

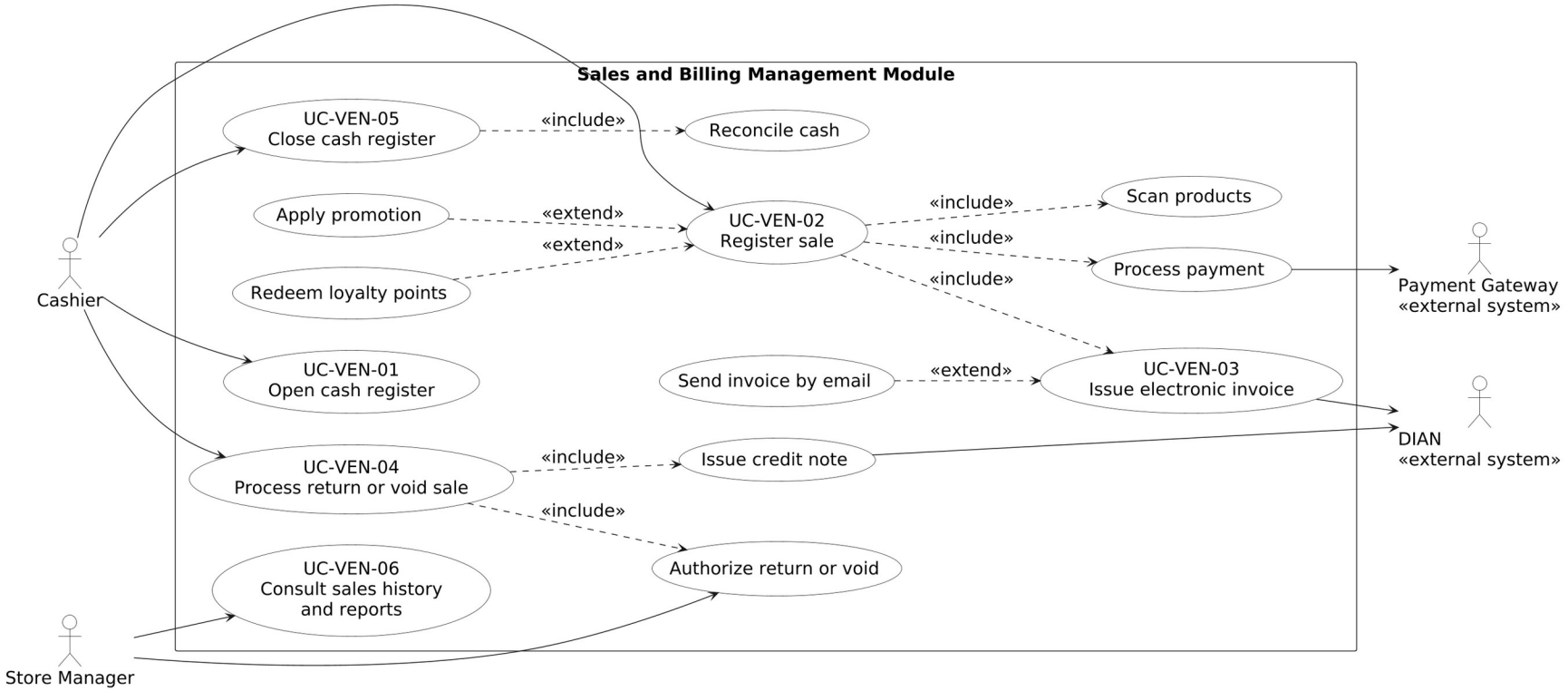
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

10.5 Use Case Diagram

Figure 8 presents the use case diagram of the Sales and Billing module, with its actors and the include and extend relationships between use cases.

Figure 8

Use Case Diagram of the Sales and Billing Module



Note. Diagram created by the authors in PlantUML.

10.6 Use Case Specification

Table 25 specifies UC-VEN-02, the main use case of the module.

Table 25

Specification of Use Case UC-VEN-02: Register sale

UC-VEN-02: Register sale	
Actors	Primary: Cashier. Secondary: Payment Gateway and DIAN (external systems).
Description	The cashier registers the products purchased by a customer, receives the payment and issues the electronic document of the sale.
Preconditions	The cashier is authenticated and has opened the cash register (UC-VEN-01).
Main flow	1. The cashier starts a new sale. 2. The cashier scans the products (include: Scan products); the system shows description, unit price, taxes and running total. 3. The system automatically applies active promotions (extend: Apply promotion). 4. The cashier asks for the customer's identification; if the customer is registered, the system shows the points balance (extend: Redeem loyalty points). 5. The cashier selects the payment method or methods and enters the amounts (include: Process payment). 6. For card or wallet payments, the system sends the request to the Payment Gateway and receives the approval. 7. The system registers the sale, decreases the stock, accumulates loyalty points and issues the electronic document sent to DIAN (include: Issue electronic invoice). 8. The system prints or emails the receipt and prepares the POS for the next sale.
Alternative flows	A1 (step 2). The barcode is not recognized: the cashier searches the product by name; if it does not exist in the catalog, the sale continues without it. A2 (step 6). The payment is declined: the system notifies the cashier, who selects another payment method. A3 (step 7). The DIAN service is unavailable: the document is stored in a contingency queue and transmitted automatically when the service returns. A4 (any step). The internet connection is lost: the POS switches to offline mode for cash sales (RNF-VEN-02).
Postconditions	Sale recorded; stock updated; electronic document issued or queued; loyalty points accumulated.
Traceability	RF-VEN-002, RF-VEN-003, RF-VEN-004, RF-VEN-005; HU-VEN-02, HU-VEN-03

Note. Prepared by the authors.

10.7 Epic E-02 and User Stories

The module corresponds to epic **E-02: Sales and Billing** of the Product Backlog. Table 26 presents its 10 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 55 story points, 39 of them in *Must* stories.

Table 26

User Stories of Epic E-02: Sales and Billing

ID	User story	RF	Priority	SP	Acceptance criteria
HU-VEN-01	As a cashier, I want to open my register with an initial cash base, so that the sales of my shift are controlled.	RF-VEN-001	Must	2	Given a closed register, when I enter the initial base, then the register opens and sales are enabled.
HU-VEN-02	As a cashier, I want to scan products with a barcode reader, so that I can register sales quickly and without typing errors.	RF-VEN-002	Must	8	Given an open register, when I scan a barcode, then the product, its price and the total appear in 2 seconds or less, and stock decreases when the sale is confirmed.
HU-VEN-03	As a cashier, I want to accept cash, card and digital wallet payments, even combined, so that customers can pay as they prefer.	RF-VEN-003	Must	8	Given a sale total, when I split the payment into cash and card, then the system validates that the sum equals the total and calculates the change.
HU-VEN-04	As a store manager, I want active promotions to be applied automatically at checkout, so that customers receive the advertised discounts.	RF-VEN-004	Should	5	Given an active two-for-one promotion, when two eligible items are scanned, then the discount is applied and shown on the receipt.
HU-VEN-05	As a customer, I want to receive my electronic invoice by email, so that I have a valid proof of purchase.	RF-VEN-005	Must	13	Given a completed sale with my email, when DIAN validates the document, then I receive the PDF and XML within 1 minute.
HU-VEN-06	As a store manager, I want to consult the sales history by date, cashier and payment method, so that I can follow the performance of my store.	RF-VEN-006	Should	3	Given a date range, when I apply the filters, then I see each sale with cashier, payment method and total.
HU-VEN-07	As a cashier, I want to see the DIAN status of each invoice and re-send rejected ones, so that every sale is legally supported.	RF-VEN-007	Must	5	Given a rejected invoice, when I correct it and re-send it, then its status changes to accepted once DIAN validates it.

ID	User story	RF	Priority	SP	Acceptance criteria
HU-VEN-08	As a store manager, I want returns and voided sales to require my authorization, so that fraud is prevented.	RF-VEN-008	Should	5	Given a return request, when the cashier submits it, then it cannot be completed without my credentials, and a credit note is issued.
HU-VEN-09	As a cashier, I want to close my register with an automatic reconciliation, so that cash differences are detected immediately.	RF-VEN-009	Must	3	Given the end of my shift, when I enter the counted cash, then I see expected and counted amounts per payment method and the difference is recorded.
HU-VEN-10	As a corporate manager, I want daily sales reports per store and product, so that I can compare the performance of the stores.	RF-VEN-010	Should	3	Given a selected date, when I open the report, then I see totals per store, cashier and product, and the best sellers.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

11. Module 3: Supplier and Purchasing Management

11.1 Module Description

The Supplier and Purchasing Management module organizes the relationship between the chain and its suppliers and covers the purchasing process. It keeps the supplier registry and the agreed price lists, suggests reorder quantities based on stock levels and sales, and manages the creation, approval, sending and cancellation of purchase orders.

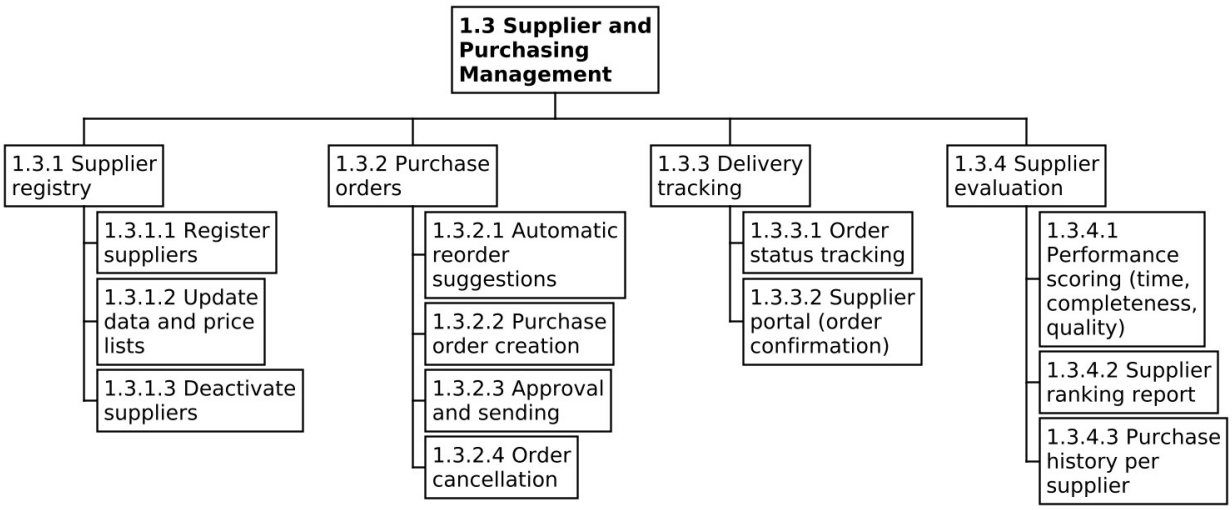
Suppliers use a free web portal to consult, confirm and schedule the delivery of their orders, and the module evaluates each supplier's performance (punctuality, completeness and quality) with the data captured when merchandise is received in the Inventory module.

11.2 Functional Decomposition Tree

Figure 9 presents the functional decomposition of the module into four functions (level 3) and twelve operations (level 4).

Figure 9

Functional Decomposition Tree of the Supplier and Purchasing Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

11.3 Functional Requirements

Table 27 specifies the 10 functional requirements of the Supplier and Purchasing module. The module covers the four CRUD operations (create, read, update and delete).

Table 27

Functional Requirements of the Supplier and Purchasing Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-SUP-001	The system shall allow registering suppliers with tax and commercial data (NIT, business name, address, contacts, products supplied, delivery days and payment terms).	C	Logistics Coordinator	NIT, business name, address and contact are mandatory; the NIT must be unique.	Must
RF-SUP-002	The system shall allow updating supplier data and their agreed price lists.	U	Logistics Coordinator	Changes are logged; new prices apply only to new purchase orders.	Should
RF-SUP-003	The system shall allow deactivating suppliers.	D	Logistics Coordinator	A deactivated supplier cannot be selected in new purchase orders; its history is preserved.	Should
RF-SUP-004	The system shall suggest reorder quantities per product and supplier based on minimum and maximum levels (RF-INV-006) and the average daily sales of the last 30 days.	R	Logistics Coordinator	Suggested quantities restore the maximum level of each product, grouped by supplier.	Should
RF-SUP-005	The system shall allow creating purchase orders manually or from a reorder suggestion.	C	Logistics Coordinator	The order includes supplier, products, quantities, agreed unit cost, destination store and expected delivery date.	Must
RF-SUP-006	The system shall require the approval of the store manager for orders above a configurable amount and shall send approved orders to the supplier by email (PDF) and through the portal.	U	Store Manager	Orders above the threshold remain Pending approval until they are approved or rejected with a reason.	Must

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-SUP-007	The system shall allow consulting purchase orders and updating their status (pending, approved, sent, confirmed, received, partially received, cancelled); suppliers confirm their orders in the portal.	R, U	Logistics Coordinator, Supplier	Every status change is recorded in the audit log; each supplier sees only its own orders.	Should
RF-SUP-008	The system shall allow cancelling a purchase order with a justification.	D	Logistics Coordinator	Only orders not yet received can be cancelled; the supplier is notified and the cancellation is logged.	Should
RF-SUP-009	The system shall allow consulting the purchase history of each supplier.	R	Logistics Coordinator	The history shows orders, amounts, dates and delivery results.	Could
RF-SUP-010	The system shall evaluate supplier performance (punctuality, completeness and quality) and produce a ranking.	C, R	Logistics Coordinator, Store Manager	The score is calculated from the receipts (RF-INV-005) and the ranking is generated automatically.	Could

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

11.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 28 specifies the non-functional requirements of the Supplier and Purchasing module.

Table 28

Non-Functional Requirements of the Supplier and Purchasing Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-SUP-01	Security (confidentiality)	Each supplier shall access only its own purchase orders and data in the portal.	Access-control tests: 100% of attempts to access other suppliers' orders rejected.
RNF-SUP-02	Performance efficiency (time behavior)	Approved purchase orders shall reach the supplier without manual intervention.	Order emailed and published in the portal within 1 minute after approval.

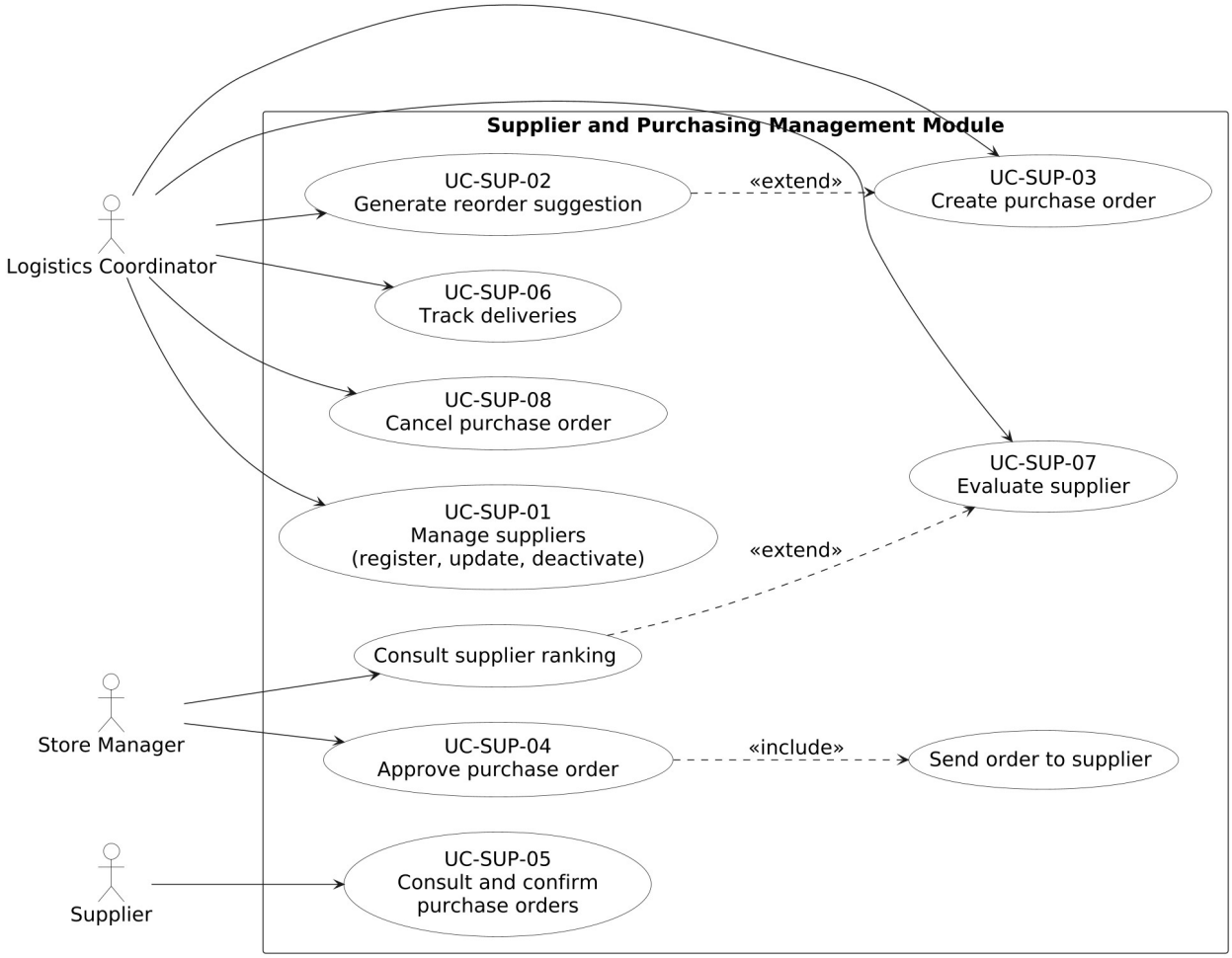
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

11.5 Use Case Diagram

Figure 10 presents the use case diagram of the Supplier and Purchasing module, with its actors and the include and extend relationships between use cases.

Figure 10

Use Case Diagram of the Supplier and Purchasing Module



Note. Diagram created by the authors in PlantUML.

11.6 Use Case Specification

Table 29 specifies UC-SUP-03, the main use case of the module.

Table 29

Specification of Use Case UC-SUP-03: Create purchase order

UC-SUP-03: Create purchase order	
Actors	Primary: Logistics Coordinator. Secondary: Store Manager (approval) and Supplier (receiver).
Description	The logistics coordinator creates a purchase order for a supplier, which is approved when required and sent to the supplier.
Preconditions	The coordinator is authenticated. The supplier is active and has an agreed price list.
Main flow	1. The coordinator selects New purchase order and chooses the supplier and the destination store. 2. The system displays the supplier's products with their agreed prices and the current stock of the store. 3. The coordinator enters products and quantities, or loads them from a reorder suggestion (extend from UC-SUP-02). 4. The system calculates subtotal, taxes and total, and validates the supplier's minimum order quantities. 5. The coordinator confirms the order. 6. If the total exceeds the approval threshold, the system sets the status Pending approval and notifies the store manager (UC-SUP-04); otherwise, the order is approved automatically. 7. Once approved, the system sends the order to the supplier by email and publishes it in the portal (include: Send order to supplier); the status changes to Sent.
Alternative flows	A1 (step 4). A quantity is below the supplier's minimum: the system warns the coordinator and prevents the confirmation until it is corrected. A2 (step 6). The store manager rejects the order: the status changes to Rejected with a reason, and the coordinator may edit and resubmit it.
Postconditions	Purchase order registered and sent to the supplier; available for merchandise receipt (UC-INV-03).
Traceability	RF-SUP-005, RF-SUP-006; HU-SUP-05, HU-SUP-06

Note. Prepared by the authors.

11.7 Epic E-03 and User Stories

The module corresponds to epic **E-03: Supplier and Purchasing** of the Product Backlog. Table 30 presents its 10 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 44 story points, 16 of them in *Must* stories.

Table 30

User Stories of Epic E-03: Supplier and Purchasing

ID	User story	RF	Priority	SP	Acceptance criteria
HU-SUP-01	As a logistics coordinator, I want to register suppliers, so that I have an updated directory.	RF-SUP-001	Must	5	Given a new supplier, when I save it without NIT or contact, then the system rejects it; with complete data, it is available for orders.
HU-SUP-02	As a logistics coordinator, I want to update supplier data and price lists, so that orders use the agreed prices.	RF-SUP-002	Should	3	Given a new price list, when I save it, then new orders use the new prices and existing orders keep theirs.
HU-SUP-03	As a logistics coordinator, I want to deactivate suppliers, so that I stop ordering from unreliable ones.	RF-SUP-003	Should	2	Given a deactivated supplier, when I create an order, then it does not appear in the list of suppliers.
HU-SUP-04	As a logistics coordinator, I want the system to suggest reorder quantities, so that I avoid stockouts and overstock.	RF-SUP-004	Should	8	Given products at or below their minimum level, when I open the suggestions, then I see quantities per supplier that restore the maximum level.
HU-SUP-05	As a logistics coordinator, I want to create purchase orders with the agreed prices, so that the stores are restocked.	RF-SUP-005	Must	8	Given a supplier with a price list, when I add a product, then its agreed price is loaded and the total is calculated.
HU-SUP-06	As a store manager, I want to approve purchase orders above a defined amount, so that spending is controlled.	RF-SUP-006	Must	3	Given an order above the threshold, when the coordinator confirms it, then it remains Pending approval until I approve or reject it.
HU-SUP-07	As a supplier, I want to consult and confirm the purchase orders assigned to me, so that I can schedule my deliveries.	RF-SUP-007	Should	5	Given an order sent to me, when I confirm it in the portal with a delivery date, then its status changes to Confirmed.
HU-SUP-08	As a logistics coordinator, I want to cancel a purchase order with a justification, so that I can correct errors.	RF-SUP-008	Should	2	Given an order not yet received, when I cancel it with a reason, then the supplier is notified and the action is logged.
HU-SUP-09	As a logistics coordinator, I want to consult the purchase history of a supplier, so that I can negotiate better conditions.	RF-SUP-009	Could	3	Given a supplier, when I open its history, then I see orders, amounts, dates and delivery results.

ID	User story	RF	Priority	SP	Acceptance criteria
HU-SUP-10	As a logistics coordinator, I want to see a performance score for each supplier, so that I can choose the most reliable ones.	RF-SUP-010	Could	5	Given receipts in a period, when I open the ranking, then suppliers are ordered by punctuality, completeness and quality.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

12. Module 4: Human Resources Management

12.1 Module Description

The Human Resources Management module covers the personnel management needed by stores that operate for extended hours or 24 hours a day: employee records, contract updates and terminations (managed by the HR officer), rotating shift planning and shift swaps (managed by the store manager), clock-in and clock-out attendance control, and the calculation of a basic payroll pre-settlement (worked hours, overtime, and night, Sunday and holiday surcharges).

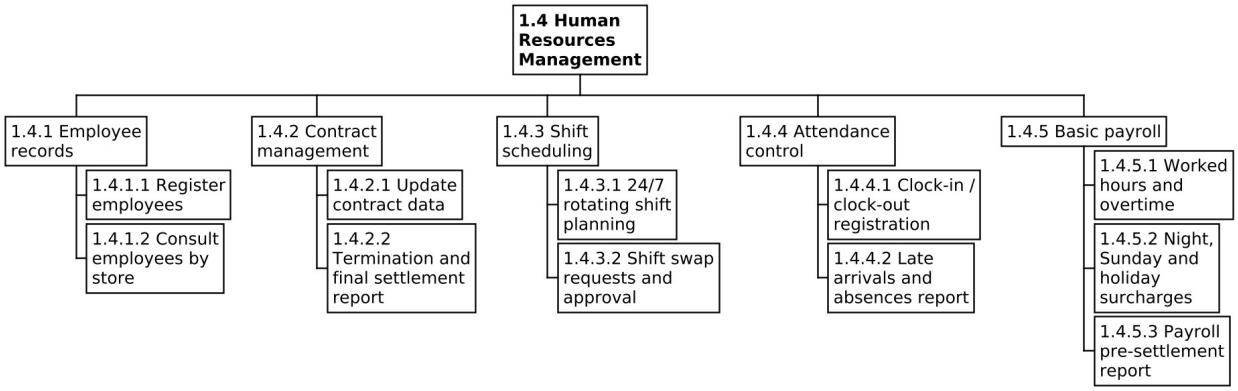
Because Colombian labor rules change over time (for example, the progressive reduction of the maximum working week to 42 hours established by Law 2101 of 2021, cited in Becerra, 2026), legal parameters are configurable. The payment of the payroll and the transmission of the electronic payroll to DIAN are outside the scope of this project; the module produces reports that the accountant uses for those processes.

12.2 Functional Decomposition Tree

Figure 11 presents the functional decomposition of the module into five functions (level 3) and eleven operations (level 4).

Figure 11

Functional Decomposition Tree of the Human Resources Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

12.3 Functional Requirements

Table 31 specifies the 8 functional requirements of the Human Resources module. The module covers the four CRUD operations (create, read, update and delete).

Table 31

Functional Requirements of the Human Resources Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-RH-001	The system shall allow registering employees with personal and contractual data.	C	HR Officer	Identification number, name, position, contract type, salary and store are mandatory; the identification number must be unique.	Must
RF-RH-002	The system shall allow consulting employees and their contracts by store.	R	HR Officer, Store Manager	The list can be filtered by store, position and status; salaries are visible only to the HR officer.	Should
RF-RH-003	The system shall allow updating an employee's contract data (position, salary and contract type).	U	HR Officer	The change is stored with date and responsible user, and the previous values are kept in the history.	Should

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-RH-004	The system shall allow terminating an employee and generating a final settlement report.	D	HR Officer	The employee is deactivated, cannot log in and leaves the schedules; the report includes pending salary, prorated benefits and vacation; the history is kept.	Could
RF-RH-005	The system shall allow planning weekly 24/7 shifts, validating the minimum staff per shift, the rest between shifts and the legal maximum weekly hours.	C, U	Store Manager	Overlapping shifts for the same employee are rejected; a schedule with uncovered shifts cannot be published.	Must
RF-RH-006	The system shall allow employees to consult their shifts and request swaps, which are applied after the coworker accepts and the store manager approves.	R, U	Employee, Store Manager	The schedule is updated and both employees are notified only after acceptance and approval.	Should
RF-RH-007	The system shall register clock-in and clock-out through the POS or a personal PIN and calculate late arrivals and absences.	C	Employee, Store Manager	The time is recorded against the published schedule; late arrivals over 5 minutes are flagged.	Must
RF-RH-008	The system shall calculate the basic monthly payroll pre-settlement of active employees with configurable parameters.	R	HR Officer	The report includes base salary, overtime, night, Sunday and holiday surcharges, deductions and net pay per employee, and can be exported.	Should

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

12.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 32 specifies the non-functional requirements of the Human Resources module.

Table 32

Non-Functional Requirements of the Human Resources Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-RH-01	Security (confidentiality)	Employee personal and salary data shall be processed according to Law 1581 of 2012 (Congreso de la República de Colombia, 2012) and accessed only by authorized roles.	Access tests: 100% of unauthorized attempts rejected and logged; salaries visible only to the HR officer.
RNF-RH-02	Maintainability (modifiability)	Legal parameters (maximum weekly hours, surcharge percentages and holidays) shall be modifiable without changing the source code.	Parameter change applied from the administration panel in less than 5 minutes.
RNF-RH-03	Usability (operability)	Clock-in and clock-out shall not delay the start of the shift.	Attendance registered in 10 seconds or less.

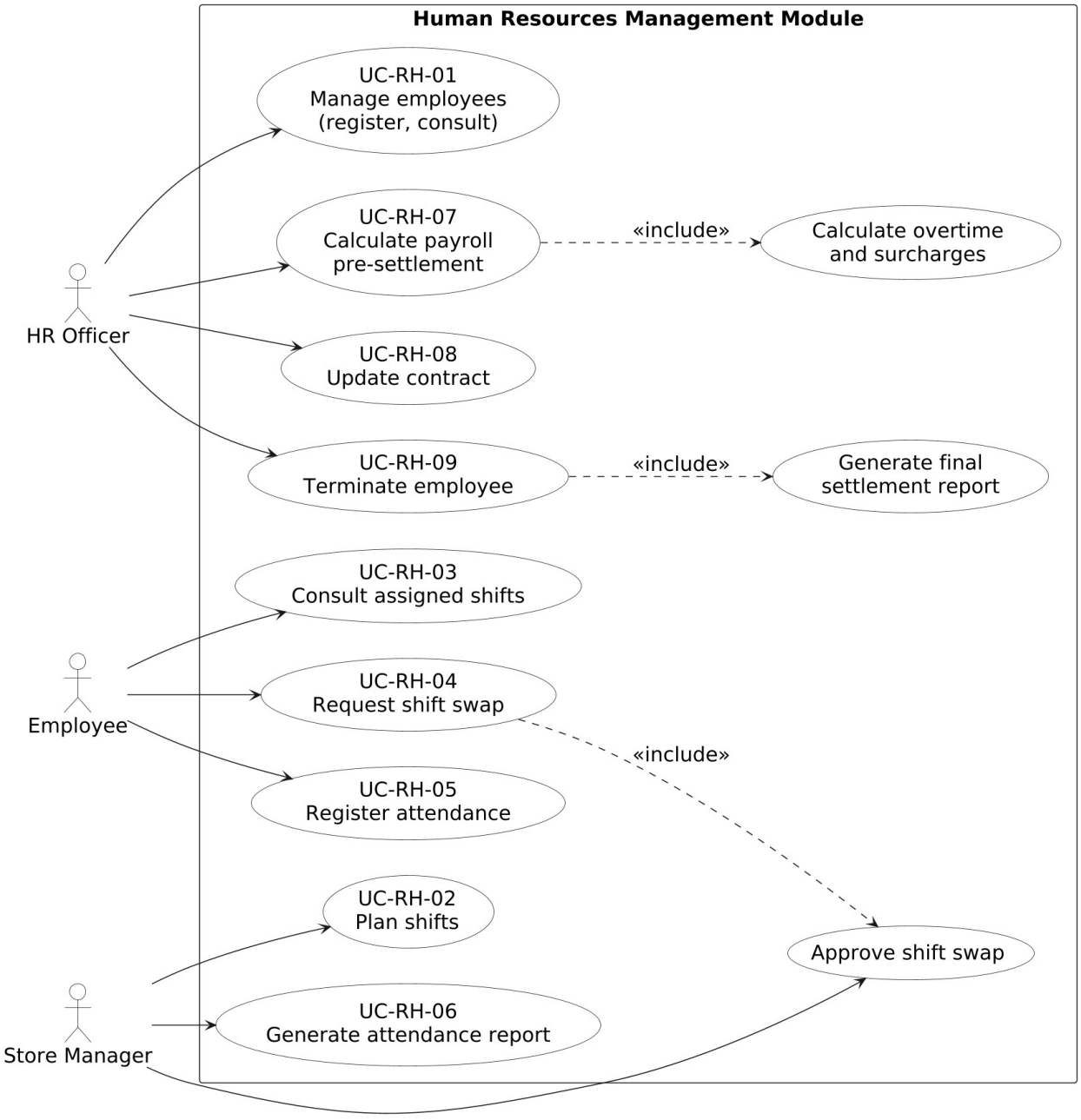
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

12.5 Use Case Diagram

Figure 12 presents the use case diagram of the Human Resources module, with its actors and the include and extend relationships between use cases.

Figure 12

Use Case Diagram of the Human Resources Module



Note. Diagram created by the authors in PlantUML.

12.6 Use Case Specification

Table 33 specifies UC-RH-02, the main use case of the module.

Table 33

Specification of Use Case UC-RH-02: Plan shifts

UC-RH-02: Plan shifts	
Actors	Primary: Store Manager. Secondary: Employee (receives the notification).
Description	The store manager builds and publishes the weekly 24/7 schedule of the store.
Preconditions	Employees are registered and assigned to the store. Shift types are configured.
Main flow	1. The store manager selects the store and the week to plan. 2. The system shows the 24/7 shift grid with the available employees and their accumulated hours. 3. The store manager assigns employees to each shift, or copies the schedule of the previous week. 4. The system validates the minimum staff per shift, the rest between shifts and the maximum weekly hours. 5. The store manager publishes the schedule. 6. The system notifies each employee of the assigned shifts.
Alternative flows	A1 (step 4). A validation fails: the system highlights the conflicting cells, and publishing is disabled until the conflict is solved or the manager records a justification (for authorized overtime). A2 (step 3). An employee has an approved absence: the system prevents assigning that employee to the affected shifts.
Postconditions	Weekly schedule published; attendance control uses it as the reference.
Traceability	RF-RH-005; HU-RH-05

Note. Prepared by the authors.

12.7 Epic E-04 and User Stories

The module corresponds to epic **E-04: Human Resources** of the Product Backlog. Table 34 presents its 8 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 46 story points, 16 of them in *Must* stories.

Table 34

User Stories of Epic E-04: Human Resources

ID	**User story**	**RF**	**Priority**	**SP**	**Acceptance criteria**
HU-RH-01	As an HR officer, I want to register employees, so that the staff list is up to date.	RF-RH-001	Must	5	Given a new employee, when I save the record without a mandatory field, then it is rejected; with complete data, the employee appears in the store.

ID	User story	RF	Priority	SP	Acceptance criteria
HU-RH-02	As an HR officer, I want to consult employees and their contracts by store, so that I can manage the staff of the chain.	RF-RH-002	Should	3	Given several stores, when I filter by store and position, then I see the matching employees and their contracts.
HU-RH-03	As an HR officer, I want to update an employee's contract, so that salary or position changes are reflected.	RF-RH-003	Should	3	Given an employee, when I change the salary, then the new value applies and the change is stored with date and user.
HU-RH-04	As an HR officer, I want to terminate an employee and generate the final settlement report, so that the process is formal.	RF-RH-004	Could	8	Given an active employee, when I register the termination, then the employee is deactivated and the settlement report is generated.
HU-RH-05	As a store manager, I want to plan weekly 24/7 shifts, so that every time slot has the minimum staff.	RF-RH-005	Must	8	Given a week grid, when a shift has fewer employees than the minimum, then the system highlights it and prevents publishing.
HU-RH-06	As an employee, I want to request a shift swap with a coworker, so that I can handle personal commitments without leaving the store uncovered.	RF-RH-006	Should	3	Given two employees with assigned shifts, when both accept and the manager approves, then the schedule is updated and both are notified.
HU-RH-07	As an employee, I want to register my clock-in and clock-out, so that my worked hours are recorded accurately.	RF-RH-007	Must	3	Given a scheduled shift, when I enter my PIN at the POS, then the time is recorded and late arrivals over 5 minutes are flagged.
HU-RH-08	As an HR officer, I want the system to calculate overtime and surcharges in the payroll pre-settlement, so that payments are accurate.	RF-RH-008	Should	13	Given attendance records for a month, when I generate the pre-settlement, then base salary, surcharges, deductions and net pay are calculated with the configured parameters.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

13. Module 5: Customer Management (CRM)

13.1 Module Description

The Customer Management (CRM) module builds customer loyalty. It registers customers together with their explicit consent for the processing of personal data, as required by Law 1581 of 2012 (Congreso de la República de Colombia, 2012), allows customers to update their data and to request its deletion, and runs a points-based loyalty program with bronze, silver and gold tiers.

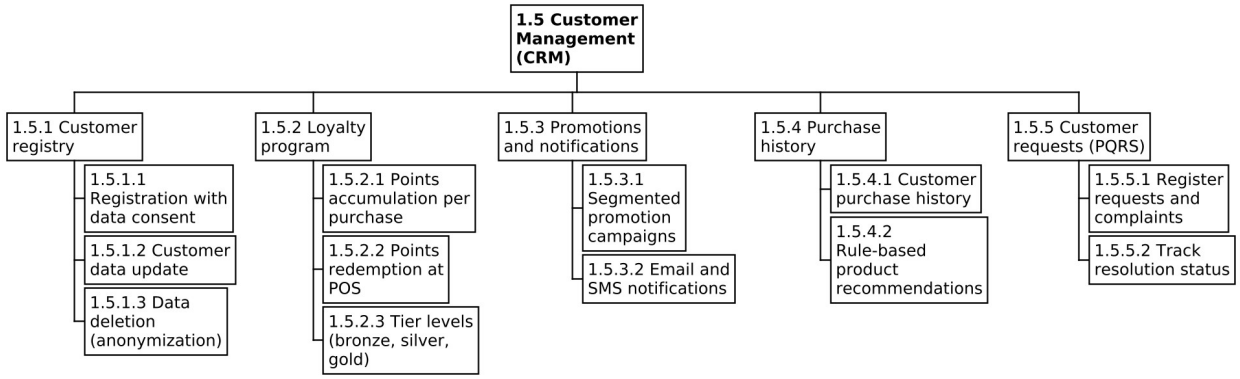
The module sends segmented promotions by email and SMS, keeps each customer's purchase history to offer rule-based product recommendations, and manages customer petitions, complaints, claims and suggestions (PQRS, from the Spanish peticiones, quejas, reclamos y sugerencias) until they are resolved.

13.2 Functional Decomposition Tree

Figure 13 presents the functional decomposition of the module into five functions (level 3) and twelve operations (level 4).

Figure 13

Functional Decomposition Tree of the CRM Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

13.3 Functional Requirements

Table 35 specifies the 10 functional requirements of the CRM module. The module covers the four CRUD operations (create, read, update and delete).

Table 35

Functional Requirements of the CRM Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-CRM-001	The system shall allow registering customers at the POS or through self-registration, recording their explicit consent for data processing and marketing messages.	C	Cashier, Customer	Name and a contact (phone or email) are mandatory; the consent is stored with its date; registration takes less than 1 minute.	Must
RF-CRM-002	The system shall allow consulting the profile and purchase history of a customer.	R	Store Manager, Corporate Manager	The profile shows invoices, amounts, dates, points and tier of the last 12 months.	Could
RF-CRM-003	The system shall allow updating customer data and consent preferences.	U	Cashier, Customer	Changes are logged; consent for marketing can be withdrawn at any time.	Should
RF-CRM-004	The system shall allow deleting (anonymizing) the personal data of a customer on request.	D	Store Manager	Personal data is removed, the loyalty account is closed and aggregated sales figures are kept.	Should
RF-CRM-005	The system shall accumulate loyalty points on each purchase according to a configurable rule and allow customers to consult their balance.	U	Cashier (automatic), Customer	Points are updated when the sale is confirmed and shown on the receipt.	Must
RF-CRM-006	The system shall allow redeeming points as a discount at the POS.	U	Cashier, Customer	The balance and the maximum redeemable percentage of the purchase are validated before applying the discount.	Must
RF-CRM-007	The system shall classify customers into bronze, silver and gold tiers according to the points of the last 12 months.	U	Corporate Manager	The tier and its benefits (for example, accumulation multipliers) are updated automatically.	Could

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-CRM-008	The system shall allow creating promotion campaigns for customer segments and sending them by email or SMS.	C	Corporate Manager; Notification Service	Only customers of the segment who accepted marketing messages receive the campaign.	Should
RF-CRM-009	The system shall generate rule-based recommendations of frequently bought products for each customer.	R	Cashier, Corporate Manager	Recommendations are based on the purchase frequency of the last 12 months.	Could
RF-CRM-010	The system shall register customer PQRS with a tracking number, assign them to the store manager, record the response and notify the customer.	C, U	Customer, Store Manager	The customer receives a tracking number and a notification when the response is recorded.	Should

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

13.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 36 specifies the non-functional requirements of the CRM module.

Table 36

Non-Functional Requirements of the CRM Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-CRM-01	Security (confidentiality)	Customers shall be able to consult, update and request the deletion of their data, and marketing messages shall be sent only with explicit consent (Congreso de la República de Colombia, 2012).	100% of campaign recipients with a consent record; deletion requests answered within the legal term.
RNF-CRM-02	Performance efficiency (time behavior)	The points balance shall be shown at the POS without delaying the sale.	Balance displayed in 1 second or less.

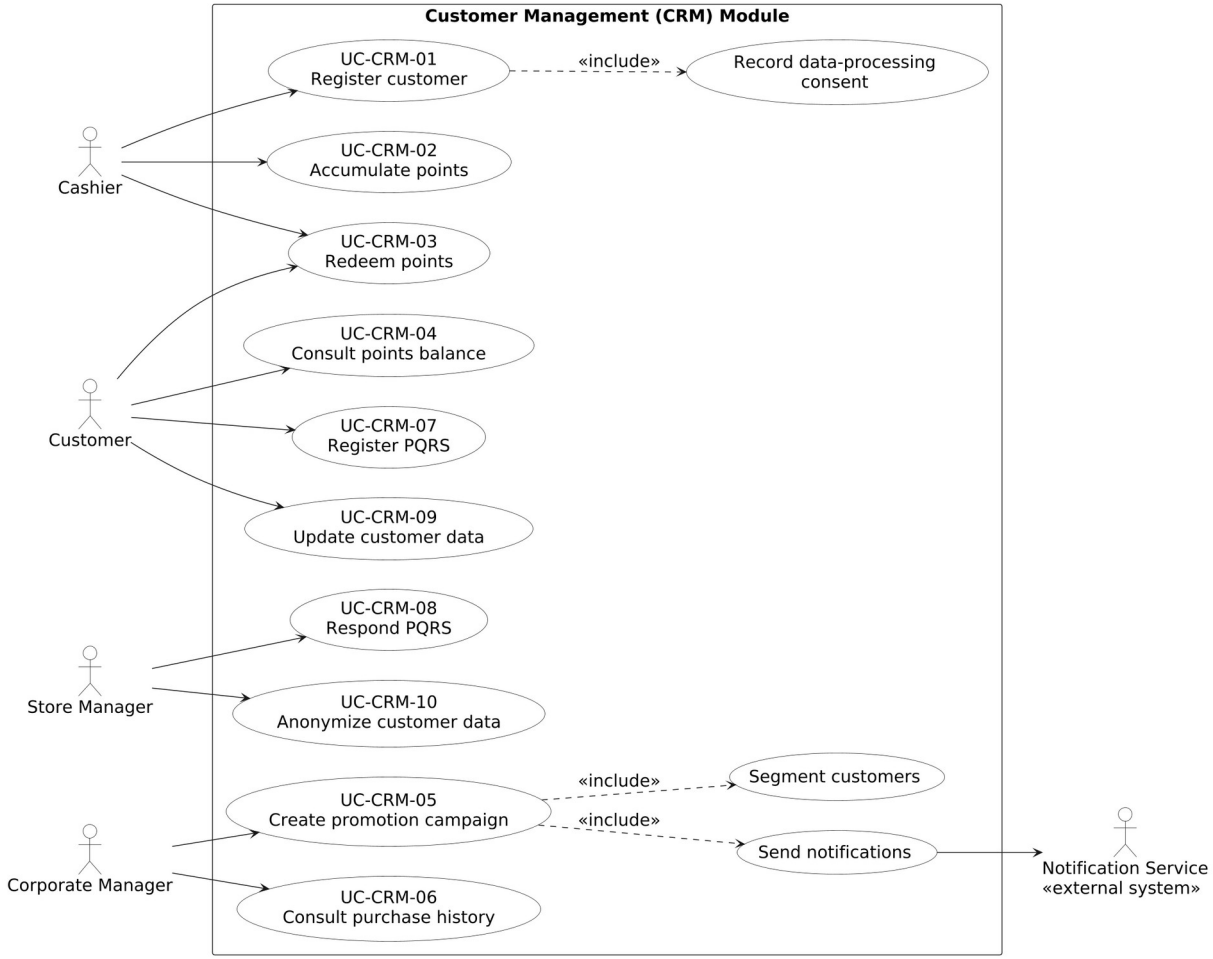
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

13.5 Use Case Diagram

Figure 14 presents the use case diagram of the CRM module, with its actors and the include and extend relationships between use cases.

Figure 14

Use Case Diagram of the CRM Module



Note. Diagram created by the authors in PlantUML.

13.6 Use Case Specification

Table 37 specifies UC-CRM-03, the main use case of the module.

Table 37

Specification of Use Case UC-CRM-03: Redeem points

UC-CRM-03: Redeem points	
Actors	Primary: Cashier. Secondary: Customer.
Description	During a sale, the customer uses loyalty points as a discount on the purchase.
Preconditions	A sale is in progress (UC-VEN-02). The customer is registered and has a points balance.
Main flow	1. The cashier enters or scans the customer's identification during the sale. 2. The system shows the customer's name, tier, available points and their equivalent value in COP. 3. The customer indicates the number of points to redeem. 4. The system validates the balance and verifies that the discount does not exceed the configured maximum percentage of the purchase. 5. The system applies the discount to the sale and reserves the points. 6. When the payment is completed, the system deducts the points and records the redemption in the customer's history.
Alternative flows	A1 (step 4). The balance is insufficient: the system shows the maximum redeemable amount. A2 (step 6). The sale is cancelled: the system releases the reserved points.
Postconditions	Points deducted; discount shown on the electronic invoice.
Traceability	RF-CRM-006, RF-VEN-004; HU-CRM-06

Note. Prepared by the authors.

13.7 Epic E-05 and User Stories

The module corresponds to epic **E-05: Customer Management (CRM)** of the Product Backlog. Table 38 presents its 10 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 35 story points, 11 of them in *Must* stories.

Table 38

User Stories of Epic E-05: Customer Management (CRM)

ID	User story	RF	Priority	SP	Acceptance criteria
HU-CRM-01	As a cashier, I want to register a new customer at the POS in less than one minute, including the consent, so that the customer starts earning points immediately.	RF-CRM-001	Must	3	Given a new customer, when I save the basic data and consent, then the customer can accumulate points in the current sale.
HU-CRM-02	As a corporate manager, I want to consult a customer's purchase history, so that I can personalize offers.	RF-CRM-002	Could	3	Given a registered customer, when I open the profile, then I see the purchases, points and tier of the last 12 months.
HU-CRM-03	As a customer, I want to update my contact data and consent preferences, so that I control how the chain contacts me.	RF-CRM-003	Should	2	Given my profile, when I withdraw the marketing consent, then I stop receiving promotions.
HU-CRM-04	As a customer, I want to request the deletion of my personal data, so that my rights under Colombian law are respected.	RF-CRM-004	Should	3	Given my request, when the store manager processes it, then my personal data is anonymized and aggregated sales are kept.
HU-CRM-05	As a customer, I want to accumulate points with every purchase, so that I am rewarded for my loyalty.	RF-CRM-005	Must	3	Given a rule of 1 point per COP 1,000, when I complete a purchase of COP 25,000, then 25 points are added and shown on the receipt.
HU-CRM-06	As a customer, I want to redeem my points at checkout, so that I get discounts on my purchases.	RF-CRM-006	Must	5	Given a balance of 500 points, when I redeem 200 at the POS, then the discount is applied and my balance becomes 300.
HU-CRM-07	As a corporate manager, I want customers to be classified into bronze, silver and gold tiers, so that frequent customers receive greater benefits.	RF-CRM-007	Could	3	Given the configured thresholds, when a customer exceeds the silver threshold, then the tier and its multiplier are updated automatically.
HU-CRM-08	As a corporate manager, I want to send promotions to specific customer segments, so that I increase repeat purchases.	RF-CRM-008	Should	5	Given the segment gold customers, when I launch a campaign, then only customers of that segment with marketing consent receive it.
HU-CRM-09	As a cashier, I want to see recommended products for a registered customer, so that I can suggest them at checkout.	RF-CRM-009	Could	5	Given a customer with purchase history, when I identify the customer at the POS, then up to three frequently bought products are suggested.

ID	User story	RF	Priority	SP	Acceptance criteria
HU-CRM-10	As a customer, I want to file a PQRS and receive a tracking number, so that I can follow it until it is resolved.	RF-CRM-010	Should	3	Given a submitted PQRS, when it is registered, then I receive a tracking number by email and a notification when the response is recorded.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

14. Module 6: Security and Reporting

14.1 Module Description

The Security and Reporting module is a cross-cutting module added in Sprint 1 so that the administration functions shared by all the business modules also have actors, requirements and complete CRUD coverage. It manages user accounts, roles and permissions per store, authentication with a second factor for administrative roles, the immutable audit log of create, update and delete operations, and the configuration of system parameters (taxes, approval thresholds, loyalty rules and labor parameters).

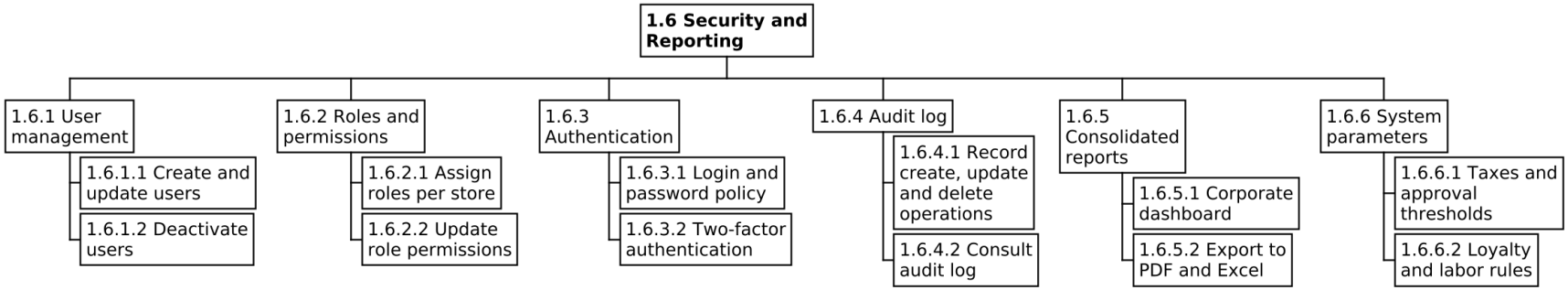
The module also provides the consolidated reports and the corporate dashboard used by the chain's management to compare sales, inventory, purchasing and staff indicators across all the stores. Its security functions are delivered first (Sprint 2), because every other module depends on them, and its reporting functions at the end of the project (Sprint 6).

14.2 Functional Decomposition Tree

Figure 15 presents the functional decomposition of the module into six functions (level 3) and twelve operations (level 4).

Figure 15

Functional Decomposition Tree of the Security and Reporting Module (Levels 2 to 4)



Note. Diagram created by the authors in PlantUML (WBS). Numbering follows the system tree (Figure 3).

14.3 Functional Requirements

Table 39 specifies the 8 functional requirements of the Security and Reporting module. The module covers the four CRUD operations (create, read, update and delete).

Table 39

Functional Requirements of the Security and Reporting Module

ID	Requirement	CRUD	Actor	Acceptance criterion	Priority
RF-SEG-001	The system shall allow creating user accounts and assigning a role (cashier, stock clerk, store manager, logistics coordinator, HR officer, corporate manager or administrator) and a store.	C	System Administrator	The email is unique; a temporary password is sent and must be changed at the first login.	Must
RF-SEG-002	The system shall allow consulting users, their roles and their status.	R	System Administrator	The list can be filtered by store, role and status.	Should
RF-SEG-003	The system shall allow updating user assignments and the permissions of each role.	U	System Administrator	Changes apply at the next login of the user and are logged.	Should
RF-SEG-004	The system shall allow deactivating users.	D	System Administrator	A deactivated user cannot log in; the history of the user's operations remains.	Must
RF-SEG-005	The system shall authenticate users with a password and require a second factor for administrative roles.	R	All users	The account is locked after 5 failed attempts; administrator, corporate manager and HR officer cannot log in without the second factor.	Must
RF-SEG-006	The system shall allow consulting the audit log of create, update and delete operations.	R	System Administrator	Each entry shows user, date, action, module and IP; the log is read-only and exportable.	Should
RF-SEG-007	The system shall provide consolidated reports and a corporate dashboard of sales, inventory, purchasing and staff across all stores.	R	Corporate Manager	Reports can be filtered by store and period, exported to PDF and Excel, and are generated in 10 seconds or less.	Should
RF-SEG-008	The system shall allow configuring system parameters (taxes, approval thresholds, loyalty rules and labor parameters).	U	System Administrator	Every change is versioned and logged and takes effect without modifying the source code.	Should

Note. C = create, R = read, U = update, D = delete (logical deletion when the history must be preserved). Priority according to the MoSCoW technique.

14.4 Non-Functional Requirements

In addition to the system-wide requirements of Section 8.5, Table 40 specifies the non-functional requirements of the Security and Reporting module.

Table 40

Non-Functional Requirements of the Security and Reporting Module

ID	ISO/IEC 25010 category	Requirement	Verification metric
RNF-SEG-01	Security (accountability)	Every create, update and delete operation of every module shall be recorded in an immutable audit log.	100% of operations logged with user, date, action, module and IP; modification attempts on the log fail in testing.
RNF-SEG-02	Security (authenticity)	Administrative roles shall use two-factor authentication.	Login tests: administrative accounts cannot access the system without the second factor.

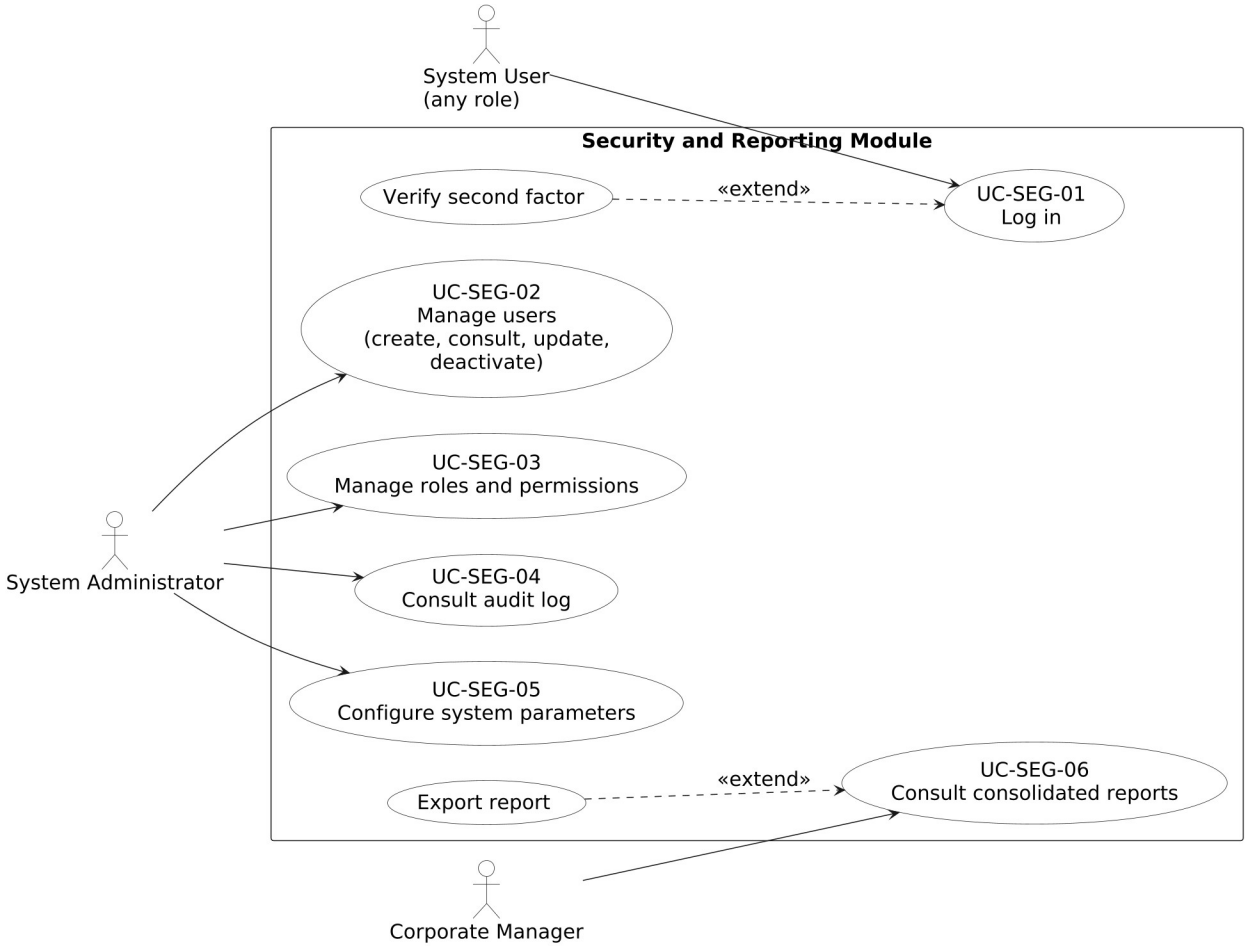
Note. Quality characteristics according to ISO/IEC 25010 (ISO/IEC, 2011).

14.5 Use Case Diagram

Figure 16 presents the use case diagram of the Security and Reporting module, with its actors and the include and extend relationships between use cases.

Figure 16

Use Case Diagram of the Security and Reporting Module



Note. Diagram created by the authors in PlantUML.

14.6 Use Case Specification

Table 41 specifies UC-SEG-02, the main use case of the module.

Table 41

Specification of Use Case UC-SEG-02: Manage users (create user account)

UC-SEG-02: Manage users (create user account)	
Actors	Primary: System Administrator. Secondary: New user (receives the credentials by email).
Description	The administrator creates a user account, assigns its role and store, and the new user activates it.
Preconditions	The administrator is authenticated with the second factor. The role and the store exist.
Main flow	1. The administrator selects New user. 2. The system requests name, email, identification number, store and role, and optionally links the account to a registered employee. 3. The administrator enters the data and confirms. 4. The system validates that the email is unique and that the role is valid for the selected store. 5. The system creates the account with a temporary password, sends it by email and records the operation in the audit log. 6. At the first login, the new user changes the temporary password; if the role is administrative, the system requires configuring the second factor (extend: Verify second factor).
Alternative flows	A1 (step 4). The email is already registered: the system rejects the operation and shows the status of the existing account; if it is deactivated, the administrator may reactivate it. A2 (step 6). The user does not activate the account within 72 hours: the temporary password expires and the administrator can send a new one.
Postconditions	Active user with one role per store; operation recorded in the audit log.
Traceability	RF-SEG-001, RF-SEG-005, RF-SEG-006; HU-SEG-01, HU-SEG-05

Note. Prepared by the authors.

14.7 Epic E-06 and User Stories

The module corresponds to epic **E-06: Security and Reporting** of the Product Backlog. Table 42 presents its 8 user stories with the related requirement, priority, estimate and acceptance criteria. The epic totals 35 story points, 12 of them in *Must* stories.

Table 42

User Stories of Epic E-06: Security and Reporting

ID	User story	RF	Priority	SP	Acceptance criteria
HU-SEG-01	As an administrator, I want to create users and assign their role and store, so that access is controlled.	RF-SEG-001	Must	5	Given a new user, when I assign the cashier role in a store, then the user can only access the POS functions of that store.
HU-SEG-02	As an administrator, I want to consult users and their roles, so that I can review who has access to each store.	RF-SEG-002	Should	2	Given several users, when I filter by store and role, then I see the matching users and their status.
HU-SEG-03	As an administrator, I want to update the permissions of a role, so that access follows the company's policies.	RF-SEG-003	Should	3	Given a role, when I remove a permission, then users with that role lose it at their next login.
HU-SEG-04	As an administrator, I want to deactivate users, so that former staff lose access.	RF-SEG-004	Must	2	Given a deactivated user, when the user tries to log in, then access is denied and the history remains.
HU-SEG-05	As a corporate manager, I want to log in with a second factor, so that my account is protected.	RF-SEG-005	Must	5	Given correct credentials, when I do not provide the second factor, then access is denied.
HU-SEG-06	As an administrator, I want to consult the audit log, so that I can trace operations.	RF-SEG-006	Should	5	Given a date range, when I search the log, then I see user, date, action, module and IP of each operation.
HU-SEG-07	As a corporate manager, I want consolidated reports of all the stores, so that I can make decisions for the chain.	RF-SEG-007	Should	8	Given a period, when I open the dashboard, then I see sales, stock, purchasing and staff indicators per store in 10 seconds or less.
HU-SEG-08	As an administrator, I want to configure taxes, approval thresholds and loyalty rules, so that the system adapts without programming.	RF-SEG-008	Should	5	Given a new approval threshold, when I save it, then new purchase orders use it and the change is logged.

Note. SP = story points estimated with Planning Poker (Fibonacci scale). RF = related functional requirement. Priority according to the MoSCoW technique.

15. Product Backlog

The Product Backlog is the prioritized list of everything the product needs (Schwaber & Sutherland, 2020). It groups the requirements of TiendaFlow into six epics, one per module, and 57 user stories, each with at least one story per CRUD operation. The backlog totals 266 story points; the 23 *Must* stories (123 story points) form the minimum viable product. The detailed user stories are presented in the section of each module (Sections 9.7 to 14.7).

15.1 Epics and Release Plan

Table 43 summarizes the epics, their size and the sprint in which each one is planned, consistent with the schedule of Section 3.2.

Table 43

Epics of the Product Backlog and Release Plan

Epic	Detail	User stories	Story points	Must story points	Planned sprint
E-01: Inventory Management	Section 9	11	51	29	Sprint 2
E-02: Sales and Billing	Section 10	10	55	39	Sprint 3
E-03: Supplier and Purchasing	Section 11	10	44	16	Sprint 4
E-04: Human Resources	Section 12	8	46	16	Sprint 5
E-05: Customer Management (CRM)	Section 13	10	35	11	Sprint 5
E-06: Security and Reporting	Section 14	8	35	12	Sprints 2 and 6
Total		**57**	**266**	**123**	

Note. Story points estimated with Planning Poker on the Fibonacci scale. Could stories are delivered only if the team's velocity allows it.

15.2 Definition of Done

The Definition of Done (DoD) is the agreement of the team that applies to all the user stories. It adapts the DoD proposed in the Sprint 1 guide to the Microsoft stack of the project. A user story is considered done only when all the following conditions are met:

1. The code has been reviewed by at least one teammate through a pull request on GitHub.
2. Unit tests are written with xUnit, and the coverage of the business logic is 80% or more (coverlet).
3. Integration tests of the affected API endpoints have been executed (Postman collection).
4. The API documentation is updated in Swagger (OpenAPI).
5. The architecture documentation (arc42 structure) and the PlantUML diagrams are updated on the GitHub Pages site.
6. Database changes are versioned with Entity Framework Core migrations for SQL Server.
7. The deployment pipeline (GitHub Actions) and its scripts are reviewed and versioned in GitHub; this replaces the Terraform item of the guide to respect the Microsoft-only rule.
8. Create, update and delete operations generate entries in the audit log (RNF-SEG-01).
9. The feature is deployed successfully in the test environment, and the Product Owner accepts it after verifying its acceptance criteria.

16. Architecture and Data Models

This section presents the UML and architecture models requested in the Sprint 1 guide. Together with the use case diagrams of each module, they describe the logical, data and context views of the system, in line with the 4+1 view model (Kruchten, 1995) and the three levels of the C4 model used in this document: context, containers and components (Brown, n.d.). All the diagrams were rendered in Google Colab with iplantuml (Section 4).

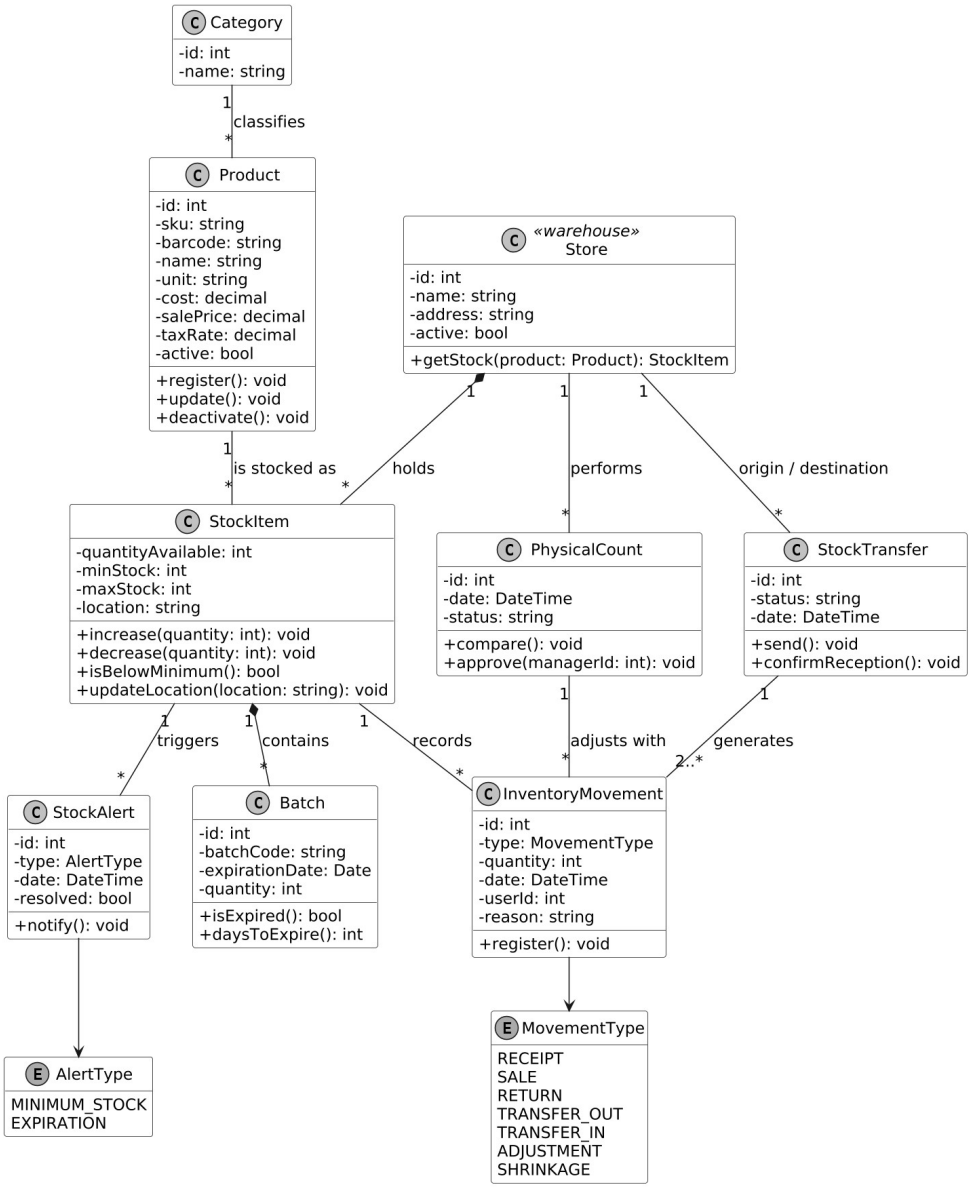
16.1 Class Diagram of the Inventory Module

Figure 17 presents the class diagram of the Inventory module with its main entities: product, warehouse and movement. In TiendaFlow, each **Store** plays the role of warehouse (stereotype <<warehouse>>) and keeps one **StockItem** per product, with its minimum and

maximum levels and its location. Each stock item contains the **Batches** received with their expiration dates and records every **InventoryMovement**, whose type is defined by the MovementType enumeration. Stock transfers and physical counts generate movements, and stock items trigger minimum stock and expiration alerts.

Figure 17

Class Diagram of the Inventory Module



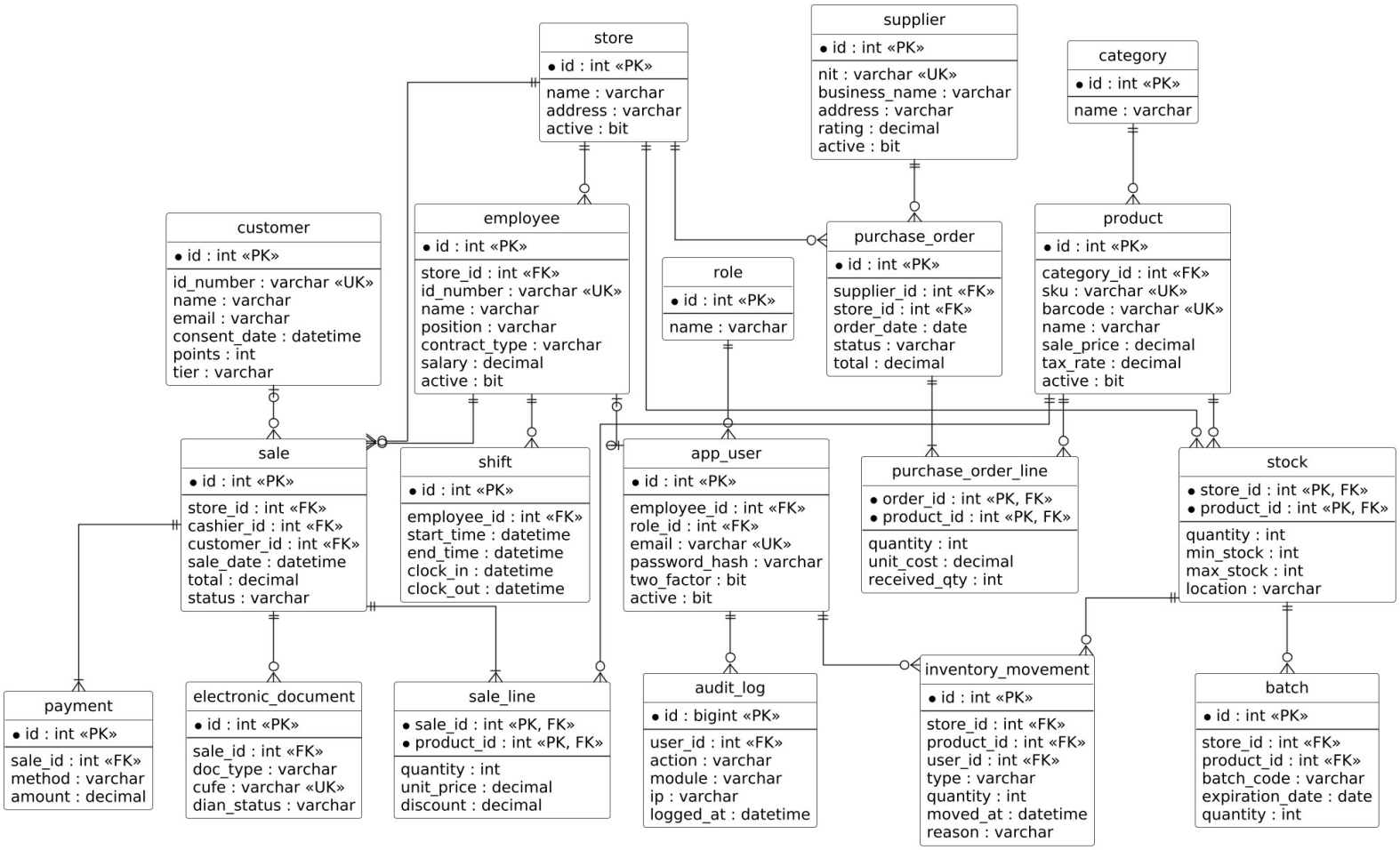
Note. Diagram created by the authors in PlantUML.

16.2 Entity-Relationship Diagram

Figure 18 presents the entity-relationship diagram of TiendaFlow in crow's foot notation. The model contains 19 entities that cover the six modules. Stock, purchase order lines and sale lines use composite keys (store and product, order and product, sale and product), which support the multi-store requirement; sales can have several payments and several electronic documents (invoice and credit notes); and every operation of an application user is recorded in the audit log.

Figure 18

Entity-Relationship Diagram of TiendaFlow



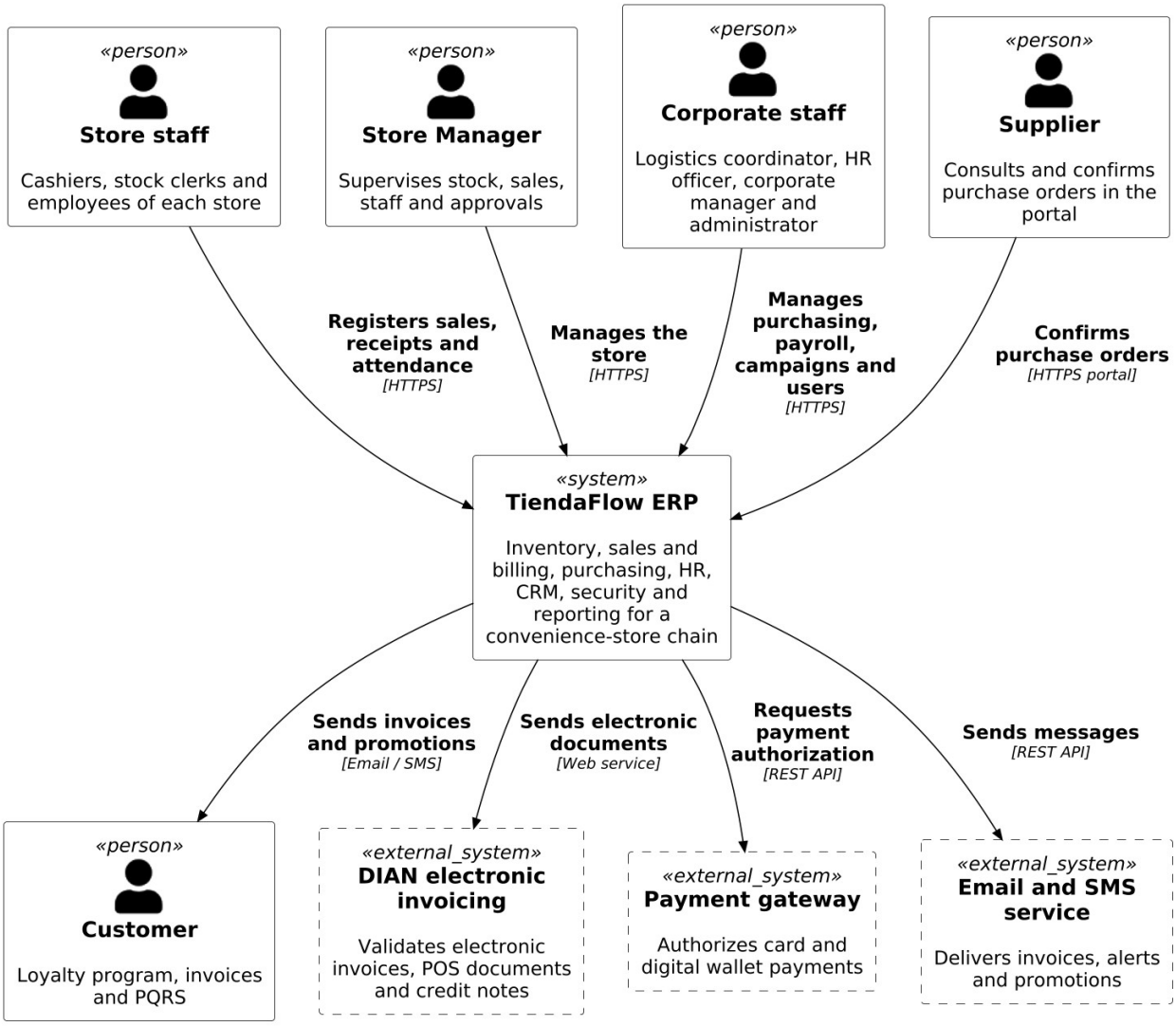
Note. Diagram created by the authors in PlantUML. PK = primary key, FK = foreign key, UK = unique key.

16.3 C4 Context Diagram

Figure 19 presents the C4 context diagram (level 1). It shows TiendaFlow as the central system, the people who use it (store staff, store managers, corporate staff, suppliers and customers) and the external systems it depends on: DIAN electronic invoicing, the payment gateway and the email and SMS service.

Figure 19

C4 Context Diagram of TiendaFlow



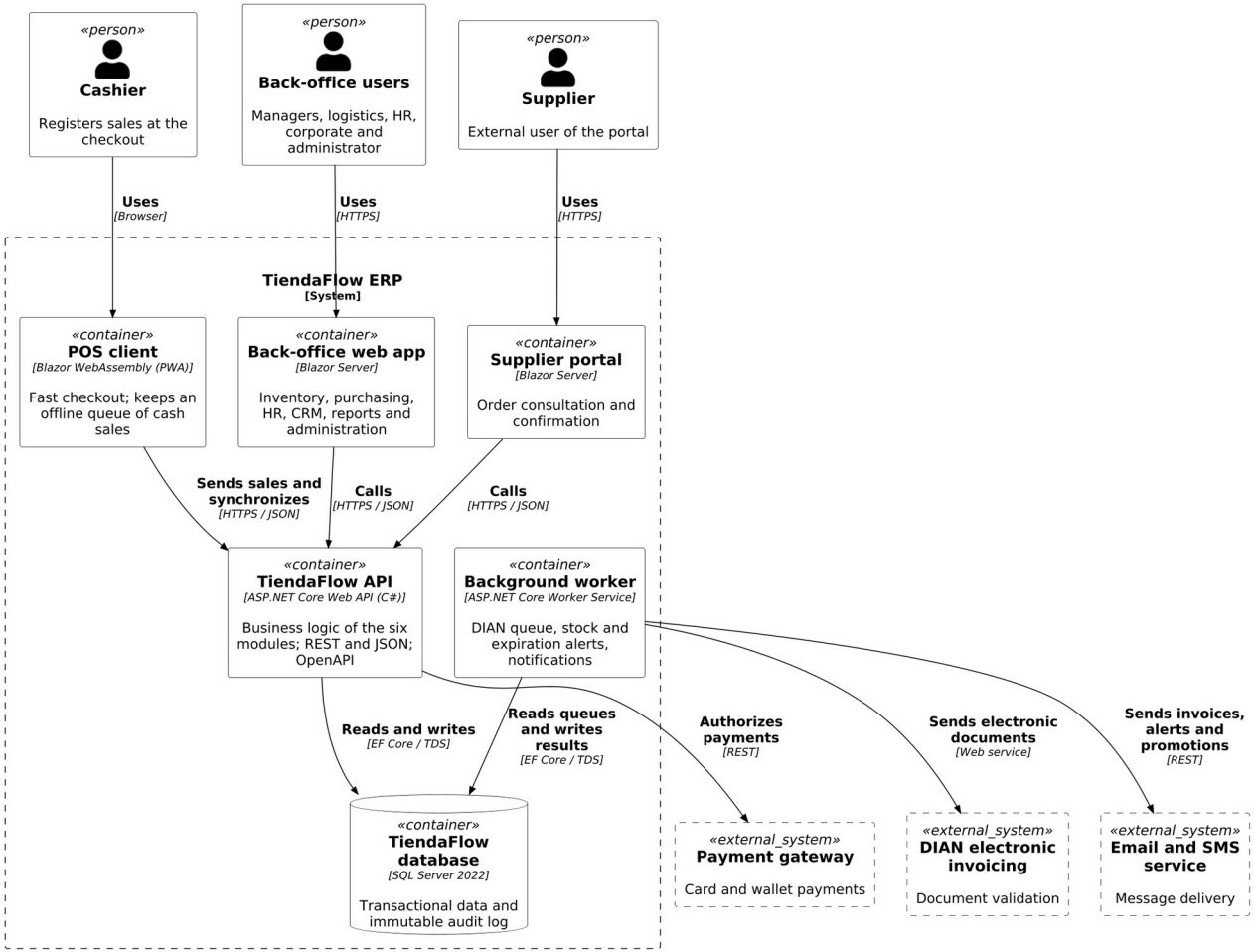
Note. Diagram created by the authors in PlantUML with the C4-PlantUML library. Dashed borders indicate external systems.

16.4 C4 Container Diagram

Figure 20 presents the C4 container diagram (level 2), which shows how the selected Microsoft technologies are distributed. The POS client is a Blazor WebAssembly progressive web application that keeps an offline queue of cash sales (RNF-VEN-02); the back-office application and the supplier portal use Blazor Server; all of them call the TiendaFlow API built with ASP.NET Core, which stores the data in SQL Server. A worker service processes the DIAN queue, the stock and expiration alerts and the notifications, so that slow external services do not delay the checkout.

Figure 20

C4 Container Diagram of TiendaFlow



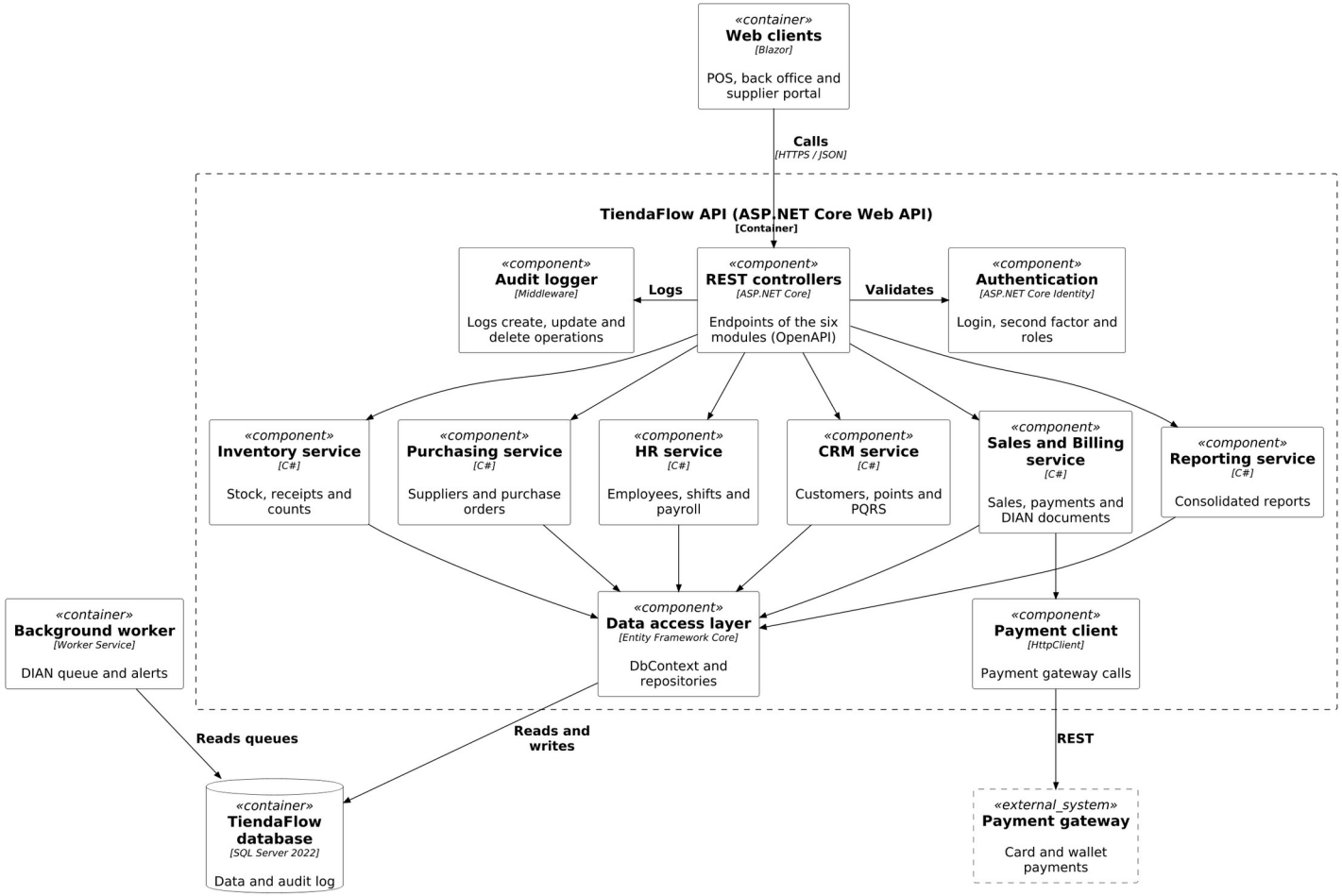
Note. Diagram created by the authors in PlantUML with the C4-PlantUML library. Dashed borders indicate external systems.

16.5 C4 Component Diagram of the API

Figure 21 presents the C4 component diagram (level 3) of the TiendaFlow API container. The REST controllers receive the requests of the web clients, validate the token and the role with the authentication component and register every create, update and delete operation through the audit logger (RNF-SEG-01). Each module of the system is implemented as a service, one per epic of the Product Backlog, and all of them access SQL Server through a data access layer built with Entity Framework Core. The Sales and Billing service uses a payment client to communicate with the payment gateway, while the background worker processes the DIAN queue and the alerts outside the API.

Figure 21

C4 Component Diagram of the TiendaFlow API



Note. Diagram created by the authors in PlantUML with the C4-PlantUML library. Dashed borders indicate external systems.

17. Mockups and Functional Prototype

The interface was designed in Figma in two artifacts. The first is the design file ERP_Mockups_TiendaFlow, which contains the design system, the three high-fidelity mockups requested in the Sprint 1 guide (the purchases, inventory and human resources dashboards) and a complete interactive prototype of the six modules, with 31 screens and 38 dialogs connected to each other. Every screen is a desktop frame of 1440 x 1024 px on an 8 px grid, built only with instances of the components and with the color and text styles of the design system. The second artifact is an earlier functional prototype generated with Figma Make, the artificial intelligence feature of Figma, which was published as an interactive web application.

17.1 Design System

Table 44 describes the design system created in the Figma file, organized in color styles, text styles and components, as indicated in the Sprint 1 guide, and Figure 22 shows the design system sheet of the file.

Table 44

Design System of TiendaFlow in Figma

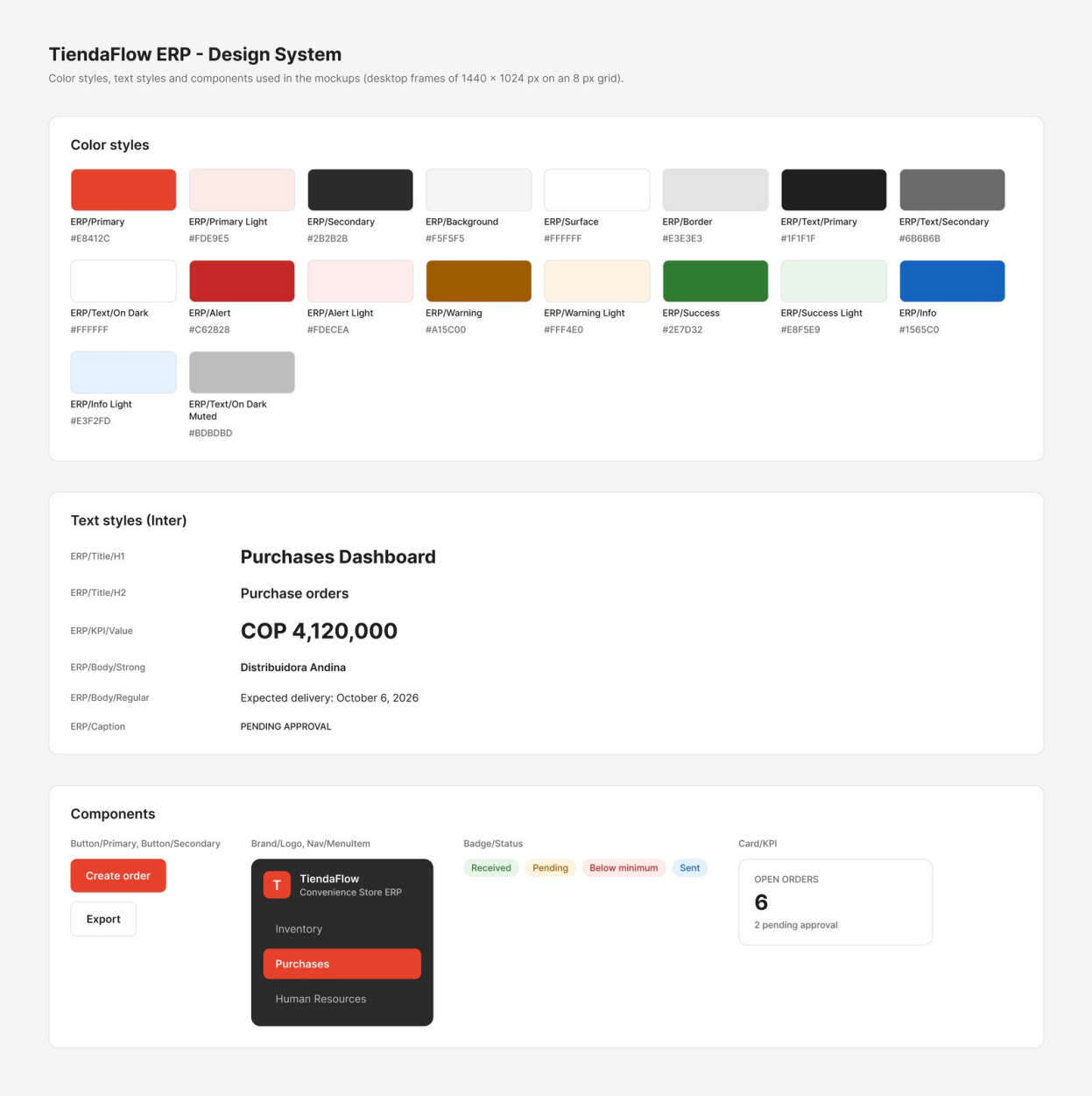
Element	Name	Definition and use
Color style	ERP/Primary and ERP/Primary Light	#E8412C and #FDE9E5. Primary buttons, active menu item and key figures
Color style	ERP/Secondary	#2B2B2B. Side navigation menu
Color style	ERP/Background, ERP/Surface and ERP/Border	#F5F5F5, #FFFFFF and #E3E3E3. Screen background, cards and dividers
Color style	ERP/Text/Primary, ERP/Text/Secondary, ERP/Text/On Dark and ERP/Text/On Dark Muted	#1F1F1F, #6B6B6B, #FFFFFF and #BDBDBD. Text on light and dark surfaces
Color style	ERP/Alert, ERP/Warning, ERP/Success and ERP/Info (each with a Light variant)	#C62828, #A15C00, #2E7D32 and #1565C0. Status of products, orders and employees

Element	Name	Definition and use
Text style	ERP/Title/H1 and ERP/Title/H2	Inter Bold 24 px and Inter Semi Bold 18 px. Screen and panel titles
Text style	ERP/KPI/Value	Inter Bold 28 px. Large figures of indicator cards
Text style	ERP/Body/Regular and ERP/Body/Strong	Inter Regular and Semi Bold 14 px. Tables, forms and emphasized values
Text style	ERP/Caption	Inter Medium 12 px. Badges, column headers and small labels
Grid style	ERP/Grid 8px	8 px layout grid applied to every screen
Component	Button/Primary and Button/Secondary	Main and secondary actions, such as Create order, Calculate payroll and Export (text property Label)
Component	Nav/MenuItem	Item of the side menu with the variants State=Default and State=Active
Component	Badge/Status	Status label with the variants Success, Warning, Alert and Info
Component	Card/KPI	Indicator card with the text properties Label, Value and Detail
Component	Brand/Logo	TiendaFlow logo for the side menu

Note. Prepared by the authors from the Figma file ERP_Mockups_TiendaFlow.

Figure 22

Design System Sheet in Figma



Note. Created by the authors in Figma. It shows the color styles, the text styles and the components of the design system.

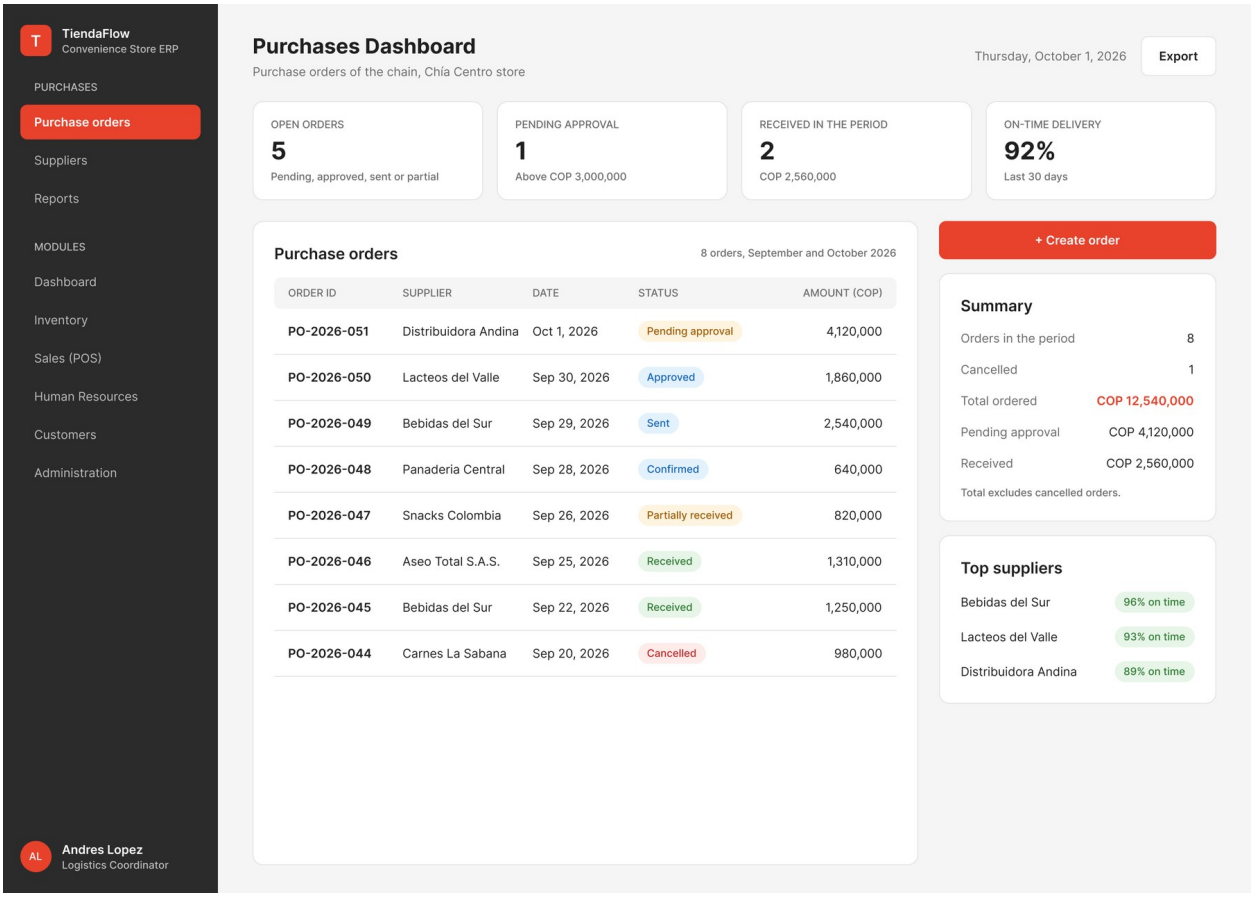
17.2 Mockups

The three mockups requested in the guide are presented in Figures 23 to 25. The purchases dashboard shows the side menu with Purchase orders, Suppliers and Reports, indicator

cards, the table of purchase orders (identifier, supplier, date, status and amount) and a right panel with the Create order button, the summary of totals and the best suppliers. The inventory dashboard shows the list of products with their current and minimum stock on the left, the critical stock alerts and the batches close to expiration in the center, and a chart of stock levels by category on the right. The payroll dashboard shows the side menu with Employees, Payroll and Reports, the table of employees (name, position, contract type, base salary, overtime and status) and a right panel with the Calculate payroll button and the totals of the pre-settlement.

Figure 23

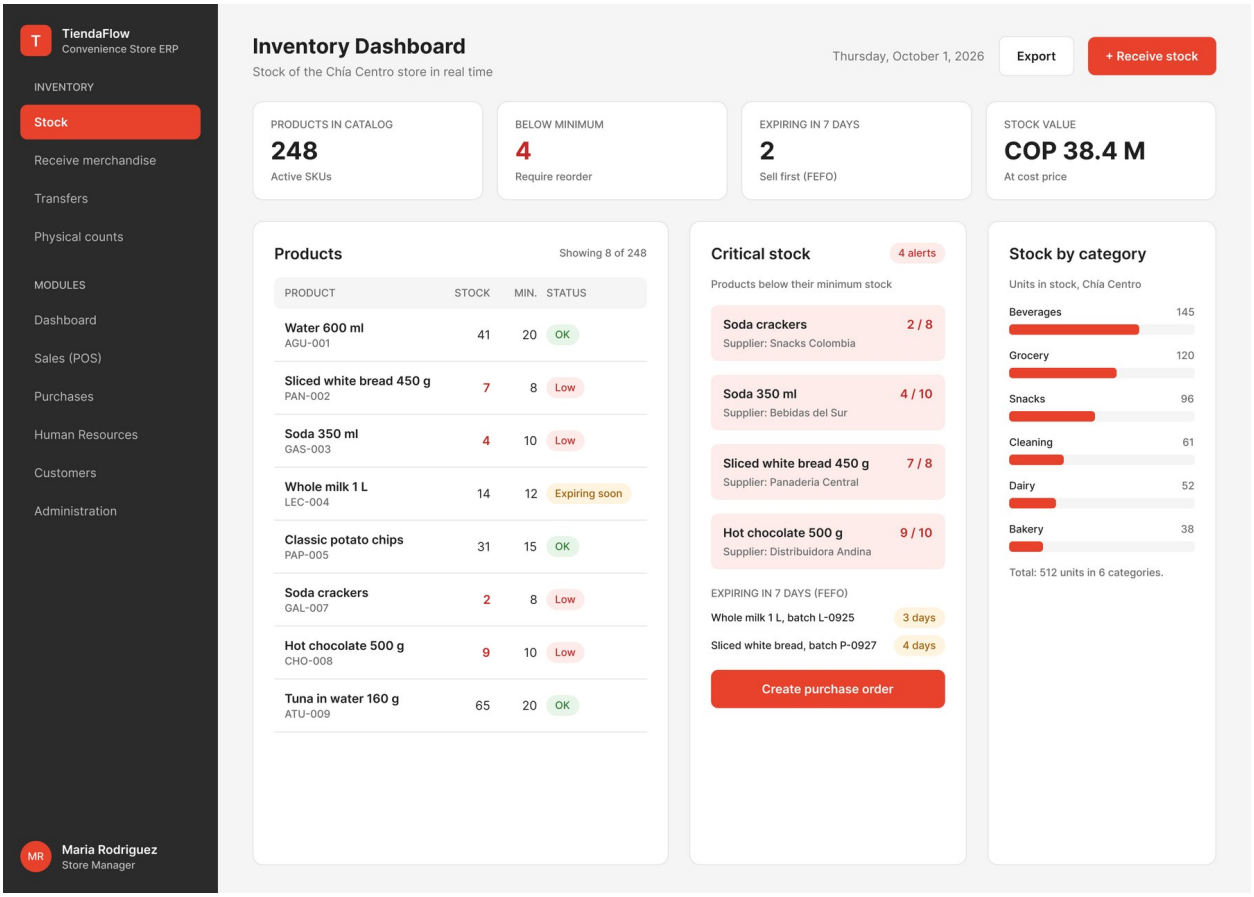
Mockup of the Purchases Dashboard



Note. Created by the authors in Figma with the components of the design system. Data are fictitious.

Figure 24

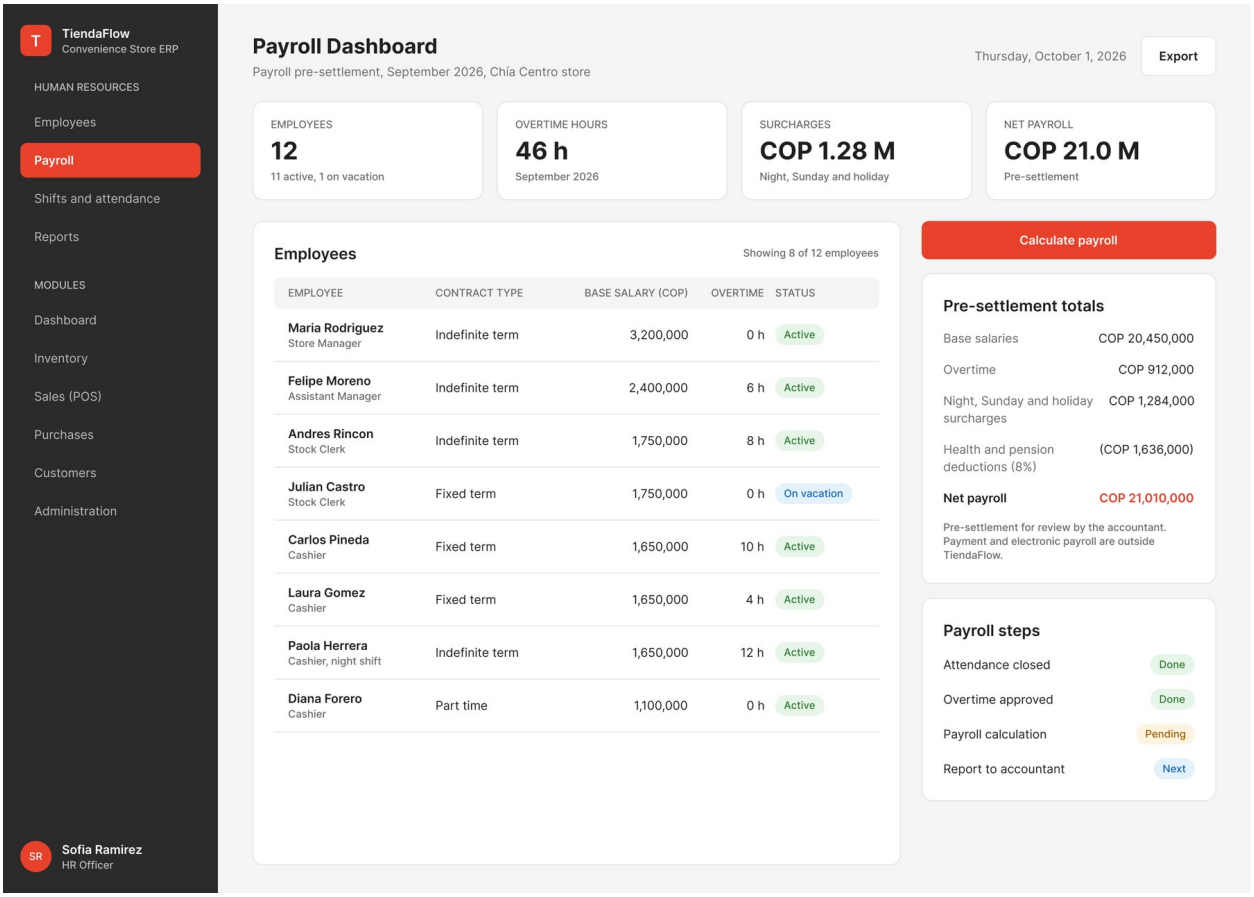
Mockup of the Inventory Dashboard



Note. Created by the authors in Figma with the components of the design system. Data are fictitious.

Figure 25

Mockup of the Human Resources (Payroll) Dashboard



Note. Created by the authors in Figma with the components of the design system. Data are fictitious.

17.3 Interactive Prototype in Figma

The design file was extended into a navigable prototype of the whole system that starts at the login screen. Each option of the side menu opens its screen, and each button opens the screen or dialog of its action; dialogs (forms, confirmations and success messages) open over the current screen and close with their Cancel or Done buttons. Table 45 summarizes the screens of each module and their main interactions, and Figures 26 and 27 show the 31 screens of the prototype.

Table 45

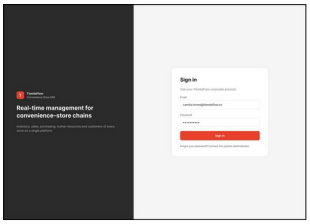
Screens and Interactions of the Interactive Prototype

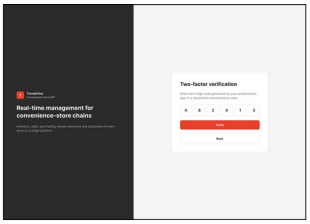
Module	Screens	Main interactions
Access	Login, two-factor verification and corporate dashboard	Sign in opens the verification; Verify opens the dashboard; View reports and Review stock alerts open the reports and the inventory
Inventory	Stock, receive merchandise, stock transfers and physical counts	Receive stock opens the receipt; Confirm receipt, New transfer, Confirm reception, Approve adjustment and Request recount open their dialogs; Create purchase order opens the purchasing form
Sales (POS)	Point of sale, sale completed, sales history and cash register	Charge and issue e-invoice opens the sale completed screen and New sale returns to the POS; Cancel sale, Print receipt, Resend to DIAN, Void sale and Close register open their dialogs
Purchases	Purchase orders, create purchase order, review purchase order, order saved, suppliers and supplier ranking	Create order, Review order and Save order implement the flow of the guide (create the order, see the result and save it); New supplier, Edit supplier and Deactivate supplier open their dialogs
Human resources	Employees, new employee, payroll, payroll calculated, shifts and attendance, and attendance report	New employee and Save employee register an employee; Calculate payroll and Send to accountant complete the pre-settlement; Update contract, Terminate employee and Publish schedule open their dialogs
Customers (CRM)	Customers, loyalty program, campaigns and PQRS	New customer, Update data, Delete personal data, Save rules, New campaign, Edit campaign and Respond open their dialogs
Administration	Consolidated reports, users and roles, audit log and system parameters	New user, Deactivate user and Save parameters open their dialogs
All screens	Side menu and header	The side menu opens every screen; Export opens the export dialog; the user at the bottom of the menu signs out to the login screen

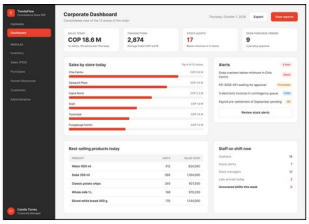
Note. The prototype contains 31 screens and 38 dialogs. It can be tested in the presentation mode of Figma from the file ERP_Mockups_TiendaFlow.

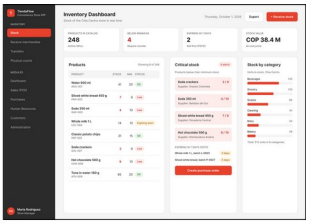
Figure 26

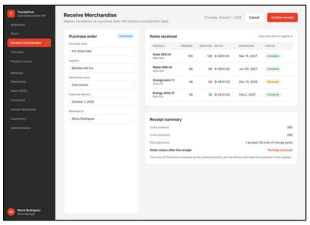
Screens of the Interactive Prototype (1 to 16)

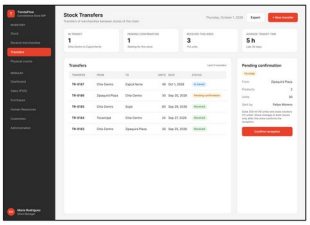

1. Login

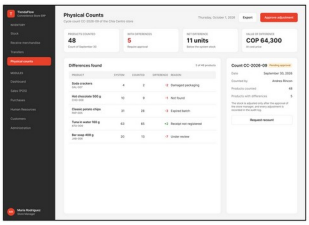

2. Two-factor verification

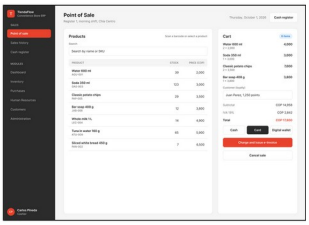

3. Corporate dashboard

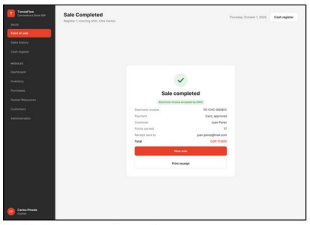

4. Inventory: stock

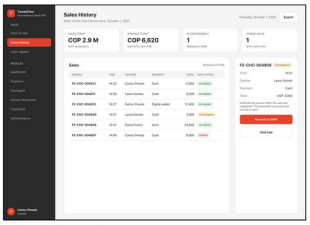

5. Receive merchandise

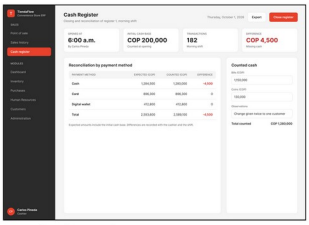

6. Stock transfers

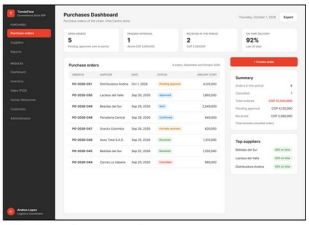

7. Physical counts

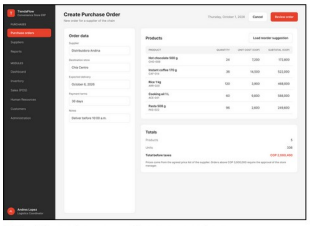

8. Point of sale

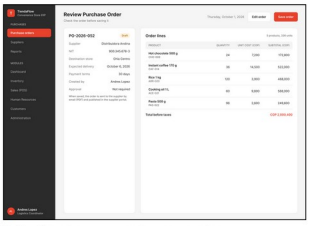

9. Sale completed

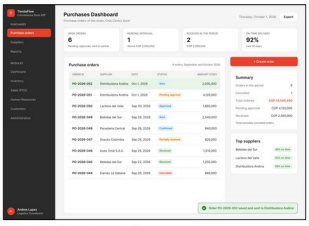

10. Sales history

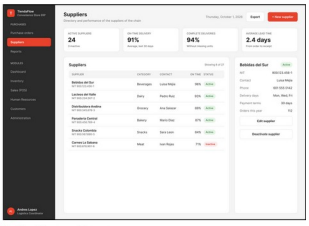

11. Cash register


12. Purchase orders


13. Create purchase order


14. Review purchase order

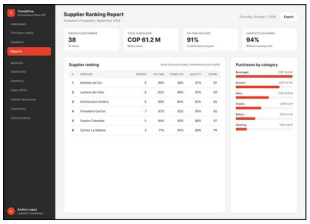

15. Order saved

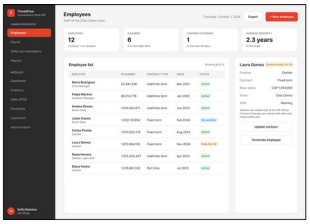

16. Suppliers

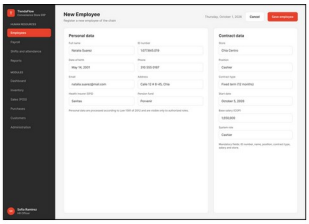
Note. Created by the authors in Figma. Access, inventory, sales and purchasing screens.

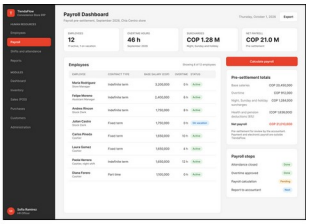
Figure 27

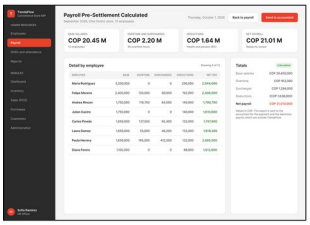
Screens of the Interactive Prototype (17 to 31)

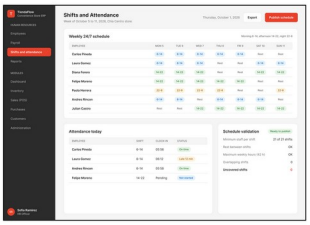

17. Supplier ranking

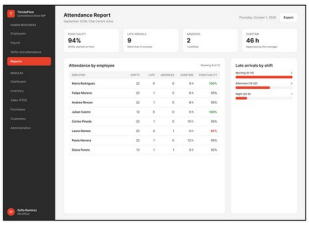

18. Employees

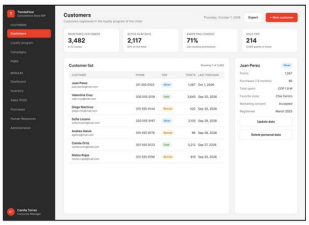

19. New employee

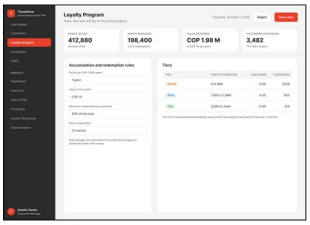

20. Payroll

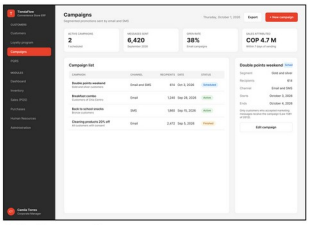

21. Payroll calculated

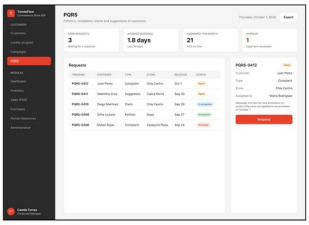

22. Shifts and attendance

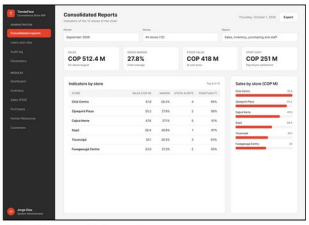

23. Attendance report

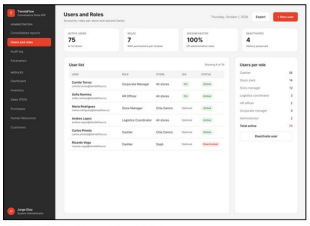

24. Customers

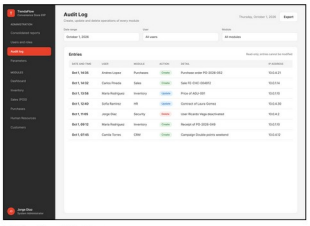

25. Loyalty program

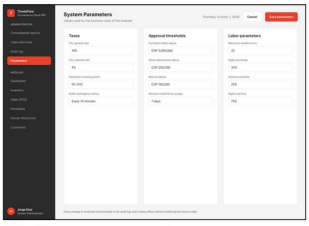

26. Campaigns


27. PQRS


28. Consolidated reports


29. Users and roles


30. Audit log


31. System parameters

Note. Created by the authors in Figma. Purchasing report, human resources, customer management and administration screens.

The flow requested in the guide can be tested as follows:

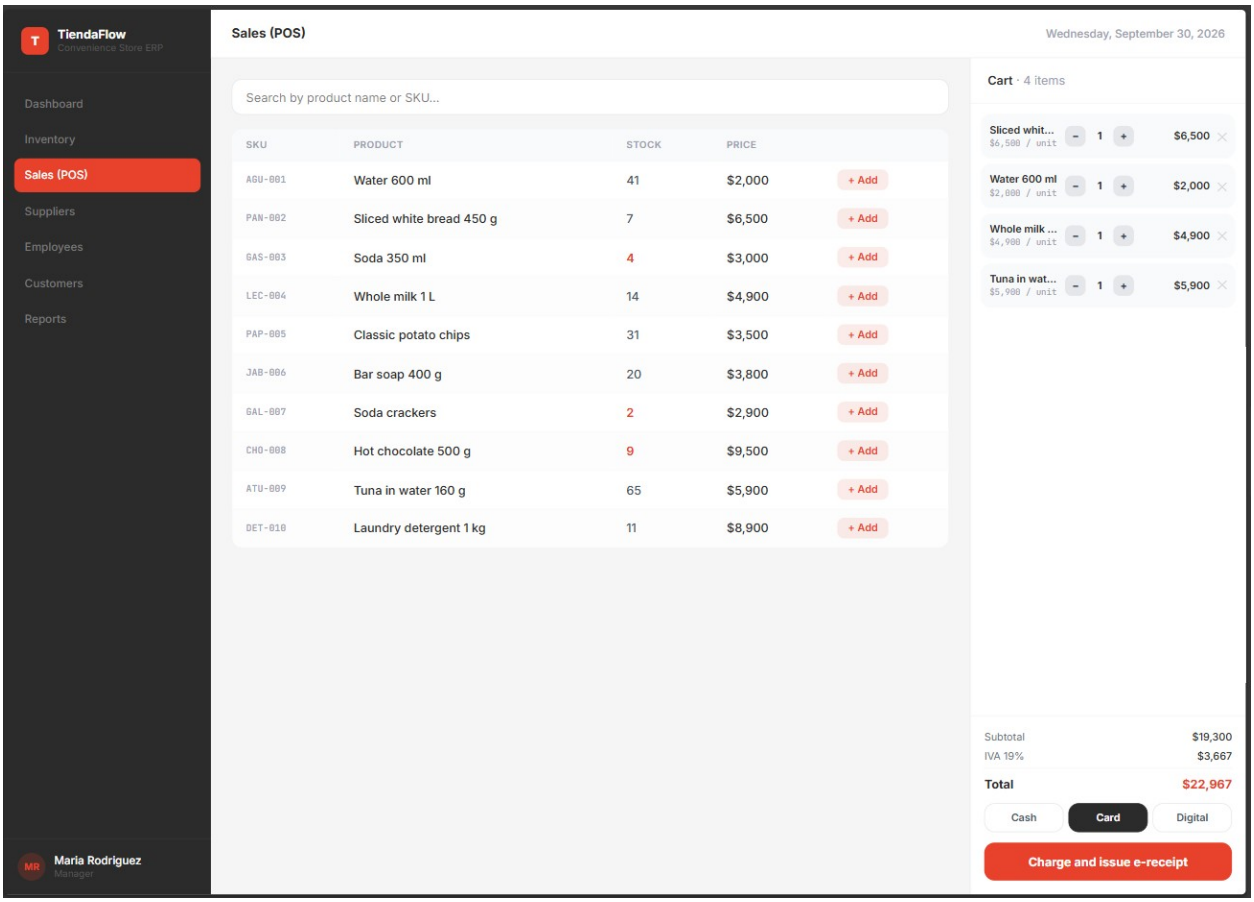
1. Sign in, verify the second factor and open Purchases in the side menu.
2. Press Create order, complete the order of Distribuidora Andina and press Review order.
3. Check the result in the review screen and press Save order.
4. Verify that order PO-2026-052 appears at the top of the purchase orders list with the status Sent and the confirmation message.

17.4 Prototype Generated With Figma Make

Before the design file, the team generated a first functional prototype with Figma Make, which can be tested at https://early-hope-36544320.figma.site/. It includes four working screens (Inventory, Sales (POS), Suppliers and Employees) that share the same product catalog, so that charging a sale decreases the stock and receiving merchandise increases it. Figures 28 and 29 show two of its screens.

Figure 28

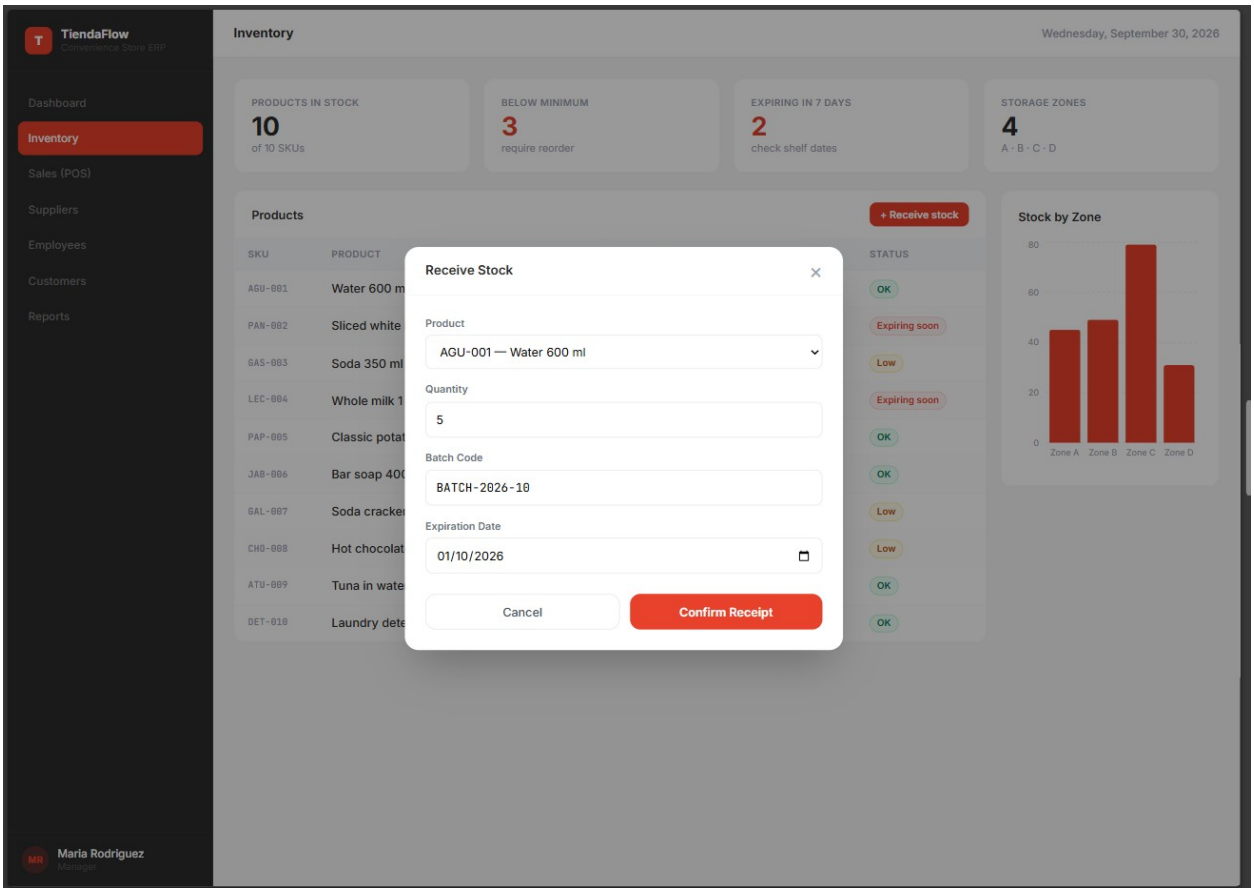
Sales (POS) Screen of the Figma Make Prototype



Note. Screen of the prototype built by the authors in Figma Make. It shows the product catalog, the cart with IVA of 19% and the payment methods.

Figure 29

Receive Stock Form of the Figma Make Prototype



Note. Screen of the prototype built by the authors in Figma Make. The form captures product, quantity, batch code and expiration date.

The prototype was generated from a single detailed prompt that defined the colors, the side menu, the shared product catalog and the behavior of each screen. An excerpt of the prompt is the following:

> Create a desktop app for a convenience store ERP called TiendaFlow. Colors: red-orange #E8412C for primary actions and the active menu item, charcoal #2B2B2B for the sidebar, light gray #F5F5F5 as background. Clicking Inventory, Sales (POS), Suppliers and Employees switches between four working screens. Charging a sale in the POS must

decrease the stock shown in Inventory; receiving stock in Inventory must increase it. Amounts in Colombian pesos, IVA 19%.

18. Reflection Questions

18.1 Why Define Acceptance Criteria Before Developing a User Story?

Acceptance criteria turn an idea into something testable and give the team, the Product Owner and the testers the same definition of done. In TiendaFlow, HU-VEN-02 requires the product and the total to appear in 2 seconds or less and the stock to decrease when the sale is confirmed. Knowing this in advance drives the design of the POS endpoint, the stock transaction and the load test, and avoids discovering late that a fast but inconsistent sale is not acceptable.

18.2 How Does PlantUML Help Keep the Architecture Documentation Up to Date?

PlantUML offers at least three advantages for a project like TiendaFlow:

1. Diagrams are plain text, so they are versioned in Git and reviewed in pull requests like the code.
2. Changes are cheap: editing one line regenerates the whole diagram with a consistent style, so the documentation is not abandoned.
3. It integrates with Visual Studio Code, Google Colab, GitHub Pages and continuous integration, which allows the images to be regenerated automatically on every commit.

18.3 What Advantages Does Figma Make Offer for Rapid ERP Prototyping?

Figma Make generates interactive screens from natural-language prompts, which lets the team validate flows such as the POS checkout with users in minutes instead of days. It complements the designer rather than replacing the designer: the artificial intelligence produces a first draft, and the designer keeps control of the design system, the accessibility and the

consistency of the screens, while Make Kits make the prototype use the real components, colors and typography of the product.

19. Documentation and Information Storage

All the information developed in this document is published and kept up to date on the documentation page of the project (Section 4), together with the PlantUML sources of the diagrams, the Google Colab notebook that renders them (Section 4), the links to the Figma design file (design system, mockups and interactive prototype) and to the Figma Make prototype, and the English presentation that summarizes each sprint. The source code of the class, entity-relationship and C4 models is included in Appendix A.

References

Becerra, D. (2026, July 16). *Alegra vs Siigo en Colombia 2026: Precios, IA, opiniones y cuál software contable elegir*. Alegra. https://blog.alegra.com/colombia/alegra-vs-siigo-colombia/

Brown, S. (n.d.). *The C4 model for visualising software architecture*. Retrieved October 1, 2026, from https://c4model.com/

Caneva, G. (2026). *Sueldo de un desarrollador de aplicaciones en Colombia en 2026*. Coderhouse. https://www.coderhouse.com/sueldos/sueldo-desarrollador-aplicaciones-colombia-2025

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Congreso de la República de Colombia. (2012). *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial.

Costbench. (2026). *SAP Business One pricing 2026*. https://www.costbench.com/software/erp/sap-business-one/

Dirección de Impuestos y Aduanas Nacionales. (2023). *Resolución 000165 de 2023*. DIAN.

ERP Research. (2026a). *Odoo pricing*. https://www.erpresearch.com/pricing/odoo

ERP Research. (2026b). *SAP Business One pricing breakdown*. https://erpresearch.com/erp/sap-business-one

Hyperbots. (2026). *SAP Business One pricing explained: Licensing, cloud costs & hidden fees in 2026*. https://blog.hyperbots.com/sap-business-one-pricing-explained-licensing-cloud-costs-hidden-fees-in-2026

IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers.

ISO/IEC. (2011). *Systems and software engineering: Systems and software quality requirements and evaluation (SQuaRE): System and software quality models* (ISO/IEC 25010:2011). International Organization for Standardization.

Kruchten, P. B. (1995). Architectural blueprints: The "4+1" view model of software architecture. *IEEE Software, 12*(6), 42-50. https://doi.org/10.1109/52.469759

Microsoft. (n.d.). *SQL Server 2022 pricing*. Retrieved September 23, 2026, from https://www.microsoft.com/sql-server/sql-server-2022-pricing

Mujica Reyes, C. E. (2026a). *Guía de laboratorio 1: Arquitectura de software y atributos de calidad* [Course material]. Universidad de Cundinamarca.

Mujica Reyes, C. E. (2026b). *Taller interactivo de diseño y especificación de un ERP genérico* [Course material]. Universidad de Cundinamarca.

NerdWallet. (2026). *Square fees: Calculator and pricing for 2026*. https://www.nerdwallet.com/business/software/learn/square-fees

Portafolio. (2026, September 15). *El dólar bajó levemente en Colombia: así cotiza la divisa este 15 de septiembre*. https://www.portafolio.co/economia/finanzas/precio-del-dolar-hoy-en-colombia-asi-cotiza-la-divisa-este-15-de-septiembre-502515

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK guide)* (7th ed.). Project Management Institute.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

Sommerville, I. (2011). *Software engineering* (9th ed.). Addison-Wesley.

Swell. (2026). *Odoo pricing in 2026: How much does Odoo ecommerce really cost*. https://www.swell.is/content/odoo-pricing

TrustedTech. (n.d.). *Microsoft 2022 on-premise products*. Retrieved September 23, 2026, from https://www.trustedtechteam.com/collections/2022

UpMenu. (2026). *Square pricing breakdown: Fees & hidden costs (2026)*. https://www.upmenu.com/blog/square-pricing

World Wide Web Consortium. (2018). *Web content accessibility guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

Appendix A: PlantUML Source Code of the Architecture and Data Models

This appendix contains the PlantUML source code of the models of Section 16. The sources of all the other diagrams are stored in the docs/uml folder of the repository and in the Google Colab notebook of the project.

A.1 Class Diagram of the Inventory Module

```
@startuml
skinparam monochrome true
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontSize 15
skinparam classAttributeIconSize 0
skinparam classBackgroundColor white
skinparam nodesep 40
skinparam ranksep 50
hide empty members

class Category {
  - id: int
  - name: string
}
class Product {
  - id: int
  - sku: string
  - barcode: string
  - name: string
  - unit: string
  - cost: decimal
  - salePrice: decimal
  - taxRate: decimal
  - active: bool
  + register(): void
  + update(): void
  + deactivate(): void
}
class Store <<warehouse>> {
  - id: int
  - name: string
  - address: string
  - active: bool
  + getStock(product: Product): StockItem
}
class StockItem {
  - quantityAvailable: int
  - minStock: int
  - maxStock: int
  - location: string
  + increase(quantity: int): void
  + decrease(quantity: int): void
  + isBelowMinimum(): bool
  + updateLocation(location: string): void
}
class Batch {
  - id: int
  - batchCode: string
  - expirationDate: Date
  - quantity: int
  + isExpired(): bool
```

```
  + daysToExpire(): int
}
class InventoryMovement {
  - id: int
  - type: MovementType
  - quantity: int
  - date: DateTime
  - userId: int
  - reason: string
  + register(): void
}
enum MovementType {
  RECEIPT
  SALE
  RETURN
  TRANSFER_OUT
  TRANSFER_IN
  ADJUSTMENT
  SHRINKAGE
}
class StockTransfer {
  - id: int
  - status: string
  - date: DateTime
  + send(): void
  + confirmReception(): void
}
class PhysicalCount {
  - id: int
  - date: DateTime
  - status: string
  + compare(): void
  + approve(managerId: int): void
}
class StockAlert {
  - id: int
  - type: AlertType
  - date: DateTime
  - resolved: bool
  + notify(): void
}
enum AlertType {
  MINIMUM_STOCK
  EXPIRATION
}

Category "1" -- "*" Product : classifies
Store "1" *-- "*" StockItem : holds
Product "1" -- "*" StockItem : is stocked as
StockItem "1" *-- "*" Batch : contains
StockItem "1" -- "*" InventoryMovement : records
InventoryMovement --> MovementType
Store "1" -- "*" StockTransfer : origin / destination
StockTransfer "1" -- "2..*" InventoryMovement : generates
Store "1" -- "*" PhysicalCount : performs
PhysicalCount "1" -- "*" InventoryMovement : adjusts with
StockItem "1" -- "*" StockAlert : triggers
StockAlert --> AlertType
@enduml
```

A.2 Entity-Relationship Diagram

```
@startuml
skinparam monochrome true
skinparam shadowing false
```

```
skinparam dpi 150
skinparam defaultFontSize 14
skinparam linetype ortho
skinparam entityBackgroundColor white
skinparam classBackgroundColor white
skinparam nodesep 30
skinparam ranksep 40
hide circle

entity "store" as store {
  * id : int <<PK>>
  --
  name : varchar
  address : varchar
  active : bit
}
entity "category" as category {
  * id : int <<PK>>
  --
  name : varchar
}
entity "product" as product {
  * id : int <<PK>>
  --
  category_id : int <<FK>>
  sku : varchar <<UK>>
  barcode : varchar <<UK>>
  name : varchar
  sale_price : decimal
  tax_rate : decimal
  active : bit
}
entity "stock" as stock {
  * store_id : int <<PK, FK>>
  * product_id : int <<PK, FK>>
  --
  quantity : int
  min_stock : int
  max_stock : int
  location : varchar
}
entity "batch" as batch {
  * id : int <<PK>>
  --
  store_id : int <<FK>>
  product_id : int <<FK>>
  batch_code : varchar
  expiration_date : date
  quantity : int
}
entity "inventory_movement" as mov {
  * id : int <<PK>>
  --
  store_id : int <<FK>>
  product_id : int <<FK>>
  user_id : int <<FK>>
  type : varchar
  quantity : int
  moved_at : datetime
  reason : varchar
}
entity "supplier" as supplier {
  * id : int <<PK>>
  --
  nit : varchar <<UK>>
  business_name : varchar
  address : varchar
```

```
  rating : decimal
  active : bit
}
entity "purchase_order" as po {
  * id : int <<PK>>
  --
  supplier_id : int <<FK>>
  store_id : int <<FK>>
  order_date : date
  status : varchar
  total : decimal
}
entity "purchase_order_line" as pol {
  * order_id : int <<PK, FK>>
  * product_id : int <<PK, FK>>
  --
  quantity : int
  unit_cost : decimal
  received_qty : int
}
entity "employee" as emp {
  * id : int <<PK>>
  --
  store_id : int <<FK>>
  id_number : varchar <<UK>>
  name : varchar
  position : varchar
  contract_type : varchar
  salary : decimal
  active : bit
}
entity "shift" as shift {
  * id : int <<PK>>
  --
  employee_id : int <<FK>>
  start_time : datetime
  end_time : datetime
  clock_in : datetime
  clock_out : datetime
}
entity "customer" as cust {
  * id : int <<PK>>
  --
  id_number : varchar <<UK>>
  name : varchar
  email : varchar
  consent_date : datetime
  points : int
  tier : varchar
}
entity "sale" as sale {
  * id : int <<PK>>
  --
  store_id : int <<FK>>
  cashier_id : int <<FK>>
  customer_id : int <<FK>>
  sale_date : datetime
  total : decimal
  status : varchar
}
entity "sale_line" as sl {
  * sale_id : int <<PK, FK>>
  * product_id : int <<PK, FK>>
  --
  quantity : int
  unit_price : decimal
  discount : decimal
```

```
}
entity "payment" as pay {
  * id : int <<PK>>
  --
  sale_id : int <<FK>>
  method : varchar
  amount : decimal
}
entity "electronic_document" as edoc {
  * id : int <<PK>>
  --
  sale_id : int <<FK>>
  doc_type : varchar
  cufe : varchar <<UK>>
  dian_status : varchar
}
entity "app_user" as usr {
  * id : int <<PK>>
  --
  employee_id : int <<FK>>
  role_id : int <<FK>>
  email : varchar <<UK>>
  password_hash : varchar
  two_factor : bit
  active : bit
}
entity "role" as role {
  * id : int <<PK>>
  --
  name : varchar
}
entity "audit_log" as audit {
  * id : bigint <<PK>>
  --
  user_id : int <<FK>>
  action : varchar
  module : varchar
  ip : varchar
  logged_at : datetime
}

category ||--o{ product
store ||--o{ stock
product ||--o{ stock
stock ||--o{ batch
stock ||--o{ mov
supplier ||--o{ po
store ||--o{ po
po ||--|{ pol
product ||--o{ pol
store ||--o{ emp
emp ||--o{ shift
store ||--o{ sale
emp ||--o{ sale
cust |o--o{ sale
sale ||--|{ sl
product ||--o{ sl
sale ||--|{ pay
sale ||--o{ edoc
emp |o--o| usr
role ||--o{ usr
usr ||--o{ audit
usr ||--o{ mov
@enduml
```

A.3 C4 Context Diagram

```
@startuml
!include <C4/C4_Context>
UpdateElementStyle("person", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("system", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("external_system", $bgColor="white", $fontColor="black",
$borderColor="black", $borderStyle=DashedLine())
UpdateElementStyle("container", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("external_container", $bgColor="white", $fontColor="black",
$borderColor="black", $borderStyle=DashedLine())
UpdateBoundaryStyle($bgColor="white", $fontColor="black", $borderColor="black")
UpdateRelStyle($textColor="black", $lineColor="black")
skinparam dpi 150
skinparam defaultFontSize 15
skinparam shadowing false
LAYOUT_TOP_DOWN()

Person(staff, "Store staff", "Cashiers, stock clerks and employees of each store")
Person(mgr, "Store Manager", "Supervises stock, sales, staff and approvals")
Person(corp, "Corporate staff", "Logistics coordinator, HR officer, corporate manager and
administrator")
Person(cus, "Customer", "Loyalty program, invoices and PQRS")
Person(sup, "Supplier", "Consults and confirms purchase orders in the portal")

System(erp, "TiendaFlow ERP", "Inventory, sales and billing, purchasing, HR, CRM, security
and reporting for a convenience-store chain")

System_Ext(dian, "DIAN electronic invoicing", "Validates electronic invoices, POS documents
and credit notes")
System_Ext(pay, "Payment gateway", "Authorizes card and digital wallet payments")
System_Ext(notif, "Email and SMS service", "Delivers invoices, alerts and promotions")

Rel(staff, erp, "Registers sales, receipts and attendance", "HTTPS")
Rel(mgr, erp, "Manages the store", "HTTPS")
Rel(corp, erp, "Manages purchasing, payroll, campaigns and users", "HTTPS")
Rel(sup, erp, "Confirms purchase orders", "HTTPS portal")
Rel(erp, cus, "Sends invoices and promotions", "Email / SMS")
Rel(erp, dian, "Sends electronic documents", "Web service")
Rel(erp, pay, "Requests payment authorization", "REST API")
Rel(erp, notif, "Sends messages", "REST API")
@enduml
```

A.4 C4 Container Diagram

```
@startuml
!include <C4/C4_Container>
UpdateElementStyle("person", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("system", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("external_system", $bgColor="white", $fontColor="black",
$borderColor="black", $borderStyle=DashedLine())
UpdateElementStyle("container", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("external_container", $bgColor="white", $fontColor="black",
$borderColor="black", $borderStyle=DashedLine())
UpdateBoundaryStyle($bgColor="white", $fontColor="black", $borderColor="black")
UpdateRelStyle($textColor="black", $lineColor="black")
skinparam dpi 150
skinparam defaultFontSize 15
skinparam shadowing false
LAYOUT_TOP_DOWN()

Person(cashier, "Cashier", "Registers sales at the checkout")
Person(staff, "Back-office users", "Managers, logistics, HR, corporate and administrator")
Person(sup, "Supplier", "External user of the portal")
```

```
System_Boundary(tf, "TiendaFlow ERP") {
  Container(pos, "POS client", "Blazor WebAssembly (PWA)", "Fast checkout; keeps an offline
queue of cash sales")
  Container(back, "Back-office web app", "Blazor Server", "Inventory, purchasing, HR, CRM,
reports and administration")
  Container(portal, "Supplier portal", "Blazor Server", "Order consultation and
confirmation")
  Container(api, "TiendaFlow API", "ASP.NET Core Web API (C#)", "Business logic of the six
modules; REST and JSON; OpenAPI")
  Container(worker, "Background worker", "ASP.NET Core Worker Service", "DIAN queue, stock
and expiration alerts, notifications")
  ContainerDb(db, "TiendaFlow database", "SQL Server 2022", "Transactional data and
immutable audit log")
}

System_Ext(dian, "DIAN electronic invoicing", "Document validation")
System_Ext(pay, "Payment gateway", "Card and wallet payments")
System_Ext(notif, "Email and SMS service", "Message delivery")

Rel(cashier, pos, "Uses", "Browser")
Rel(staff, back, "Uses", "HTTPS")
Rel(sup, portal, "Uses", "HTTPS")
Rel(pos, api, "Sends sales and synchronizes", "HTTPS / JSON")
Rel(back, api, "Calls", "HTTPS / JSON")
Rel(portal, api, "Calls", "HTTPS / JSON")
Rel(api, db, "Reads and writes", "EF Core / TDS")
Rel(worker, db, "Reads queues and writes results", "EF Core / TDS")
Rel(api, pay, "Authorizes payments", "REST")
Rel(worker, dian, "Sends electronic documents", "Web service")
Rel(worker, notif, "Sends invoices, alerts and promotions", "REST")
@enduml
```

A.5 C4 Component Diagram

```
@startuml
!include <C4/C4_Component>
UpdateElementStyle("container", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("component", $bgColor="white", $fontColor="black", $borderColor="black")
UpdateElementStyle("external_system", $bgColor="white", $fontColor="black",
$borderColor="black", $borderStyle=DashedLine())
UpdateBoundaryStyle($bgColor="white", $fontColor="black", $borderColor="black")
UpdateRelStyle($textColor="black", $lineColor="black")
skinparam dpi 150
skinparam defaultFontSize 16
skinparam shadowing false
LAYOUT_TOP_DOWN()

Container(clients, "Web clients", "Blazor", "POS, back office and supplier portal")

Container_Boundary(api, "TiendaFlow API (ASP.NET Core Web API)") {
  Component(audit, "Audit logger", "Middleware", "Logs create, update and delete
operations")
  Component(ctrl, "REST controllers", "ASP.NET Core", "Endpoints of the six modules
(OpenAPI)")
  Component(auth, "Authentication", "ASP.NET Core Identity", "Login, second factor and
roles")
  Component(inv, "Inventory service", "C#", "Stock, receipts and counts")
  Component(ven, "Sales and Billing service", "C#", "Sales, payments and DIAN documents")
  Component(sup, "Purchasing service", "C#", "Suppliers and purchase orders")
  Component(rh, "HR service", "C#", "Employees, shifts and payroll")
  Component(crm, "CRM service", "C#", "Customers, points and PQRS")
  Component(rep, "Reporting service", "C#", "Consolidated reports")
  Component(data, "Data access layer", "Entity Framework Core", "DbContext and
repositories")
```

```
  Component(pay, "Payment client", "HttpClient", "Payment gateway calls")
}

ContainerDb(db, "TiendaFlow database", "SQL Server 2022", "Data and audit log")
Container(worker, "Background worker", "Worker Service", "DIAN queue and alerts")
System_Ext(gw, "Payment gateway", "Card and wallet payments")

Rel(clients, ctrl, "Calls", "HTTPS / JSON")
Rel_R(ctrl, auth, "Validates")
Rel_L(ctrl, audit, "Logs")
Rel(ctrl, inv, "")
Rel(ctrl, ven, "")
Rel(ctrl, sup, "")
Rel(ctrl, rh, "")
Rel(ctrl, crm, "")
Rel(ctrl, rep, "")
Rel(inv, data, "")
Rel(ven, data, "")
Rel(sup, data, "")
Rel(rh, data, "")
Rel(crm, data, "")
Rel(rep, data, "")
Rel(ven, pay, "")
Rel(pay, gw, "REST")
Rel(data, db, "Reads and writes")
Rel(worker, db, "Reads queues")
@enduml
```